

BIG DATA UNICT

SANTO LO BIANCO

L-31

L'era dei dati

Oggi viviamo in un periodo in cui vengono prodotti enormi quantità di dati ogni secondo.

I **big data** sono **insiemi di dati così grandi, complessi e generati rapidamente da non poter essere gestiti efficacemente con le tecnologie tradizionali** di archiviazione ed elaborazione.

Le principali caratteristiche dei big data sono:

- **Dati prodotti velocemente** → i dati vengono generati in continuazione da smartphone, sensori, telecamere, siti web, transazioni online...;
- **Eterogenei** → i dati possono avere forme molto diverse: testo, immagini, video, audio, file di log dei server...;
- **Potenzialmente ricchi di informazioni** → se analizzati correttamente, possono fornire conoscenze utili per prendere decisioni, fare previsioni o individuare comportamenti nascosti.

I **big data** vengono spesso descritti attraverso le **5V**:

- 1) **Volume** → quantità di dati → più il volume cresce, più è difficile memorizzare ed elaborare i dati;
- 2) **Velocità** → è la rapidità con cui i dati vengono generati e devono essere elaborati → in molti casi è necessario analizzarli quasi istantaneamente;
- 3) **Varietà** → indica la diversità dei tipi di dati → dati strutturati (TABELLE SQL), semi strutturati(JSON, XML) e non strutturati(foto, video);
- 4) **Veridicità** → riguarda la qualità e l'affidabilità dei dati → i dati possono essere incompleti, duplicati, rumorosi, errati → prima di analizzarli si dovrebbero pulire;
- 5) **Valore** → i dati hanno valore solo se producono informazioni utili → l'utilizzatore deve poter trovare senso nei dati raccolti o nell'elaborazione di essi;

I Big Data non riguardano soltanto le grandi quantità di dati, ma coinvolgono un'intera catena di processi e tecnologie.

L'**acquisizione dei dati** è la fase in cui i dati vengono raccolti dalle varie sorgenti come: social network, sensori e dispositivi IoT, smartphone, ecc. . L'obiettivo è acquisire grandi quantità di dati in modo rapido e continuo.

Una volta acquisiti, i dati devono essere memorizzati in **infrastrutture di stoccaggio** e poiché la mole da immagazzinare è enorme, si utilizzano: sistemi distribuiti, cloud storage, file system distribuiti.

L'infrastruttura di stoccaggio deve garantire elevata capacità, affidabilità e disponibilità ad espandersi facilmente.

I Big Data possono contenere informazioni sensibili per questo motivo deve essere garantire:

- **sicurezza** → proteggendo i dati da accessi non autorizzati;
- **privacy** → assicurando che l'utilizzo sia nel rispetto delle normative.

I dati devono essere organizzati in sistemi che ne permettano una gestione efficiente, ossia i database. Si distinguono principalmente in:

- **database relazionali(SQL)** → adatti a dati strutturati ossia dati dove c'è organizzazione tabellare e schema fisso;
- **database non relazionali (NoSQL)** → progettati per gestire grandi volumi di dati eterogenei e distribuiti.

Nei big data solitamente si preferisce NoSQL perché sono più flessibili e scalabili.

L'**Analytics** e il **Data Mining** questa fase consiste nell'analizzare i dati per estrarre conoscenza. Le tecniche che entrano in gioco sono: analisi statistica, machine learning, individuazione di pattern e correlazioni, previsione di eventi futuri.

Quindi, l'obiettivo principale è trasformare i dati grezzi in informazioni utili per prendere decisioni.

L'elaborazione dei big data richiede una grande potenza di calcolo, per questo è importante anche l'**infrastruttura computazionale**. Si utilizzano:

- cluster di computer;
- calcolo distribuito;
- elaborazione parallela;

In questo modo il lavoro viene suddiviso tra molte macchine che collaborano tra loro.

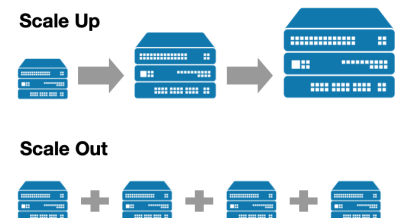
Infine, dopo l'elaborazione dei dati i risultati devono essere **presentati** in modo comprensibile. Si utilizzano: grafici, dashboard, mappe, diagrammi e report interattivi. Una buona visualizzazione permette di interpretare rapidamente grandi quantità di informazioni.

SCALE-UP/ SCALE-OUT

Quando si progettano sistemi informatici moderni, soprattutto nel contesto dei Big Data e dei servizi cloud, uno dei problemi fondamentali è capire come aumentare le prestazioni quando la quantità di utenti o dati cresce. Il problema di cui stiamo parlando si chiama scalabilità.

In generale, esistono due strategie principali per rendere un sistema più potente:

- **scale-up(scalabilità verticale)** → potenziare una singola macchina;
- **scale-out(scalabilità orizzontale)** → aggiungere più macchine che lavorano insieme.



Queste due strategie rappresentano due modi completamente diversi di affrontare la crescita di un sistema.

La **scalabilità verticale** consiste nel **rendere più potente un singolo server**, aumentano le sue risorse hardware:

- CPU più veloci o più numerose;
- maggiore quantità di RAM;
- dischi più performanti.

Si potenzia una sola macchina invece di aggiungerne altre.

I vantaggi sono legati ai consumi energetici/di raffreddamento e la maggiore semplicità architetture.

Gli svantaggi sono: costo elevato, single point of failure ossia che si ha un unico punto di rottura, limiti fisici nell'aumentare e la scalabilità limitata.

La **scalabilità orizzontale** consiste nell'**aumentare la potenza del sistema aggiungendo più macchine(nodi)**, che lavorano insieme come un unico sistema distribuito. Invece di “fare una macchina più potente”, si aggiungono più macchine più semplici.

I vantaggi sono dati da: costo più basso, tolleranza ai guasti e scalabilità elevata.

I due svantaggi principali sono: costi energetici più alti e maggiore complessità del sistema.

CALCOLO SU LARGA SCALA

Il calcolo su larga scala è l'insieme di tecniche e infrastrutture progettate per:

- gestire enormi quantità di dati;
- eseguire calcoli estremamente complessi.

In entrambi i casi si utilizzano: **cluster di macchine**, **elaborazione parallela** e **reti ad alta prestazione**.

La differenza però sta nel tipo di problema da risolvere, difatti **HPC** risolve problemi computazionalmente intensivi; invece, il calcolo **Big data** lavora con una mole di dati molto estesa.

HIGH PERFORMANCE COMPUTING

L'**High Performance Computing**(HPC) è un paradigma di **calcolo orientato alla massima potenza computazionale per risolvere problemi complessi**.

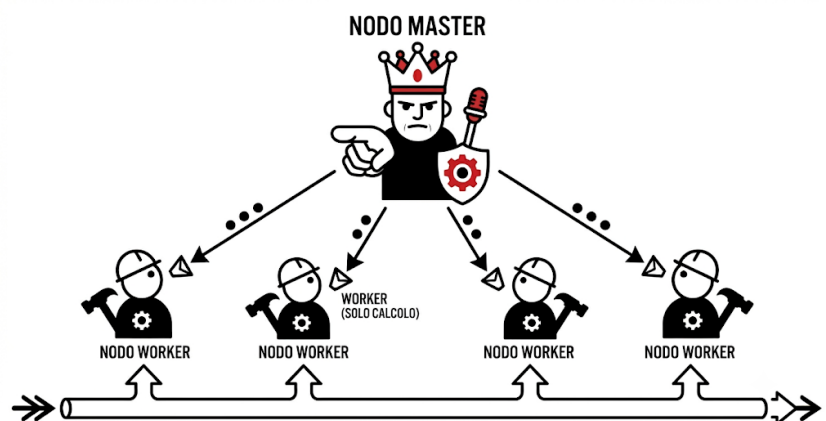
Un sistema HPC è tipicamente un cluster di **nodi indipendenti collegati** tra loro mediante **rete ad alta velocità**. C'è la presenza di uno storage centralizzato anche se ogni nodo possiede: CPU multicore, RAM locale e un disco locale che permettono di lavorare in modo autonomo.

Ogni nodo accede solo alla propria RAM e comunica utilizzando la rete e i protocolli di comunicazione come Message Passing Interface.

La fault tolerance è bassa dato che se un nodo fallisce il calcolo può compromettersi. Esempi di HPC sono: simulazioni meteorologiche, fisica quantistica o modellazione molecolare.

I nodi del sistema possono essere di tipo:

- **nodo master** → è il regista del sistema che decide come dividere il lavoro e assegna i compiti ai worker. Inoltre, gestisce tutta la comunicazione e infine raccoglie i risultati.
- **nodo worker** → è il lavoratore ossia: riceve un pezzo del problema, risolve i calcoli e restituisce il risultato al master.



Ma come comunicano i nodi? Qui entra in gioco il concetto di **MPI**(**Message Passing Interface**), citato prima, che è uno **standard usato nei sistemi distribuiti per far comunicare i processi**.

In pratica:

- un processo può inviare messaggi ad altri processi;
- un processo può ricevere messaggi da altri processi.

Tutto avviene in modo esplicito, non condividendo la memoria ma appunto scambiando i messaggi tra processi.

BIG DATA

Il **Big Data Computing** è invece orientato alla **gestione e analisi di grandi volumi di dati eterogenei**. L'obiettivo principale è calcolare ed elaborare enormi dataset distribuiti.

I dati sono distribuiti su molti nodi (architettura scale-out) e l'elaborazione avviene parallelamente. La memoria è distribuita e ogni nodo lavora su un pezzo del dataset.

I dati sono divisi in blocchi e ogni nodo contiene sia storage che capacità di calcolo.

L'idea fondamentale dell'architettura Big Data è questa: *sposto il calcolo vicino ai dati, non i dati verso il calcolo*. Quindi, eseguo il task sul nodo dove i dati sono già presenti → **data locality**.

I blocchi sono replicati per fault tolerance, permettendo il funzionamento anche se uno dei nodi smette di funzionare.

Le operazioni tipiche applicate a grandi quantità di dati sono:

- **filter** → seleziona solo i dati che rispettano una condizione;
- **map** → trasforma ogni elemento indipendentemente dagli altri;
- **reduce** → aggrega tutti gli elementi in un unico risultato;
- **join** → unisce due dataset usando una chiave comune.

Il collo di bottiglia spesso non è la CPU, ma il disco a causa dei continui I/O.

La rete in questo caso è usata il meno possibile e si evita di spostare grandi dataset. Si preferisce scambiare solo risultati intermedi.

MAP-REDUCE

Introduzione all'elaborazione dei Big Data

Uno dei modelli di programmazione più importanti per l'elaborazione dei Big Data è il modello **MapReduce**. Questo modello è stato ideato da Google tra il 2003 e il 2004, dato che la stessa azienda doveva trovar un modo per elaborare grandi quantità di dati contenuti in migliaia di computer.

Molti problemi che sono concettualmente semplici diventano enormemente costosi quando i dati sono giganteschi → algoritmi semplici ma la quantità di dati è elevata(terabyte, petabyte).

Dato che Google aveva sviluppato questo software proprietario non disponibile al pubblico, Apache e Yahoo svilupparono un progetto open-source ispirato a MapReduce, cioè **Hadoop**.

Oggi Hadoop e il paradigma MapReduce sono usati come piattaforme general purpose per:

- memorizzare grandi quantità;
- elaborare big data;
- eseguire analisi distribuite;
- fare data mining;

Anche se oggi esistono tecnologie più moderne, MapReduce rimane il modello storico che ha dato origine all'elaborazione distribuita dei Big Data.

Il modello preso in questione nasconde tutta la complessità della programmazione distribuita, questo agevola la programmazione per sviluppatori senza alcuna esperienza.

Altrimenti, dovremmo:

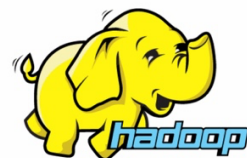
- distribuire i dati nei vari nodi;
- coordinare i processi;
- gestire la comunicazione tra macchine;
- sincronizzare i risultati;
- attenzionare i guasti.

Hadoop citato prima è **una piattaforma open source sviluppato dalla comunità Apache**.

Nasconde gran parte della programmazione distribuita, permettendo al programmatore di scrivere funzioni map e reduce.

Hadoop è composto da due sottocomponenti principali:

- **MapReduce Runtime** → motore di elaborazione il cui compito è:
 - dividere il lavoro e assegnarlo ai vari server;
 - monitorare l'esecuzione;
 - raccogliere i risultati finali e li combina.
- **Hadoop File Systems** → **file system distribuito** → singoli file vengono spezzati in blocchi e divisi in molti server → percezione di avere un unico grande disco.



I server di Hadoop possono guastarsi, ma non è un problema dato che è prevista una **fault tolerance**. Infatti, se si dovesse guastare un nodo: il coordinatore assegna automaticamente quel task ad un altro server.

Per evitare la perdita di dati, **HDFS mantiene più copie dello stesso blocco**.

Nel paradigma tradizionale si utilizza un solo nodo composto da: CPU, RAM e disco. Se servono più prestazioni si aumenta la potenza della macchina(scale-up) e per dataset piccoli o medi funziona molto bene.

Con grandi dataset, anche se si aumentano le prestazioni il limite è il disco. Infatti, supponendo che la capienza sia sufficiente a contenere i dati da elaborare, le **letture sequenziali sono il bottle neck del sistema**.

La conclusione è che si è dovuto cercare un nuovo modo per fare calcolo su larga scala. Quindi, non si potenzia un singolo computer bensì si costruisce un sistema distribuito.

Con **Large Scale Computing** si intende l'**elaborazione di grandissime quantità di dati** utilizzando molti computer che lavorano insieme.

Le due sfide importanti sono:

- Come si distribuisce il calcolo? Si deve decidere la parte di lavoro da assegnare ad ogni computer, i dati e infine si devono raccogliere i risultati finali.
- Programmazione parallela/distribuita è complicata → bisogna gestire: comunicazione tra nodi, sincronizzazione, scambio dei dati, bilanciare il carico, gestire i guasti e coordinare i risultati. → tutti devono collaborare senza interferire tra loro e se un nodo si blocca l'elaborazione deve proseguire.

Per risolvere questi problemi sono nate nuove architetture di calcolo distribuito. Queste architetture sono:

- **Cluster of commodity Linux nodes** → insieme di computer i cui componenti sono economici e facilmente reperibili e il quale sistema operativo è Linux.
- **Commodity network** → i nodi del cluster devono comunicare tra loro e si utilizza il protocollo Ethernet, invece di reti specializzate molto costose. Questo significa:
 - costi contenuti;
 - facile manutenzione;
 - possibilità di aggiungere facilmente nuovi nodi.

Queste architetture non sono opposte, ma componenti della stessa architettura distribuita.

Il commodity cluster è la realizzazione pratica dello scale-out sopraccitato, utilizzando computer economici.

Ma come funziona l'esecuzione? **Il calcolo si porta vicino i dati e non viceversa**. Se dovessimo spostare i dati all'interno della rete sprecheremmo troppo tempo.

Quindi, **invece di spostare i dati si sposta il programma nei nodi possidenti dei dati**. Attraverso questa strategia si riduce il traffico di rete.

La ridondanza apportata da questo modello è fortemente consigliata, dato che come già detto si ha una tolleranza ai guasti maggiore e si ha una maggiore velocità di lettura.

La lettura è più veloce dato che Hadoop sceglie la copia più vicina o quella meno occupata. In questo modo il carico viene distribuito e la lettura diventa veloce.

I file all'interno di queste architetture vengono divisi in chunk memorizzati su diverse macchine. I **dati** vengono replicati 2,3 volte e **conservati in server di rack differenti**(armadi diversi).

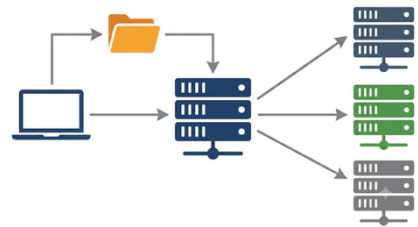
Nel cluster esiste un nodo speciale chiamato **nodo Master**(o **NameNode**) che contiene informazioni sui file, ossia:

- quanti chunk lo compongono;
- dove si trova ciascun chunk;
- quali sono le repliche.

È come un indice che sa dove sono memorizzati tutti i pezzi del file. Per questo motivo è fondamentale e anch'esso è sottoposto ad operazioni di backup per evitare single point of failure.

Una directory del file system sa dove trovare il master node, quindi:

- 1) client contatta la directory del file system;
- 2) la directory indica dove si trova il Master;
- 3) il Master comunica la posizione dei chunk;
- 4) il Client legge direttamente dai chunk dei vari nodi(dai DATANODE).



MAP-REDUCE

MapReduce non è nato dal nulla, ma affonda le sue radici nella **programmazione funzionale**. Il concetto a cui si ispira è:

- **Map** → *prende una funzione f e si applica a tutti gli elementi di una lista* → ogni elemento viene trasformato in modo indipendente;
- **Fold** → *prende una funzione di aggregazione e combina iterativamente gli elementi di una lista in un unico risultato*;

MapReduce prende queste idee e le adatta al calcolo distribuito su cluster. Analizziamo quindi:

- **MAP** → prende in input una coppia chiave-valore (k,v) e restituisce un elenco di coppie chiave valore $(k_1,v_1), (k_2,v_2), \dots, (k_n,v_n)$. Il numero di coppie può essere 0 o più.

Il framework:

- raccoglie tutte le coppie con la stessa chiave;
- le raggruppa.

Ad esempio:

- Alice: "mi piace il calcio" → ("mi",1), ("piace", 1), ("il",1), ("calcio");

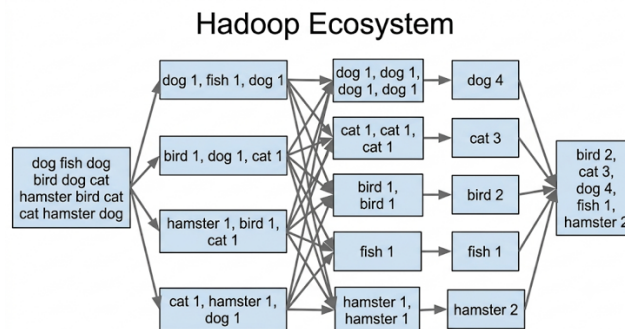
- Bob: "calcio calcio bello" → ("calcio",1), ("calcio",1), ("bello",1);

GROUP BY KEY → ("bello",1), ("calcio",[1,1,1]), ("il",1), ("mi",1), ("piace",1).

- **REDUCE** → prende in input una chiave e una lista di valori $(k,[v_1, v_2, \dots, v_n])$ e li combina. Continuando l'esempio: ("mi",1), ("piace",1), ("il",1), ("calcio",3), ("bello",1).

Il **GROUP BY KEY** citato prima è la parte automatica nel framework che permette di:

- **SORT** → ordinare le coppie per chiave(k);
- **SHUFFLE** → raggruppare i risultati le coppie con la stessa chiave.



Ma cosa succede quando scriviamo realmente **Map(k,v)** e **Reduce(k,list(v))**?

Il sistema effettua le seguenti operazioni:

- 1) prende i dati enormi e li suddivide in blocchi nei vari cluster;
- 2) **scheduling dell'esecuzione** decide quale macchina esegue Map e quale Reduce → lo fa attraverso il principio di **data locality** (si sposta il programma dove sono i dati);
- 3) dopo il Map si effettua il Group Key che ordina i valori per chiave e li raggruppa per la stessa → serve a preparare la Reduce;
- 4) durante l'esecuzione se un nodo non funziona, il lavoro viene riassegnato automaticamente recuperando i dati delle repliche → computazione continua;
- 5) lo scambio dei dati tra nodi, ad esempio output del Map al Reduce viene fatto via rete (ethernet cluster).

Ogni risultato intermedio (operazione Map) viene conservato sul disco locale del worker e non va subito nell'HDFS.

Il **Master node** è il componente che: **coordina l'esecuzione di Map e Reduce su tutti i nodi del cluster.**

Il Master tiene traccia dello stato di ogni task, ossia: **idle** (inattivo), **in-progress** (in esecuzione), **completed** (completato).

Quando un worker finisce un lavoro e diventa disponibile, il Master gli assegnerà un nuovo task (idle) e quindi il **sistema è dinamico e bilanciato.**

Quando i Map task terminano: vengono prodotti i risultati intermedi che vengono divisi in R file (uno per ogni reducer). Infine, il worker invia la posizione dei file intermedi e la dimensione in modo tale che il Master sappia dove sono i dati per la reduce.

A sua volta queste informazioni saranno inviate ai Reduce che sanno dove leggere i dati.

Per il controllo del corretto funzionamento del sistema, il Master controlla lo stato dei worker attraverso il **ping**. Se il master non riceve segnali, assume che il worker sia fallito e riassegna il task.

Quanti Map e Reduce task? Solitamente il numero M di Map deve essere maggiore del numero di nodi del cluster, questo perché i nodi lavorano a turni e appena che finiscono un task ne prendono un altro → **Dynamic Load Balancing.**

Il numero R di reduce è più piccolo di M dato che ogni reduce produce un file finale e troppi reduce vorrebbero dire troppi file di output(overhead inutile).

Quando siamo vicini alla fine della fase, si possono generare copie di backup delle attività e si sceglie il nodo che finisce prima. In questo modo, si riduce notevolmente il tempo di completamento del job.

Se l'ordine delle operazioni non cambia il risultato ottenuto poi dal Reduce, allora potremmo aggregare i risultati sullo stesso nodo dove si fa Map. Questo porta a dei benefici, infatti: il **Combiner** riduce il traffico di rete, facendo una **pre-aggregazione locale**. Invece di inviare milioni di coppie (k,v) se ne mandano solo poche.

Dato che tutte le coppie con la stessa chiave devono finire nello stesso cluster(altrimenti non possono essere aggregate), come si decide quale reducer deve elaborare una certa chiave?

Questo problema ha una soluzione che è la **Partition Function**, ossia una funzione che assegna ogni chiave a un reducer. Questa funzione di default usa normalmente:

$$\text{hash}(\text{key}) \bmod R$$

dove:

- **hash(key)** → calcola un numero dalla chiave;
- **R** → numero di Reducer;
- **$\bmod R$** → permette di scegliere quale reducer userà la chiave.

La funzione sopracitata viene usata durante lo Shuffle e decide a quale shuffle inviare la chiave.

Due esempi di come viene utilizzato il paradigma di MapReduce per parallelizzare algoritmi su grandi quantità di dati sono:

- **JOIN** → obiettivo è unire due tabelle e si può fare attraverso il modello MapReduce nel seguente modo:
 - **Map** → trasforma ogni tupla in una coppia chiave-valore usando come chiave l'attributo di Join(ad esempio cognome);
 - **Shuffle** → raggruppa automaticamente tutte le tuple con la stessa chiave(es. cognome);
 - **Reduce** → combina le tuple provenienti dalle tabelle e produce il risultato della Join.

In questo modo l'operazione di Join viene eseguita in parallelo su più nodi del cluster.

- **MATRIX- MULTIPLICATION** → obiettivo è calcolare il prodotto tra una matrice e un vettore e si può fare attraverso il modello MapReduce nel seguente modo:
 - **Map** → ogni nodo elabora una parte della matrice e produce i prodotti $m_{ij} \times v_j$, associandoli alla riga i ;
 - **Shuffle** → raggruppa tutti i prodotti appartenenti alla stessa riga;
 - **Reduce** → somma tali prodotti per ottenere il valore finale x_i .

Anche qui il lavoro viene distribuito tra molte macchine, accelerando il calcolo.

Questo ci aiuta a capire che MapReduce non è un algoritmo specifico, ma un modello di programmazione distribuita.

SPARK

MapReduce innovativo ma inefficiente

L'evoluzione da MapReduce a **Spark** nasce dall'esigenza di superare alcuni limiti emersi con l'utilizzo di MapReduce nell'elaborazione dei Big Data.

Sebbene abbia rappresentato una svolta nella computazione distribuita, ma con il tempo si è rivelato poco flessibile ed efficiente per applicazioni più complesse.

MapReduce mette a disposizione solo due operazioni fondamentali, ossia Map e Reduce. Questo potrebbe limitare l'esecuzione di operazioni più complesse, infatti il **codice MapReduce diventa lungo e difficile da scrivere**.

Diversi problemi non sono quindi semplici da descrivere con questo approccio rigido.

Un ulteriore svantaggio è il collo di bottiglia nelle prestazioni, che limita la velocità complessiva. Nel caso di **MapReduce il collo di bottiglia principale è il disco**, infatti, le continue scritture sul HDFS possono introdurre latenza nell'elaborazione.

Spark invece mantiene i dati sulla RAM quando possibile, che è enormemente più veloce degli hard disk e degli SSD.

MapReduce non si adatta perfettamente a grosse applicazioni che richiedono elaborazioni molto complesse dato che ogni passaggio è un job separato.

Data-Flow System e Spark

Un **Data-Flow System** è un sistema in cui l'elaborazione è rappresentata come un grafo di operazioni, dove i dati scorrono da un'operazione all'altra. Ogni nodo del grafo esegue un'elaborazione e produce dati che vengono passati ai nodi successivi.

MapReduce è molto semplice dato che ha solo due **livelli**(stages) costituiti da: **MapTask** e **ReduceTask**.

I dati vengono elaborati dai mapper e poi inviati ai reducer, per questo motivo si dice che **MapReduce utilizza due ranghi(stages) di elaborazione**. Non è possibile inserire facilmente altre elaborazioni intermedie dato che bisogna creare un nuovo job MapReduce.

I sistemi moderni(tipo Spark) estendono il concetto di MapReduce, permettendo di avere un numero arbitrario di operazioni(ergo di avere più di due stages).

I data-flow systems moderni generalizzano questo modello attraverso:

- 1) **Consentire qualsiasi numero di attività(stages)** → si costruisce una vera e propria pipeline di elaborazione costituita da più livelli;
- 2) **Consentire funzioni diverse da Map e Reduce**, ad esempio una pipeline Spark:

```
file.filter(...)  
  .map(...)  
  .groupByKey(...)  
  .sort(...)  
  .take(10)
```

- 3) **Il flusso deve essere in una sola direzione**, infatti vengono costruiti i DAG ossia i grafi aciclici direzionati. Dato che si segue una sola direzione possiamo avere la **proprietà di blocco**, cioè di garantire che l'operazione aspetti di ricevere i dati necessari prima di continuare. Inoltre, si può garantire il **recupero dei singoli task** che di tutto il lavoro.

Apache Spark è un **framework open-source per l'elaborazione distribuita dei Big-Data**, che nasce con l'obiettivo di mantenere i vantaggi di MapReduce escludendo però i limiti principali.

In Spark, le trasformazioni scorrono lungo una sola direzione e il numero di trasformazioni è maggiore. Questo permette a Spark di essere più espressivo e descrivere algoritmi complessi in modo semplice.

Spark mantiene l'idea di elaborazione distribuita, ma introduce numerose innovazioni, ossia:

- **condivisione veloce dei dati** → i dati vengono condivisi direttamente in memoria tra le varie operazioni evitando il collo di bottiglia introdotto dalla scrittura su disco condiviso;
- **caching dei dati per query ripetitive** → mantenere i dati in memoria per riutilizzarli successivamente → evita di rileggere lo stesso dataset dal disco più volte;
- **grafi di esecuzione generali(DAG)** → Spark costruisce una **DAG** o **Directed Acyclic Graph** → partendo dal grafo decide automaticamente l'ordine migliore per eseguire le operazioni e permette numerosissime ottimizzazioni;
- **funzioni più ricche di Map e Reduce** → mette a disposizione moltissime operazioni già pronte → evita di costruire diversi job separati;
- **compatibile con Hadoop** → Spark non sostituisce necessariamente Hadoop, ma può lavorare insieme ad esso → può utilizzare HDFS come file system distribuito e YARN come gestore delle risorse del cluster.

Il concetto fondamentale di Spark è l'**RDD** ossia **Resilient Distributed Dataset**, in termini più semplici è una collezione distribuita di dati, immutabile e tollerante ai guasti, che può essere elaborata in parallelo.

In pratica, un RDD rappresenta il livello più basso del modello di programmazione Spark e permette di descrivere le trasformazioni sui dati in modo esplicito, come una sequenza di operazioni.

Tuttavia, lavorare direttamente con gli RDD può risultare complesso e poco efficiente da un punto di vista della programmazione e ottimizzazione. Per questo motivo Spark introduce API di alto livello, ossia strumenti astratti e semplici da utilizzare come:

- **dataframe** → rappresentano i dati come una tabella strutturata a righe e colonne;
- **dataset** → simili ai DataFrame ma con una tipizzazione più forte → cioè ogni riga ha un tipo ben definito.

Queste API di alto livello permettono di esprimere le operazioni mediante una programmazione dichiarativa tipo SQL.

Architettura SPARK

Un cluster è un insieme di più computer che collaborano tra loro per risolvere problemi computazionalmente troppo grandi per una singola macchina.

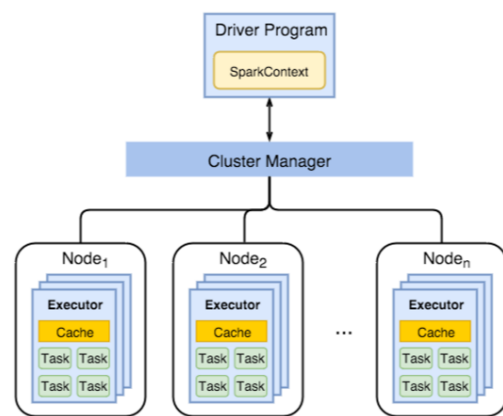
Tuttavia, il semplice fatto di avere più macchine non è sufficiente: è necessario un sistema in grado di gestire, coordinare e assegnare i compiti ai vari nodi del cluster. Questo ruolo è svolto da un framework come Apache Spark.

I componenti chiave di Spark sono:

- 1) la **gestione del cluster è affidata** a un sistema esterno o integrato → YARN, Mesos o il cluster manager standalone di Spark → il suo compito è composto dal monitoraggio delle risorse disponibili nel cluster, allocare le risorse alle applicazioni e gestire la concorrenza tra jobs.

- 2) **applicazione Spark** è il programma che si vuole eseguire nel cluster → composta da due elementi fondamentali:

- i. **driver process** → cervello dell'applicazione che si occupa di: eseguire il programma principale, costruire il piano di esecuzione, dividere il lavoro in task ed assegnare i task ai nodi → coordina il lavoro;



- ii. **executor process** → processi che si occupano di eseguire i task e restituire i risultati comunicando continuamente lo stato di avanzamento → fanno il lavoro pesante.

- 3) **modalità d'esecuzione** → Spark può essere eseguito in due modalità principali:

- i. **modalità locale** → tutto gira su una singola macchina(driver ed executor) → utile per studio, test e per singoli dataset;
- ii. **modalità cluster** → applicazione viene distribuita su più macchine permettendo scalabilità e gestione dei Big Data reali.

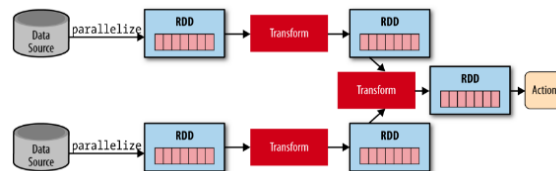
Tutto questo permette a **Spark di trasformare un programma unico in un insieme di task distribuiti ed eseguirli in parallelo su più nodi.**

I dati come già citato prima possono essere rappresentati tramite gli RDD ossia una struttura dati distribuita e immutabile(solo lettura). Possiamo vederla come una collezione di record su cui Spark può eseguire operazioni in parallelo.

Un RDD può essere visto come una generalizzazione delle coppie **Chiave-Valore**, esempio: `RDD[(String, Double)] → (a, 10.3), (b, 15.7), (b, 19.1).`

Gli RDD sono suddivisi in partizioni, che vengono distribuiti sui vari nodi del cluster per aumentare le prestazioni. I modi in cui vengono suddivise le partizioni possono essere:

- **HashPartitioner** → usa l'hash della chiave e i record con la stessa chiave finiscono nella stessa partizione → utile per operazioni tipo groupByKey o reduceByKey;
- **RangePartitioner** → ordina i dati in base alla chiave e divide i dati in range(intervalli) → utile quando si effettuano operazioni del tipo sorting;
- **CustomPartitioner** → partizionamento definito dal programmatore, che solitamente si utilizza quando è utile una logica specifica.



Gli RDD possono essere creati in due modi, ovvero: o da dati esterni tipo CSV o da trasformazioni di altri RDD. Quest'ultimo ci permette di impilare gli RDD creando una catena di trasformazioni.

Gli RDD sono adatti quando si:

- si applica la stessa operazione a tutti gli elementi;
- si lavora a basso livello sui dati;
- si ha bisogno di controllo fine sulle trasformazioni;
- si vuole gestire direttamente partizioni e logica distribuita.

Spark può conservare in memoria(o su disco) i risultati intermedi di un'elaborazione, così da non doverli ricalcolare ogni volta che servono.

Il **caching** indica che i risultati intermedi devono essere conservati in memoria(non sul disco) e utilizzati quando servono. Quindi, si utilizza il metodo **.cache()** per indicare di conservare in memoria, in modo tale che le elaborazioni successive saranno molto più veloci.

Il metodo **.persist()** fa la stessa cosa, ma permette di scegliere dove memorizzare l'RDD. Le diverse opzioni per memorizzare l'RDD sono:

- **MEMORY_ONLY** → salvato solo in RAM → alcune partizioni non vengono memorizzate e quando serviranno Spark le ricalcolerà;
- **MEMORY_AND_DISK** → le partizioni vengono salvate prime in memoria e se non c'è spazio sul disco;
- **MEMORY_ONLY_SER** → i dati vengono serializzati per occupare meno memoria → vengono trasformati in sequenze di byte invece di oggetti → lo svantaggio è che sono necessarie più CPU per convertirli in oggetti;
- **MEMORY_AND_DISK_SET** → dati serializzati conservati in memoria e se non c'è spazio sul disco;
- **DISK_ONLY** → dati conservati soltanto sul disco.

In Spark le operazioni sugli RDD si dividono in due grandi categorie, vale a dire: **trasformazioni** e **azioni**. A queste operazioni si aggiungono le variabili condivise broadcast e accumulatori.

Le **trasformazioni sono operazioni che creano un nuovo RDD a partire da uno esistente**, applicando una funzione deterministica(stesso input genera lo stesso output). Queste operazioni non eseguono il calcolo immediatamente ma costituiscono un piano di esecuzione(DAG). → map(), filter(), join(), union(), intersection(), distinct()

Spark utilizza la **lazy evaluation** ossia che nulla viene eseguito finché non è strettamente necessario. Il calcolo parte solo quando c'è un'azione.

Le **azioni sono operazioni che avviano realmente l'esecuzione e restituiscono un risultato o lo salvano**. Quando sono presenti le azioni, Spark segue questa impostazione:

- Spark esegue tutto il DAG delle trasformazioni;
- calcola i dati;
- restituisce o salva il risultato.

Le azioni avviano il calcolo dell'intero DAG e restituiscono un risultato al driver, non un nuovo RDD. Le azioni tipicamente sono: count(), collect(), reduce(), save().

Spark permette di condividere dati tra nodi del cluster con due strumenti:

- **broadcast variable** → una **variabile in sola lettura distribuita** su tutti i nodi del cluster → evita di inviare la stessa variabile più volte ad ogni task → riduce il traffico di rete e migliora le performance;
- **accumulators** → variabili condivise utilizzate per **contatori** o **somme distribuite** → i worker scrivono il valore e il driver raccoglie il risultato finale.

Il **Task Scheduler** di Spark è il componente che prende un programma e lo trasforma in un insieme di task eseguiti sul cluster, organizzati in un DAG.

Spark non è limitato a pipeline lineari tipo MapReduce dato che può gestire:

- rielaborazioni multiple;
- riuso di dati;
- ramificazioni;
- join.

Le pipeline funzionano dove è possibile, infatti per operazioni come groupBy si deve interrompere la pipeline e creare un nuovo stage. Il groupBy richiede: redistribuzione dei dati tra nodi e scambio di partizioni.

Il DataFrame in Spark è una struttura dati che organizza le informazioni in forma tabellare(righe e colonne), molto simile ad una tabella di un database relazionale. A differenza degli RDD, che sono collezioni generiche di oggetti senza una struttura esplicita, i DataFrame impongono uno schema.

Questa struttura "rigida" permette a Spark di lavorare più efficientemente, inoltre viene utilizzato Catalyst Optimizer. Quest'ultimo è un motore che analizza le query e le riscrive in modo più efficiente. Query come Join, GroupBy o filtri vengono ottimizzati e risultano più veloci.

Di seguito è riportato un po' di codice in Python che permette di lavorare con Spark.

```
#CREA LA SESSIONE SPARK
spark = SparkSession.builder.getOrCreate()
```

Creazione Sessione Spark

Prima di iniziare a lavorare con i dati, deve essere creata una sessione Spark, che corrisponde al punto di ingresso che

inizializza e coordina tutto l'ambiente di esecuzione.

.getOrCreate() indica a Spark di avviare o riutilizzare un contesto che gestisce tutte le operazioni sui dati.

Caricamento dati

Attraverso il codice riportato a sinistra possiamo caricare dei dataset. In questo caso in un unico DataFrame vengono caricati tutti i file .csv contenuti nella cartella.

```
df = spark.read.csv(
    "temperature_data/*.csv",
    header=True,
    sep=";",
    inferSchema=True
)
```

Query con Spark

Questo codice rappresenta un approccio dichiarativo basato sulle DataFrame API di PySpark, in cui l'utente descrive le trasformazioni da applicare ai dati senza specificare il flusso di esecuzione. Spark, tramite il Catalyst Optimizer, traduce queste operazioni in un piano di esecuzione distribuito ottimizzato.

```
df_giornaliero = df.withColumn(
    "DATA_GIORNO", to_date("DATARIL")
).groupBy("DATA_GIORNO", "ID_STAZ")\
.agg(sum("VALORE").alias("PRECIP"))
```

Utilizzo SQL Query

Si possono utilizzare le query SQL, se si ha più familiarità.

```
query1 = """
SELECT ID_STAZ, ROUND(sum(VALORE),2) AS TOT_PRECIPITAZIONI
FROM global_temp.precipitazioni
WHERE VALORE is not null
GROUP BY ID_STAZ
"""

calcolo_precipitazioni = spark.sql(query1)
calcolo_precipitazioni.show()
```

Il **Dataset** è un'estensione dei DataFrame che introduce la **tipizzazione forte (type-safe)**.

Questo significa che i dati non sono solo righe generiche, ma oggetti con un tipo definito, e molti errori possono essere rilevati già in fase di compilazione.

In pratica, i Dataset combinano i vantaggi degli RDD e dei DataFrame: da un lato mantengono la flessibilità e la sicurezza degli RDD (come l'uso di funzioni lambda), dall'altro sfruttano le ottimizzazioni dei DataFrame e di Spark SQL.

Le API Dataset sono disponibili solo in **Scala e Java**, mentre in **Python non esistono davvero**. In PySpark si usano praticamente solo DataFrame.

Infine, se dovessimo mettere a confronto Spark con MapReduce:

- **SPARK** → generalmente più veloce perché lavora in memoria, però la dipendenza dalla memoria e l'utilizzo di molta RAM, può degradare se le risorse sono limitate. Dal punto di vista della programmazione è molto più semplice da usare perché offre API di alto livello.
- **HADOOP MAP-REDUCE** → è più lento dato che fa continue scritture sul disco, però è più robusto in ambienti con risorse limitate.

RICERCA DI ITEMS SIMILI

Introduzione

Molti problemi possono essere espressi come la **ricerca di insiemi simili**. Ciò vuol dire che, si devono trovare i vicini più prossimi (near-neighbors) in uno spazio multidimensionale attraverso misure di distanza o similarità.

La ricerca di insiemi vicini può avvenire per i seguenti motivi:

- verifiche di eventuale plagiarismo;
- sistemi di raccomandazione;
- ricerca per immagini.

Un problema comune è il seguente:

“Dato **O oggetti** dove ogni oggetto è descritto da **d feature**, l’obiettivo è trovare gruppi di oggetti simili”.

Ogni oggetto è un punto in uno spazio d-dimensionale, infatti, le feature sono le coordinate del punto e l’oggetto stesso è il vettore. $O_i = (f_1, f_2, \dots, f_n)$

Si definisce una funzione di similarità che misura due oggetti sono simili:

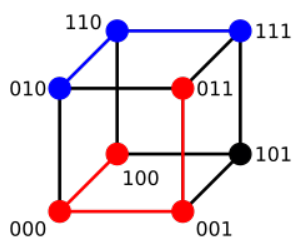
$$\text{Similarity}(O_1, O_2) \in [0(\text{DIVERSI}), 1(\text{IDENTICI})]$$

Le operazioni richieste devono essere veloci senza scansionare il dataset ogni volta e il clustering deve essere quasi lineare/sub-quadratico, scalabile anche su miliardi di punti.

Ci sono diversi modi per calcolare la similarità o quanto differiscono due punti, e questi sono:

- **HAMMING DISTANCE** → la distanza di Hamming misura il **numero di posizioni in cui due sequenze di uguale lunghezza differiscono**.

Quindi, più grande è il valore e più sono diversi i punti.



Nell’esempio proposto a destra possiamo prendere due punti come:

- **110**;
- **011**;

Se calcolassimo la distanza tra i due punti troveremo che:
HammingDistance = 2

- **COEFFICIENTE DI JACCARD** → è una misura di similarità tra insiemi, utilizzata quando i dati sono binari o a “presenza/assenza”. Dati due insiemi A e B:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Si prendono quindi gli **elementi comuni** e **si dividono per gli elementi totali presenti** nei due insiemi.

- **COSINE SIMILARITY** → misura che serve a capire quanto due oggetti sono simili in direzione, indipendentemente dalla loro dimensione o lunghezza. Dati due vettori A e B:

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \|B\|}$$

Un esempio di calcolo del cosine similarity è il seguente:

A = Antony

B = Jonny

Ora contiamo quante volte compaiono i caratteri:

A) $a = 1, n = 2, t = 1, o = 1, y = 1, j = 0$;

B) $a = 0, n = 2, t = 0, o = 1, y = 1, j = 1$;

Ora facciamo il prodotto tra: $A = (1, 2, 1, 1, 1, 0)$ e $B = (0, 2, 0, 1, 1, 1)$

$$A * B = 1*0 + 2*2 + 1*0 + 1*1 + 1*1 + 1*0 = 0+4+0+1+1+0 = 6$$

Calcoliamo le norme di A e B:

$$A) \sqrt{1^2 + 2^2 + 1^2 + 1^2 + 1^2 + 0^2} = \sqrt{1 + 4 + 1 + 1 + 1 + 0} = \sqrt{8}$$

$$B) \sqrt{0^2 + 2^2 + 0^2 + 1^2 + 1^2 + 1^2} = \sqrt{0 + 4 + 0 + 1 + 1 + 1} = \sqrt{7}$$

Infine:

$$\frac{A * B}{\|A\| \|B\|} = \frac{6}{\sqrt{7} * \sqrt{8}} = \frac{6}{\sqrt{56}} = \mathbf{0.80}$$

Quando si devono trovare elementi simili ad uno preso in considerazione, possiamo agire in tre modi. In seguito, sono riportati 3 modi diversi per la risoluzione del problema:

- 1) **THE NEAREST NEIGHBOR (il più vicino)** → si prende il punto più vicino in assoluto.

Formalmente:

$$\arg \min_{o \in O} \text{distance}(o, q(\text{punto preso in considerazione}))$$

Ad esempio:

- a) Il documento più rilevante;
- b) L'utente più simile

- 2) **r – Nearest Neighbors** → trovi tutti i punti entro una distanza r attorno a q.

Ad esempio:

- a) clustering locale;

- 3) **(r,c) – Approx Nearest Neighbors** → accetti un errore controllato e si prendono punti approssimati → si perde un po' di precisione e si guadagna enormemente in performance.

Si prende un raggio ideale r e un fattore di tolleranza $c > 1$. Quindi si cerca:

$$\text{distance} \leq c * r$$

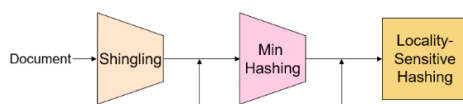
Gli **embedding** trasformano dati complessi (testi, immagini, utenti) in vettori numerici, creando uno spazio in cui oggetti simili sono vicini tra loro. In questo modo la similarità diventa un problema geometrico.

Il problema è che cercare i più simili tra milioni di punti è troppo costoso. Per questo si usa **LSH** (Locality-Sensitive Hashing), che divide i vettori in gruppi (**bucket**) in modo che quelli simili finiscano nello stesso gruppo.

Così, invece di confrontare tutto il dataset, si cercano i simili solo dentro pochi bucket, rendendo la ricerca molto più veloce.

Si passa da una complessità $O(N^2)$ ad un costo quasi lineare $O(N)$.

Una pipeline classica per trovare documenti simili su larga scala è la seguente:



- 1) **Shingling** → trasformare ogni documento in un insieme di pezzi → documento non più visto come testo, ma come insieme di sottostringhe.
- 2) **MinHashing** → prende l'insieme lo trasforma in una firma piccola(signature) → una lista corta di numeri che rappresenta il documento.
Nonostante ciò la capacità di stimare quanto due insiemi sono simili è mantenuta.
- 3) **LSH(Locality-Sensitive Hashing)** → le signature simili vengono messi negli stessi bucket e attraverso ciò si trovano subito firme simili.

Shingling

Uno **shingle** è una sequenza di k elementi consecutivi del testo, dove:

- K è la lunghezza della finestra;
- Token sono i caratteri o le parole;

È preferibile rispetto al semplice conteggio: conserva l'ordine, cattura la struttura del testo e distingue frasi con stesse parole ma ordine diverso.

La scelta di k è un **trade precisione e robustezza**, infatti:

- **K piccolo** → troppi documenti simili per caso(poco robusto);
- **K grande** → più preciso ma più sparso(troppo preciso);

Solitamente $K = 5$ è buono per piccoli documenti, invece $K = 10$ è meglio per documenti lunghi.

Gli shingles generati poi si fanno diventare dei numeri con l'hash, in modo tale che ogni documento diventa: un vettore binario le cui dimensioni sono i singoli shingles.

La similarità verrà poi calcolata attraverso la Jaccard similarity, dove l'intersezione corrisponderà agli shingles in comune e l'unione a tutti gli shingles possibili.

MinHashing

L'obiettivo del **MinHashing** è **convertire grandi insiemi in firme corte**, preservando però la similarità tra gli elementi. Gli insiemi vengono codificati mediante vettori booleani(0/1), dove 0 indica l'assenza dello shingle e 1 la presenza.

Dagli insiemi si passa alle matrici booleane, dove:

- le righe rappresentano gli elementi(shingles);
- le colonne rappresentano gli insiemi(documenti).

La matrice M è definita come:

$$M(e, s) = 1$$

se e solo se l'elemento e appartiene al documento s .

	Documents			
Shingles	1	1	1	0
	1	1	0	1
	0	1	0	1
	0	0	0	1
	1	0	0	1
	1	1	1	0
	1	0	1	0

I documenti sono disposti lungo le colonne, invece gli elementi lungo le righe.

Ad esempio, gli shingles presenti nell'insieme universale sono: {casa, cane, gatto, topo}

Nell'immagine che segue abbia la disposizione:

Shingle	D1	D2	D3
casa	1	1	0
cane	1	1	0
gatto	1	0	1
topo	0	0	1

La matrice risultante è generalmente molto sparsa. La similarità tra due colonne è la similarità di Jaccard tra gli insiemi corrispondenti.

L'idea del MinHashing è applicare una funzione hash a ogni colonna C ottenendo una firma:

$$h(C)$$

tale che:

- 1) la **firma** sia abbastanza **piccola** da **poter essere mantenuta in RAM**;
- 2) la **similarità tra le firme riflette la similarità** tra le **colonne originali(documenti)**.

Si deve quindi trovare una funzione $h(\cdot)$ tale che:

- se $\text{sim}(C_1, C_2)$ è alta, allora con alta probabilità:

$$h(C_1) = h(C_2)$$

- se $\text{sim}(C_1, C_2)$ è bassa, allora con alta probabilità:

$$h(C_1) \neq h(C_2)$$

La funzione hash dipende dalla misura di similarità utilizzata e non tutte le metriche di similarità possiedono una funzione Hash adatta.

Per la similarità di Jaccard esiste una funzione hash specifica, ovvero MinHashing.

Il MinHashing funziona nel seguente modo:

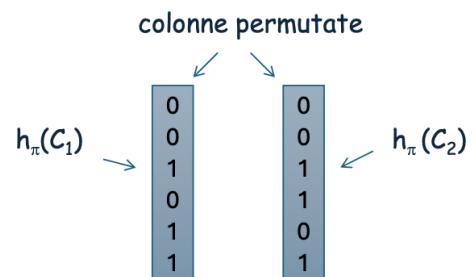
si permutano casualmente le righe della matrice booleana mediante una permutazione casuale π .

Per una colonna C, definiamo:

$$h_{\pi}(C)$$

come l'indice della prima riga(nell'ordine permutato) che contiene un valore 1.

$$h_{\pi}(C) = \min_{\pi}(C)$$



Questo passaggio viene effettuato molte volte attraverso permutazioni indipendenti tra loro. La firma del documento sarà quindi:

$$[h_1(C), h_2(C), h_3(C), \dots, h_n(C)]$$

La probabilità che due colonne abbiamo lo stesso valore MinHash è uguale alla loro similarità di Jaccard.

La probabilità che siano simili si calcola attraverso la seguente formula:

$$Prob[h_l(C_1) = h_l(C_2)] = \frac{\text{righe con entrambe le colonne 1}}{\text{righe con almeno una delle due colonne 1}}$$

La probabilità è uguale alla similarità:

$$Prob[h_l(C_1) = h_l(C_2)] = Sim(C_1, C_2)$$

Correttezza del MinHash → TEOREMA

Se due documenti sono davvero simili, allora MinHash li riconosce come simili con alta probabilità, se non sono simili, MinHash non li confonde quasi mai.

Si introducono un po' di concetti:

- $\delta \rightarrow$ quanto errore tolleriamo;
- $\varepsilon \rightarrow$ quanto spesso possiamo sbagliare;
- $k \rightarrow$ numero di MinHash che si utilizza $\rightarrow$ più k è grande e meno errori si fanno;
- $s \rightarrow$ soglia di similarità alta;
- $c \rightarrow$ soglia inferiore che scarta la possibilità che due documenti siano simili.

Ci sono due casi:

- A) DOCUMENTI SIMILI $\rightarrow$ Se $Sim(C_i, C_j) \geq s$ allora la similarità stimata non sarà troppo più piccola del vero valore $Prob[Sim(C_i, C_j) \geq (1 - \delta)s] \geq 1 - \varepsilon$.
Solo pochi falsi negativi.
- B) DOCUMENTI NON SIMILI $\rightarrow$ Se $Sim(C_i, C_j) \leq c$
allora $Prob[Sim(C_i, C_j) \leq (1 + \delta)c] \geq 1 - \varepsilon$.
Solo pochi falsi negativi.

DIMOSTRAZIONE (CASO A $\rightarrow$ IL CASO B SI DIMOSTRA IN MODO SIMILE)

Fissiamo una coppia di colonne C_i, C_j con $Sim(C_i, C_j) \geq s$, allora dobbiamo dimostrare che

$$Prob[Sim(C_i, C_j) \geq (1 - \delta)s] \geq 1 - \varepsilon$$

Consideriamo la variabile aleatoria $X = X_1 + \dots + X_k$ dove X_l vale 1 se $h_l(C_i) = h_l(C_j)$ altrimenti vale 0.

Il **Chernoff Bound** dice che: **se sommi tante variabili random indipendenti, la probabilità di grande deviazione dalla media è piccolissima.**

$$\text{Tradotto: } P(X \neq E[X]) \approx 0$$

Quindi applicando il Chernoff Bound abbiamo che:

$$Prob[X < (1 - \delta) \cdot E[X]] < e^{-\frac{\delta^2 E[X]}{2}}$$

Verifichiamo quanto X è più piccolo della media aspettata $E[X]$. Il valore della probabilità diventa esponenzialmente piccolo.

$$P(\text{errore grande}) \leq \varepsilon$$

Sappiamo quindi che:

$$\Pr[h_{\pi}(C_1) = h_{\pi}(C_2)] = \text{Sim}(C_1, C_2)$$

La probabilità che due documenti abbiano lo stesso MinHash è uguale alla loro similarità di Jaccard.

Ora si può estendere il concetto a più funzioni hash, ossia che: la similarità tra due signature è la frazione di hash in cui coincidono.

Quindi, invece di fare k permutazioni casuali delle righe, operazione costosa e complessa, si utilizzano k funzioni hash universali, che simulano l'effetto di una permutazione casuale.

Si applica la funzione hash agli indici e si prende l'indice minore il cui valore del componente è uguale a 1.

Shingle	Hash
ab	15
bc	8
ca	21
de	4

Si inizializza $h(C)[i] = \infty$ e si procede nella seguente maniera:

- Se $15 < h(C)[i](\infty)$ allora $h(C)[i] = 15$ (assegnazione) → verificato;
- Se $8 < h(C)[i](15)$ allora $h(C)[i] = 8$ (assegnazione) → verificato;
- Se $21 < h(C)[i](8)$ allora $h(C)[i] = 21$ (assegnazione) → non verificato;
- Se $4 < h(C)[i](8)$ allora $h(C)[i] = 4$ (assegnazione) → verificato.

Alla fine, il valore di $h(C)[i] = \min(C)$ sarà 4.

Con k funzioni hash si otterrà una signature del tipo: $\text{sig}(C) = [h(C)[1], h(C)[2], \dots, h(C)[k]]$.

In questo modo non dovremmo effettuare continue permutazioni.

Locality Sensitive Hashing

Il **Locality Sensitive Hashing**(LSH) è una tecnica usata nel contesto del Big Data per trovare in modo efficiente elementi simili tra loro, senza dover confrontare tutte le coppie possibili.

L'idea fondamentale è generare per ogni oggetto uno **sketch** (firma compatta) molto più corto del numero di caratteristiche originali d, in modo tale che **la similarità tra vettori venga trasformata in una probabilità di "uguaglianza" tra i loro sketch**. In altre parole, oggetti simili hanno un'alta probabilità di produrre sketch uguali.

LSH è una tecnica randomizzata, quindi non garantisce risultati esatti ma risultati molto probabili. Inoltre, permette un accesso locale ai dati migliorando le prestazioni in contesti di disco o dati distribuiti.

Nel caso della distanza di Hamming, si considerano vettori binari di dimensioni d. Una funzione può essere costruita scegliendo un insieme I di r random posizioni e proiettando il vettore solo su queste posizioni.

$$h(p) = p[I]$$

Due vettori hanno probabilità di collisione proporzionale alla loro similarità. Quindi, maggiore è la similarità e maggiore è la probabilità che:

$$h(p) = h(q)$$

Qual è la probabilità che due vettori producano lo stesso hash?

La probabilità che sia uguale è:

$$s = \frac{d - D(p, q)}{d}$$

dove:

- **d** è il numero di dimensioni;
- **D(p,q)** è la **distanza di hamming** → 1 se diversi, 0 se uguali.

Questa formula si considera quando si ha una sola coordinata, ma devono coincidere tutte le r coordinate selezionate. Per questo motivo la probabilità diventa:

$$\Pr[h(p) = h(q)] = s^r$$

Se $s = 0.9$ e si ha che:

- $r = 2 \rightarrow 0.9^2 = 0.81$
- $r = 5 \rightarrow 0.9^5 = 0.54$

Quindi, aumentando r gli oggetti poco simili smettono quasi completamente di collidere. Il problema è che anche documenti molto simili possono non collidere e si rischia di perderli → Falsi Negativi.

Per ridurre il rumore e controllare il comportamento del sistema, si ripete **b** volte il processo, ottenendo più proiezioni indipendenti $h_1(p), h_2(p), h_3(p), \dots, h_b(p)$.

Si definisce quindi una nuova firma:

$$g(p) = \langle h_1(p), h_2(p), h_3(p), \dots, h_b(p) \rangle$$

Due oggetti vengono considerati candidate match se almeno una delle funzioni produce lo stesso valore:

$$\exists i : h_i(p) = h_i(q)$$

Questo permette di identificare rapidamente coppie potenzialmente simili, riducendo drasticamente il numero di confronti necessari rispetto al confronto diretto tra tutti gli oggetti.

Se si hanno b funzioni indipendenti, la probabilità che nessuna coincida è:

$$(1 - s^r)^b$$

Si aumenta b perché se una funzione hash sbaglia magari la seconda o la terza trovano collisione, riducendo i falsi negativi.

La probabilità che almeno due bande di documenti diversi si trovano nello stesso bucket è pari a:

$$1 - (1 - s^r)^b$$

FAMIGLIE DI FUNZIONI LOCALITY-SENSITIVE HASHING (LSH)

L'idea generale è trovare delle funzioni hash che permettono di individuare coppie di oggetti simili senza calcolare tutte le distanze tra tutte le coppie.

Nel caso precedente si è analizzato il MinHash e stimato tra gli oggetti la similarità di Jaccard: non attraverso gli insiemi ma applicando funzioni hash e trovando le parti comuni delle signature.

Se due oggetti sono simili $A \approx B$ allora è probabile che: $h_i(A) = h_i(B)$.

Minhash è solo una famiglia di funzioni LSH, ma esistono anche altre famiglie per tipi di distanza:

FAMIGLIA	DISTANZA
• Random Projection/ SimHash	distanza coseno
• p-stable LSH	distanza euclidea

Il concetto è sempre lo stesso, ossia si devono creare funzioni hash dove:

- oggetti vicini $\rightarrow$ stesso hash con alta probabilità;
- oggetti lontani $\rightarrow$ stesso hash con bassa probabilità.

Ci sono **3 condizioni** che una famiglia LSH deve rispettare:

- 1) le **coppie vicine** devono avere **più probabilità** di diventare candidate rispetto alle coppie più lontane;
- 2) **indipendenza statistica** $\rightarrow$ le funzioni della famiglia devono fornire informazioni indipendenti. Se una funzione:

$$h_1(A) = h_1(B)$$

non deve automaticamente significare che

$$h_2(A) = h_2(B)$$

altrimenti le funzioni conterrebbero sempre la stessa informazione.

- 3) **efficienza computazionale** $\rightarrow$ si deve evitare il confronto completo tra tutte le coppie, per questo motivo deve essere più efficiente l'utilizzo di LSH. Inoltre, devono essere combinabili dato che si devono evitare falsi positivi e falsi negativi.

Una **Locality-Sensitive function** è una funzione hash che prende due oggetti e decide se sono simili abbastanza da poter essere considerati candidate pair.

In pratica: $f(x)$ prende un oggetto x e restituisce un valore hash.

Se due oggetti hanno lo stesso valore hash: $f(x) = f(y)$ allora sono una coppia candidata.

Una singola funzione può sbagliare e per questo motivo si utilizza una famiglia:

$$\mathbf{F} = \{f_1, f_2, \dots, f_n\}$$

Una famiglia F è:

$$(d_1, d_2, p_1, p_2) - \text{sensitive}$$

Ciò vuol dire che:

- se $d(x, y) \leq d_1$ (distanza di Hamming) allora $P(f(x) = f(y)) \geq p_1$ (probabilità di collisione);
- se $d(x, y) \geq d_2$ (distanza di Hamming) allora $P(f(x) = f(y)) \leq p_2$ (probabilità di collisione);

Quindi deve valere:

$$p_1 > p_2$$

In altre parole, che i vicini collidono più spesso dei lontani e che tipo: se A e B sono simili avranno lo stesso hash più volte rispetto ad A e C che sono più lontani.

Ad esempio, la proprietà generale di MinHash è che per qualsiasi coppia di soglie $0 < d_1 < d_2 \leq 1$ si ha:

$$(d_1, d_2, 1 - d_1, 1 - d_2) - \text{sensitive}$$

Prendendo valori di distanze come $d_1 = \frac{1}{3}$ e $d_2 = \frac{2}{3}$ allora si ha che:

- $P_1(h(x) = h(y)) = 1 - \frac{1}{3}$
- $P_2(h(x) = h(y)) = 1 - \frac{2}{3}$

Pertanto:

$$\left(\frac{1}{3}, \frac{2}{3}, \frac{2}{3}, \frac{1}{3}\right) - \text{sensitive}$$

La funzione prodotta è quella a forma di S o curva morbida, dove non si ha la sicurezza se ricadano nello stesso bucket.

La funzione desiderata è una funzione a gradino perfetto, in altre parole:

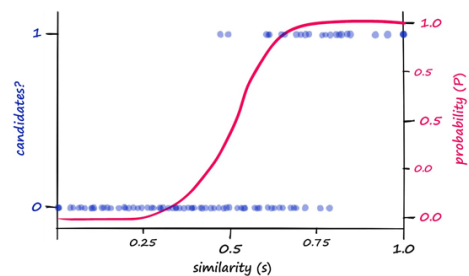
$$P(\text{candidate}) = \begin{cases} 1 & \text{se } d(x, y) < d_0 \\ 0 & \text{se } d(x, y) > d_0 \end{cases}$$

Con una singola funzione hash non si può ottenere perché la probabilità varia in modo graduale.

Il gradino si ottiene con le **costruzioni AND e OR**, dove:

- **AND** → si richiede che tutte le funzioni hash coincidano → abbassa la probabilità per elementi poco simili;
- **OR** → si preano gruppi o bande ed è necessario che una sola coincida → aumenta la probabilità di trovare gli elementi realmente simili.

Combinando AND e OR e aumentando il numero di hash e bande si ottiene una funzione che si assomiglia molto ad un gradino.



AMPLIFICARE LE FUNZIONI LOCALITY-SENSITIVE

L'**amplificazione delle funzioni Locality-Sensitive** consiste nel prendere una funzione LSH "debole" e renderla molto più efficace utilizzando combinazioni di funzioni. L'idea è la stessa delle bande nella MinHash signature matrix.

Supponiamo di avere una famiglia $F = (d_1, d_2, p_1, p_2)$ con $p_1 > p_2$, però la separazione non è abbastanza netta. Si vuole che:

- simili $\rightarrow$ probabilità vicina a 1;
- diversi $\rightarrow$ probabilità vicina a 0.

La **prima tecnica è l'AND** che prevede la costruzione di una nuova famiglia F' dove:

$$f(x) = (f_1(x), f_2(x), \dots, f_r(x))$$

Due oggetti hanno lo stesso hash solo se:

$$f_1(x) = f_1(y) \quad \text{e} \quad f_2(x) = f_2(y) \quad \dots \quad \text{e} \quad f_r(x) = f_r(y)$$

Cioè devono coincidere tutte.

Se ogni funzione ha probabilità p di collisione, le p coincidono e quindi si ha:

$$p^r$$

La nuova famiglia diventa:

$$(d_1, d_2, p_1^r, p_2^r)$$

Aumentando il valore di r la probabilità diminuisce. La probabilità che entrambe le funzioni dicano di "sì" è il prodotto delle probabilità.

La **seconda tecnica è la costruzione di OR**, in altre parole creiamo una nuova funzione usando: **b funzioni**, ma questa volta basta che **una sola** dia **collisione**.

Pertanto:

$$f_i(x) = f_i(y)$$

per almeno un valore di i .

La probabilità di non collidere in tutte le funzioni b è pari a:

$$(1 - p)^b$$

e la probabilità di collidere almeno una volta è:

$$1 - (1 - p)^b$$

La nuova famiglia è:

$$(d_1, d_2, 1 - (1 - p_1)^b, 1 - (1 - p_2)^b)$$

Aumentando b la probabilità cresce, quindi per coppie simili dove p è alto allora la probabilità si avvicina a 1.

Solitamente il procedimento classico è: **AND $\rightarrow$ OR** cioè prima rendiamo difficile la collisione e poi aumentiamo le possibilità con tante bande. In alternativa si possono seguire approcci **OR $\rightarrow$ AND** o alternati tipo: **AND $\rightarrow$ OR $\rightarrow$ AND** o **OR $\rightarrow$ AND $\rightarrow$ OR**.

ALTRE MISURE

Una **distanza è una funzione $d(x,y)$ che misura quanto due oggetti sono lontani**. Per essere una vera metrica di distanza deve rispettare 4 proprietà:

- 1) **non negatività** → la distanza non può essere negativa perché non assume un senso.
- 2) **identità** → se la distanza è zero allora i due oggetti sono uguali.
- 3) **simmetria** → la distanza da A a B è uguale alla distanza da B ad A
- 4) **disuguaglianza triangolare** → $d(x,y) \leq d(x,z) + d(z,y)$ ovvero che la strada diretta non può essere più lunga della strada passando da un punto intermedio.

Distanza di Hamming

La **distanza di Hamming** si usa con i vettori che **misura il numero di posizioni in cui due vettori sono diversi**.

Ad esempio:

$$x = 11001$$

$$y = 00101$$

confrontando posizione per posizione si ha che la distanza di Hamming è uguale a 3.

Per costruire una famiglia LSH per la distanza di Hamming si deve creare una funzione che dica che due vettori sono simili.

Si definisce allora una funzione che prende la componente i-esima del vettore. Riprendendo l'esempio si ha che:

$$f_3(x) = 0$$

perché prende la terza posizione.

Dunque:

$$f_i(x) = f_i(y) \Leftrightarrow x_i = y_i$$

La probabilità di collisione si calcola come segue:

$$P(f(x) = f(y)) = 1 - \frac{h(x,y)}{d}$$

dove:

- $h(x,y)$ è la distanza di Hamming;
- d è la dimensione del vettore.

Conoscendo la probabilità possiamo adesso definire la famiglia nel seguente modo:

$$(d_1, d_2, 1 - \frac{d_1}{d}, 1 - \frac{d_2}{d})$$

In questo caso come nel caso precedente (distanza Jaccard) possiamo dire che:

- distanza piccola → collisione probabile;
- distanza grande → collisione rara.

Distanza del Coseno

Supponiamo di avere due vettori: $A = (a_1, a_2, \dots, a_n)$ e $B = (b_1, b_2, \dots, b_n)$. La **cosine similarity** misura quanto due vettori puntano nella stessa direzione:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

dove:

- $A \cdot B$ è il **prodotto scalare**;
- $\|A\|$ è la **lunghezza del vettore**;
- θ è l'**angolo tra i due vettori**.

La distanza del coseno è direttamente l'angolo $d(A, B) = \theta$. Perciò:

- Vettori con la stessa direzione, sono simili $\rightarrow \theta = 0$;
- Vettori perpendicolari, sono poco simili $\rightarrow \theta = 90^\circ$;
- Vettori opposti, sono completamente diversi $\rightarrow \theta = 180^\circ$.

Possiamo approssimare dicendo che: se la distanza è piccola l'angolo è molto piccola ergo i vettori sono simili. Per creare una famiglia LSH si utilizzano degli iperpiani casuali, che singolarmente dividono lo spazio in due metà.

In due dimensioni è semplicemente una retta che separa il piano in due regioni: una parte positiva e una parte negativa.

Il **vettore normale** è un vettore ha una direzione perpendicolare all'iperpiano:



Come si può osservare nell'immagine il vettore normale punta verso una delle due metà. Si ha un punto(vettore) x , il prodotto scalare è:

$$v \cdot x$$

e misura quanto x è orientato nella direzione di v .

La formula è:

$$v \cdot x = |v| |x| \cos(\theta)$$

dove θ è l'angolo tra v e x .

Sappiamo che:

- per $\theta < 90$ allora $v \cdot x > 0 \rightarrow$ il vettore punta più o meno nella stessa direzione di v ;
- per $\theta = 90$ allora $v \cdot x = 0 \rightarrow$ il vettore si trova nell'iperpiano $\rightarrow$ è il punto di confine;
- per $\theta > 90$ allora $v \cdot x < 0 \rightarrow$ il vettore si trova nella direzione opposta rispetto a v .

Ricorda che i risultati sono questi perché c'è $\cos(\theta)$.

La funzione hash fa solo questo controllo:

$$h_v(x) \begin{cases} +1 & \text{se } v \cdot x \geq 0 \\ -1 & \text{se } v \cdot x < 0 \end{cases}$$

Se due vettori sono vicini(angolo piccolo), è difficile che un iperpiano casuale cada proprio tra loro. Quindi:

- angolo piccolo → stesso lato ergo $h(x) = h(y)$;
- angolo grande → lato diverso più probabile ergo $h(x) \neq h(y)$.

La signature sarà del tipo: $(-1,+1,+1,..., -1)$ ed è equivalente della firma MinHash, solo che ogni bit della firma è ottenuto da un iperpiano casuale anziché da una permutazione.

Per il Random Hyperplane(che è una famiglia di funzioni LSH per cosine distance) si ha che la probabilità è la seguente:

$$P(h(x) = h(y)) = 1 - \frac{\theta(x, y)}{\pi}$$

dove $\theta(x, y)$ è la distanza coseno espressa come angolo.

Procedendo come per MinHash si ha che:

$$0 < d_1 < d_2 \leq \pi$$

infatti: $p_1 = 1 - \frac{d_1}{\pi}$ e $p_2 = 1 - \frac{d_2}{\pi}$.

Infine, possiamo ricavare la formula della famiglia LSH che è:

$$(d_1, d_2, 1 - \frac{d_1}{\pi}, 1 - \frac{d_2}{\pi})$$

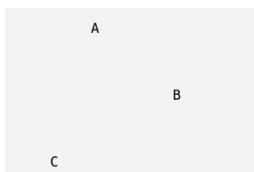
Distanza Euclidea

Quando si lavora con la distanza euclidea si considera sempre un vettore di numeri reali. La formula generale(o norma) è:

$$d(x, y) = (\sum_{i=1}^n |x_i - y_i|^r)^{1/r}$$

In base al valore di r , si sceglie una distanza diversa. Infatti:

- $r = 1$ → distanza di Manhattan;
- $r = 2$ → distanza euclidea;
- altre distanze.

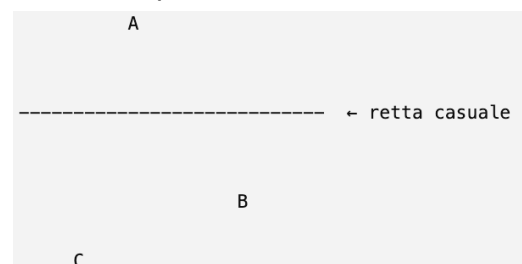
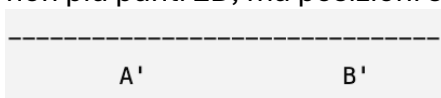


Considerando di avere dei punti in un piano, la distanza euclidea misura la distanza vera tra i punti:

$$d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

Ora si vuole scegliere una retta casuale nello spazio:

Questa retta è come se fosse l'asse di partenza. Ora si vuole schiacciare ogni punto sulla retta ottenendo non più punti 2D, ma posizioni su una linea 1D.



Ora si vuole dividere la retta ottenuta in pezzi di lunghezza a:



Ogni intervallo è un **bucket** e la funzione hash è **$h(x)$ = numero del bucket in cui cade la proiezione di x** .

Pertanto, se dopo la proiezione A' e B' cadono nello stesso intervallo allora $h(A) = h(B)$.

Perché funziona? Perché due punti vicini nello spazio originale rimangono probabilmente vicini anche dopo la proiezione.

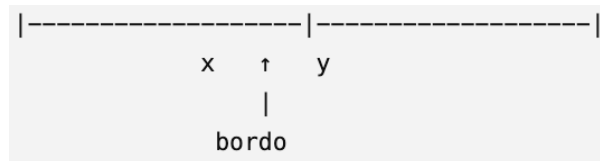
La famiglia LSH è:

$F = \{h_{linea,a}\}$ o $F = \{h_1, h_2, \dots, h_n\}$, cioè tutte le possibili funzioni ottenute scegliendo rette casuali diverse.

Perché si utilizza la distanza euclidea? Per capire quanto due punti lontani possono ricadere sullo stesso bucket. Si può calcolare la distanza proiettata attraverso la seguente formula:

$$d_{proiettata} = d(x, y) \cos(\theta)$$

Due punti molto vicini possono cadere in bucket diversi, ossia che il confine del bucket deve cadere proprio nello spazio tra loro.



La **probabilità che il bordo del bucket cada** proprio tra x e y è:

$$\frac{d}{a}$$

dove: d è la distanza tra x e y, a è la lunghezza del bucket.

La probabilità che il bordo non cada tra loro è:

$$1 - \frac{d}{a}$$

I punti considerati vicini sono quelli a distanza $d < \frac{a}{2}$ e la probabilità che cadono nello stesso bucket è $p(h(x) = h(y)) \geq \frac{1}{2}$.

I punti considerati lontani (di cui si ha garanzia) sono quelli a distanza $d > 2a$ e la probabilità che cadono nello stesso bucket è di $p(h(x) = h(y)) \leq \frac{1}{3}$.

Tutto quello che ricade tra $\frac{1}{2}a$ e $2a$ cade nella zona dove non si ha alcuna garanzia se siano punti vicini o lontani.

Quindi la formula generale della famiglia LSH per la distanza euclidea è:

$$\left(\frac{1}{2}a, 2a, \frac{1}{2}, \frac{1}{3}\right)\text{-sensitive}$$

RIDUZIONE DELLA DIMENSIONALITA'

La rivoluzione della **riduzione della dimensionalità** è quella di **rappresentare i dati con meno variabili**(dimensioni), pur mantenendo quasi tutte le informazioni più importanti.

In big data molti problemi possono essere rappresentati con una matrice. Ad esempio:

- Web → matrice di transizioni tra pagine (PAGERANK);
- Sistemi di raccomandazione → matrice utenti x prodotti;
- Social Network → matrice di adiacenza tra utenti.

Prendendo l'esempio i sistemi di raccomandazione e si pensa di avere:

- ◆ 10 milioni di utenti;
- ◆ 500 mila film.

Si lavora con circa $10^7 \cdot 5 \cdot 10^5 = 5 \cdot 10^{12}$ ossia 5 trilioni di celle, il che si dimostra estremamente lento e complesso.

Molto spesso i dati hanno delle ridondanze e per siffatto motivo possiamo **trovare una matrice più piccola che approssimi la matrice originale**. La nuova rappresentazione, ossia la matrice ristretta, deve conservare gran parte dell'informazione.

Le matrici ristrette si ottengono mediante la riduzione della dimensionalità, cioè algoritmi che:

- 1) Prendono $x = (x_1, x_2, x_3, \dots, x_n)$ in input;
- 2) Restituiscono $z = (z_1, z_2, z_3, \dots, z_k)$.

con $n \gg k$.

Si riduce la dimensionalità perché:

- alcune feature possono essere irrilevanti;
- molte feature sono ridondanti → ad esempio l'altezza in metri e centimetri;
- è difficile visualizzare graficamente 100 dimensioni, però se la rappresentazione è 2d o 3d è molto più semplice.

L'**unsupervised feature selection** è una tecnica di **feature extraction**. A differenza della feature selection, non si limita a selezionare le feature ma vengono create nuove feature definite come funzioni di tutte le feature.

FEATURE SELECTION	UNSUPERVISED FEATURE SELECTION
La feature selection sceglie alcune feature esistenti. Ad esempio: - ALTEZZA ✓; - PESO ✓; - ETA' ✗; - REDDITO ✗. Le informazioni rimangono esattamente le stesse ma alcune non vengono prese in considerazione.	Dalle feature precedenti si ricavano nuove colonne. Ad esempio: - ALTEZZA ✓; - PESO ✓; - ETA' ✓; - REDDITO ✓. $z_1 = 0.8 \cdot ALTEZZA + 0.3 \cdot ETA'$ $z_2 = 0.6 \cdot REDDITO - 0.3 \cdot PESO$ Queste feature non esistevano prima, sono combinazioni delle precedenti.

Dato che il metodo è unsupervised allora non si considerano le etichette delle classi, infatti vengono ignorate le variabili qualitative. Si guardano solo dove i punti si trovano nello spazio.

Anche se i dati sembrano avere tantissime dimensioni, in realtà occupano una piccolissima porzione dello spazio.

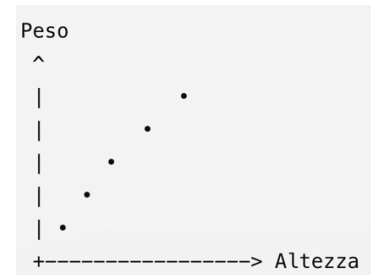
Gli assi del sottospazio effettivamente occupato sono l'effettiva rappresentazione dei dati; infatti, **vengono creati nuovi assi che rappresentano meglio la distribuzione dei dati.**

Ad esempio:

l'asse migliore non è né orizzontale e né verticale è inclinato e descrive meglio la distribuzione.

La PCA ruota gli assi fino ad allinearli con la direzione in cui i dati variano di più.

Questa nuova direzione diventa la prima componente principale.



Il concetto matematico alla base della riduzione della dimensionalità è il rango della matrice. Il rango di una matrice è il numero massimo di colonne/righe linearmente indipendenti, ossia che nessun vettore della matrice può essere ottenuto combinando gli altri.

Il rango si utilizza per capire quante direzioni servono per descrivere i dati. In questo modo si individuano gli assi su cui giacciono i dati e si ruotano affinché diventino gli assi principali.

Ci sono degli errori dovuti all'approssimazione e la PCA cerca di minimizzare l'errore.

Dall'utilizzo della PCA si possono ottenere vantaggi come:

- scoprire correlazioni nascoste tra i dati;
- rimuovere feature ridondanti e rumorose;
- visualizzare in 2d/3d e capire se ci sono: anomalie, cluster o relazioni che nello spazio originali sarebbero impossibili da vedere.

Infine, anche se i dati sono descritti da molte feature, spesso la loro struttura reale può dipendere anche da poche direzioni fondamentali.

Unsupervised Feature Selection

L'idea della riduzione della dimensionalità è: dato un insieme d-dimensionale, proiettare i dati in uno spazio con meno dimensioni preservando quante più informazioni possibili.

Si sceglie la **proiezione che minimizza il quadrato dell'errore** quando si ricostruiscono i dati. L'**errore** è la **distanza tra il punto originale e il punto proiettato.**

Si utilizza il quadrato per svariati motivi:

- evita che sommando errori positivi e negativi si annullino;
- penalizza gli errori molto grandi;
- otteniamo un problema di ottimizzazione con una funzione matematica ben definita, che permette di utilizzare al meglio gli strumenti matematici.

La **Principal Component Analysis** cerca le direzioni lungo le quali si muovono i dati attraverso gli autovalori.

Una **matrice** può essere vista come una **trasformazione geometrica**, ossia che moltiplicata per un vettore, la matrice può modificare il vettore nei seguenti modi:

- allungarlo;
- accorciarlo;
- ruotarlo;
- cambiarne la direzione.

Ad esempio:

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix} \cdot \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

Il vettore è stato allungato, infatti, è diventato due volte più lungo.

Quando si moltiplica la matrice all'autovettore, il risultato è l'autovettore stesso ma moltiplicato per un numero. Quindi:

$$Me = \lambda e$$

dove:

- ♦ **M** è la **matrice**;
- ♦ **e** è l'**autovettore**;
- ♦ **λ** è l'**autovalore**.

L'**autocoppia** è semplicemente **(λ , e)** che soddisfano $Me = \lambda e$.

Perché sono speciali? Perché normalmente cambia la direzione del vettore, ma quando non cambia si dice autovettore.

L'autovalore dice quando cambia la lunghezza dell'autovettore, e può cambiare pure il verso se ha valore negativo.

Come si collega alla PCA?

Nella PCA si ha una matrice di dati, da cui si ricava la **matrice di covarianza** ossia la matrice contenente la correlazione tra feature. Gli **autovettori di questa matrice indicano le direzioni principali** lungo cui si distribuiscono i dati e gli **autovalori** indicano quanta **variabilità c'è in ogni direzione**.

La PCA non è interessata alla lunghezza del vettore, ma solo alla direzione. Per questo motivo si assume sempre che l'autovettore sia un vettore unitario, ossia $\|e\| = 1$.

Matematicamente si trovano dalla seguente definizione:

$$Me = \lambda e$$

$$Me - \lambda e = 0 \quad \text{si può riscrivere } \lambda e = \lambda I e$$

$$Me - \lambda I e = 0$$

$$(\mathbf{M} - \lambda \mathbf{I})\mathbf{e} = \mathbf{0}$$

Dunque, si cerca per quei valori di λ per cui la matrice $M - \lambda I$ ha una soluzione diversa da 0. Per trovare questi valori si usa:

$$\det(M - \lambda I) = 0$$

Questa equazione permette di trovare tutti gli autovalori. Ad esempio:

$$\begin{aligned} \det\left(\begin{vmatrix} 2 & 1 \\ 1 & 3 \end{vmatrix} - \begin{vmatrix} \lambda & 0 \\ 0 & \lambda \end{vmatrix}\right) \\ = \det\left(\begin{vmatrix} 2-\lambda & 1 \\ 1 & 3-\lambda \end{vmatrix}\right) \\ = (2-\lambda)(3-\lambda) - (1 \cdot 1) \\ = 6 + \lambda^2 - 2\lambda - 3\lambda - 1 \\ = \lambda^2 - 5\lambda + 5 \end{aligned}$$

Dato che si è detto che il determinante deve essere posto pari a 0, si ottiene che:

$$\lambda^2 - 5\lambda + 5 = 0$$

Si risolve l'equazione di secondo grado e si trovano i due autovalori.

Notiamo che:

- Se la matrice è $2 \times 2 \rightarrow$ il polinomio da risolvere è di grado 2;
- Se la matrice è $3 \times 3 \rightarrow$ il polinomio da risolvere è di grado 3;
-

In generale otteniamo fino a n autovalori.

Pivotale di Chio

Calcolare direttamente il determinante di una matrice $n \times n$ usando la definizione classica sarebbe costoso; infatti, la definizione del determinante contiene infatti un numero enorme di termini: $n!$

Si può calcolare il determinante in modo più efficiente, con algoritmi che permettono di calcolare in $O(n^3)$.

Un algoritmo che permette di calcolare con la complessità citata su è **la condensazione pivotale di Chio**.

L'idea generale è: **trasformare il problema del determinante di una matrice $n \times n$ in quello del determinante di una matrice più piccola $(n-1) \times (n-1)$.**

Possiamo ricorsivamente fare queste trasformazioni fino ad avere matrici 1×1 o 2×2 . Ovviamente il determinante della matrice più piccola non corrisponderà al determinante della matrice di partenza.

Il punto di partenza attorno al quale si costruisce la matrice più piccola nella condensazione. Il determinante si calcola nel seguente modo:

$$\det(A) = \frac{\det(B)}{a_{11}^{n-2}}$$

dove: a_{11} è il **pivot**.

Power Iteration

Avendo una matrice M si vuole trovare l'autocoppia principale (λ_1, e_1) , cioè:

- ♦ e_1 l'autovettore associato al più grande autovalore;
- ♦ λ_1 l'autovalore più grande in valore assoluto.

L'autocoppia ovviamente deve soddisfare $Me = \lambda e$.

Il problema è che trovare tutti gli autovalori può essere costoso, soprattutto se M è grande.

La **Power Iteration** permette di trovare approssimativamente l'autovettore principale in modo molto più semplice.

Si parte da un vettore qualsiasi non nullo, ovvero $x_0 \neq 0$ e si continua a moltiplicare per M . Si ottiene quindi:

$$x_1 = Mx_0$$

$$x_2 = Mx_1$$

...

$$x_{k+1} = Mx_k$$

Dopo abbastanza iterazioni, la direzione di x_k tende verso quella dell'autovettore principale. Infatti, ci fermiamo quando il vettore non cambia più, ad esempio quando:

$$\|x_{k+1} - x_k\| < \varepsilon \quad \text{con } \varepsilon \text{ molto piccolo (ossia il vettore è convergente)}$$

La differenza si calcola mediante la **Norma di Frobenius**, ossia una norma che permette di applicare la norma euclidea ad una matrice:

$$\|M\|_F = \sqrt{\sum_{ij} M_{ij}^2}$$

Infine, una volta trovato l'autovettore principale x , si deve trovare l'autovalore. Questo vale se x è normalizzato, ossia:

$$x^T x = 1$$

Per capire perché, partiamo dalla definizione:

$$Mx = \lambda x$$

Moltiplichiamo a sinistra per x^T :

$$x^T Mx = \lambda x^T x$$

Ma x è normalizzato perché $x^T x = 1$ quindi:

$$x^T Mx = \lambda$$

C'è una condizione importante, ovvero che ci deve essere un autovalore dominante:

$$|\lambda_1| > |\lambda_2|$$

cioè il primo autovalore deve essere strettamente maggiore in valore assoluto del secondo.

PCA

Come già accennato nei paragrafi precedenti la PCA è uno dei metodi principali per fare dimensionality reduction. Infatti, si cerca un numero minore di dimensioni che permetta di rappresentare i dati perdendo meno informazione possibile.

Si trovano le direzioni lungo le quali sono presenti più punti dati e le direzioni che contengano la massima varianza. La massima varianza perché i punti sono molto dispersi e ciò vuol dire che quella direzione contiene molta informazione sulla struttura del dataset.

La direzione si trova nel seguente modo:

- 1) **costruiamo la matrice di covarianza** dei dati $\Sigma = \frac{1}{N-1} \mathbf{M}^T \mathbf{M}$;
- 2) **calcoliamo gli autovalori e gli autovettori** $\Sigma e = \lambda e$;
- 3) **viene sottratta la media da ogni vettore**, spostando il centro della nuvola dei dati nell'origine.

Soffermandoci sull'ultimo punto, possiamo dire che **la PCA non è interessata a sapere dove si trova la nuvola nello spazio ma alla sua forma e alle sue direzioni di variazione.**

Tutti gli autovettori trovati sono messi in colonne di una matrice:

$$E = \begin{bmatrix} | & | & | \\ e_1 & | & e_n \\ | & | & | \end{bmatrix}$$

Questa matrice contiene tutti i nuovi assi o le direzioni. Se moltiplicassimo la matrice M(dati originali) e la matrice E si otterrebbe:

$Z = ME \rightarrow$ contenente tutti i dati espressi nelle nuove coordinate(rappresentazione PCA)

Singular Value Decomposition(SVD)

La **Singular Value Decomposition(SVD)** è una tecnica algebrica che permette di scomporre una matrice complessa di dati A in tre matrici più semplici e interpretabili:

$$A = U\Sigma V^T$$

Dove:

- **A** è la **matrice di partenza m x n**;
- **U** **matrice m x r che collega le righe originali ai concetti latenti r**. Ogni riga di U descrive un elemento originale(ad esempio un utente) attraverso i r concetti latenti;
- **Σ** è una **matrice r x r**, dove **lungo la diagonale principale sono presenti i valori singolari**. I valori singolari sono le radici degli autovalori;
- **V^T** è una **matrice r x n** derivata dalla matrice V. La **matrice V contiene le righe che raffigurano le colonne di A rispetto ai r concetti latenti**.

Ad esempio, se A contenesse utenti x film avremmo che:

- **U** → rappresenta come gli utenti si collocano rispetto ai concetti latenti;
- **Σ** → quanto sono importanti i concetti;
- **V** → come i film si collocano rispetto ai concetti latenti.

Questa tecnica riduce la dimensionalità dei dati estraendo i “concetti latenti”(fattori nascosti) sottostanti.

La matrice A può essere espressa come somma di matrici di rango 1. Infatti:

$$A = \sum_i \sigma_i u_i v_i^T$$

dove:

- ◆ σ_i → i-esimo valore singolare;
- ◆ u_i → i-esima colonna di U;
- ◆ v_i → i-esima colonna di V;

Il prodotto è una matrice di rango 1(salvo il caso di vettori nulli), e la costruzione di A si fa attraverso tante piccole matrici.

v_i e u_i sono ortonormali perché rappresentano direzioni indipendenti e normalizzate dello spazio. Di fatto, se $u_1^T u_2 \neq 0$ si avrebbero due direzioni che in parte si sovrapporrebbero e non sarebbero componenti completamente indipendenti.

La SVD non serve solo a scomporre una matrice, ma rappresenta i dati con meno variabili e introducendo il minimo errore possibile.

Invece di utilizzare D coordinate per ogni punto, proiettiamo lungo il primo vettore v_1 . L'asse v_1 viene scelto per minimizzare la somma degli errori di ricostruzione al quadrato.

$$\min \|A - B\|_F^2 = \sum_{i=1}^N \sum_{j=1}^D \|x_{ij} - z_{ij}\|^2$$

Le coordinate dei punti sul nuovo spazio ridotto sono date dal prodotto $U\Sigma$.

Per ridurre le dimensioni di una matrice A, **si azzerano i valori singolari più piccoli** e si rimuovono le righe/colonne corrispondenti in U e V^T .

Quale rappresentazione a k dimensioni perde meno informazioni possibile?

L'errore si misura con:

$$\|A - B\|_F^2 \text{ (*norma di Frobenius*)}$$

Il teorema sulla migliore approssimazione (Eckart-Young) dice: se voglio sostituire una matrice A con una matrice più semplice di rango k, la SVD dice esattamente quale scegliere per commettere il primo errore possibile.

Se tengo solo i primi k valori singolari, costruisco:

$$B = A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$$

Quindi, si costruisce seguendo i primi k valori singolari la matrice di approssimazione di rango k.

Solitamente si sceglie k che permette di conservare l'80-90% dell'energia totale. Per calcolare l'energia conservata possiamo utilizzare la seguente formula:

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq 80\% - 90\%$$

Come si trovano concretamente U, Σ , V partendo da A? La strategia è trasformare il problema della SVD in un problema che sappiamo già risolvere: la decomposizione agli autovalori.

Partiamo da:

$$A = U \Sigma V^T$$

Facciamo il prodotto tra $A^T A$:

$$A^T A = (U \Sigma V^T)^T (U \Sigma V^T)$$

Usando le proprietà della trasposta:

$$A^T A = V \Sigma^T U^T U \Sigma V^T \quad \text{sapendo che } U^T U = I$$

rimane:

$$A^T A = V \Sigma^T \Sigma V^T$$

Dato che una matrice diagonale per la sua trasposta da D^2 si arriva ad una forma standard della decomposizione agli autovalori di una matrice simmetrica.

La forma è la seguente $M = Q \Lambda Q^T$. Ponendo $Q = V$ e $\Lambda = \Sigma^T \Sigma$ possiamo quindi ricavare V e Σ , infatti poi basta applicare la radice quadrata agli elementi di Λ .

Si replica questo passaggio anche per calcolare U.

Come algoritmo pratico si utilizza il Power Iteration su $A^T A$ per trovare la prima autocoppia (x, λ) . Si sottrae l'effetto trovando la matrice residua $M' = A^T A - \lambda x x^T$ e si riapplica il processo per i successivi autovalori/autovettori.

La complessità è $O(\min(nm^2, n^2m))$ ossia quando può costare calcolare la SVD completa di una matrice $m \times n$. Però molte volte non è utile calcolare tutta la SVD ma solo i primi k valori singolari.

La complessità per il calcolo della matrice per i primi k valori singolari è proprio ciò che vogliamo nella riduzione dimensionale.

Uno dei più grandi vantaggi della SVD nei sistemi di raccomandazione è la capacità di superare il problema del “matching sintattico” traducendo tutto nel “matching semantico”.

I vantaggi nell'utilizzo sono:

- **Migliore Approssimazione a Basso Rango** → Garantisce matematicamente la minima perdita di informazione rispetto alla norma di Frobenius $\|A - B\|_F$.
- Risoluzione della Sinonimia e Polisemia → Raggruppa i dati in base al significato sottostante anziché i singoli elementi.

I limiti sono:

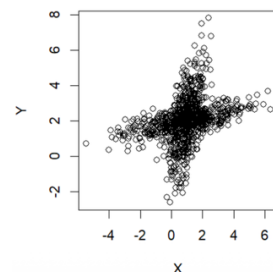
- Difficoltà di Interpretazione;
- **Perdita della sparsità** → anche se la matrice di partenza è altamente sparsa, le matrici risultanti sono sparse.

DECOMPOSIZIONE CUR

I metodi di riduzione della dimensionalità classici, tipo SVD e PCA, cercano direzioni ortogonali che massimizzino la varianza o minimizzino l'errore di ricostruzione.

Tuttavia, presentano tre grossi problemi in contesti reali di Big Data:

- Perdita della sparsità → le matrici che si ottengono sono molto dense;
- Difficoltà di interpretazione → un vettore singolare non rappresenta un elemento reale, ma un'astrazione matematica;
- **Problemi geometrici con distribuzioni a croce** → quando i dati si sviluppano lungo assi non ortogonali.



La decomposizione CUR esprime una matrice $A \in \mathbb{R}^{m \times n}$ come il prodotto di tre matrici:

$$A = C \cdot U \cdot R$$

dove:

- **C**(columns, $m \times c$) è una **matrice contenente le c colonne reali campionate dalla matrice di partenza**;
- **R**(rows, $r \times n$) è una **matrice contenente le r righe reali campionate dalla matrice di partenza**.
- **U**(Intersezione, $c \times r$) una **piccola matrice densa che funge da collegamento o pseudoinversa dell'intersezione tra le colonne e le righe campionate**.

La CUR è vantaggiosa perché:

- Mantiene la sparsità: poiché C e R sono sottoinsiemi diretti di A, conservando gli zeri originali;
- Interpretabilità immediata: i vettori della base sono righe/colonne reali del problema.

Algoritmo di Campionamento e Calcolo di U

Per scegliere righe e colonne per minimizzare l'errore $\|A - CUR\|_F$ si introducono degli algoritmi di campionamento, ossia:

- **ColumnSelect** → le colonne non vengono scelte a caso, ma in base al loro peso energetico, infatti:

per ogni colonna $x \in \{1, \dots, n\}$ si calcola la probabilità di selezione $P(x)$ con

$$P(x) = \frac{\sum_i A(i,x)^2}{\sum_{i,j} A(i,j)^2} = \frac{\|A_{*,x}\|^2}{\|A\|_F^2}$$

si estraggono c Colonne con reinserimento in base alla distribuzione $P(x)$.

Intuitivamente, una colonna con valori molto grandi contiene più energia.

Per le righe si fa la stessa cosa.

- **Riscalamento** → Non possiamo mettere le colonne direttamente dentro C , perché alcune colonne hanno più probabilità di estrazione molto maggiore rispetto altre. La formula per il riscalamento è:

$$C(:,i) = \frac{A(:,j)}{\sqrt{cP(j)}}$$

$\sqrt{cP(j)}$ serve a correggere il bias introdotto dal campionamento.

Quindi, se compare una colonna la riscalo in base alla sua probabilità. Il risultato è una stima statisticamente corretta della matrice originale.

- **Calcolo della Matrice U** → Per calcolare la matrice U si devono seguire i seguenti passaggi:

1. Si parte da W ossia la matrice $c \times r$ nata dall'intersezione tra le colonne scelte in C e le righe scelte in R ;
2. Si calcola la SVD di $W = XYZ^T$;
3. Si calcola la pseudoinversa di Moore-Penrose W^+ , ponendo:
$$\begin{cases} 1/Z_{ii} & \text{se } Z_{ii} \neq 0 \\ 0 & \text{se } Z_{ii} = 0 \end{cases}$$
4. Si imposta $U = W^+ = YZ^+ X^T$.

Teorema di Garanzia Teorica e Gestione dei Duplicati

Scegliendo abbastanza righe e colonne della matrice A in modo appropriato, allora CUR è garantito, con alta probabilità, di produrre un'approssimazione non troppo peggiore della migliore approssimazione possibile di rango k .

Quante righe e colonne si devono scegliere?

$$c, r = O\left(\frac{k \log k}{\epsilon^2}\right)$$

Se si sceglie un numero sufficiente di righe r e colonne c allora si ottiene quella garanzia.

Un particolare problema del campionamento con probabilità è la selezione delle righe/colonne con reintroduzione. Infatti, una colonna può essere estratta più volte e questo vuol dire che esiste uno spreco di memoria.

Perciò, la soluzione è la seguente:

$$C_s = [\sqrt{n}c_i \ c_j]$$

dove n è il numero di volte che la colonna c_i viene presa in considerazione.

CUR è interessante rispetto alla SVD, perché avendo una matrice enorme e molto sparsa, dove la maggior parte degli elementi è zero, si comporta in modo più efficiente.

Difatti, se SVD fa diventare i fattori densi, anche se la matrice di partenza A è sparsa; invece, la CUR prendendo direttamente colonne e righe di A rimarrà sparsa.

Inoltre, la CUR richiede pochissimo spazio di memoria e si calcola in una frazione del tempo minore.

NoN-Negative Matrix Factorization

La **Non-Negative Matrix Factorization** (fattorizzazione di matrici non negative) permette di cercare due matrici più piccole W e H tali che:

$$A \approx WH \quad \text{con il vincolo fondamentale } W \geq 0 \text{ e } H \geq 0$$

Più chiaramente, la NMF sta cercando di scomporre una grande matrice in due matrici più piccole senza mai utilizzare numeri negativi.

La matrice W contiene i componenti o caratteristiche fondamentali presenti nei dati, mentre H indica quanto ciascun componente è presente in ogni osservazione.

La NMF produce spesso una rappresentazione per “parti”, più facilmente interpretabili.

Si parte solitamente su una matrice A che contiene il numero di osservazioni che ad esempio compaiono nei documenti. La NMF può trovare:

$$W = \text{topic} \quad H = \text{peso di un topic (in un documento)}$$

Quindi, **W scopre quali sono i topic fondamentali** mentre **H dice quanto ogni topic caratterizza il documento**. H indica quanto un topic caratterizza il documento.

La matrice A è la matrice dati originali con m features/parole e n osservazioni/documenti.

La matrice W è la matrice dizionario o dei vettori base ed è una $m \times K$ dove k sono i Topic e la matrice H è la matrice di attivazione/pesi e indica la presenza/importanza dei Topic nei vari documenti.

Il **rango K** viene **scelto** in modo tale che $(m+n)K \ll m*n$ per garantire la compressione.

Possiamo fare un esempio:

si hanno le righe → governo, presidente della repubblica, sinistra, fascisti(merda), solipsismo, nichilismo e esistenzialismo.

K potrebbe essere $K = 2$ con: politica e filosofia. Poi nei documenti si fa il conteggio quante volte compare un elemento appartenente a politica e filosofia, ottenendo ad esempio:

- Documento 1: 0.8 politica + 0.2 filosofia;
- Documento 2: 0.6 politica + 0.4 filosofia.

Questo costringe l’algoritmo ad apprendere una scomposizione basata sulle parti componenti dell’oggetto e la non-negatività induce naturalmente ad avere sparsità nei dati.

L’approssimazione $A \approx WH$, viene calcolata minimizzando una funzione di distanza, esempio quella euclidea. Il fattore di correzione permette di raggiungere l’approssimazione descritta.

$$H_{a\mu} \leftarrow H_{a\mu} \frac{\sum_i W_{ia} \frac{A_{i\mu}}{(WH)_{i\mu}}}{\sum_i W_{ia}}$$

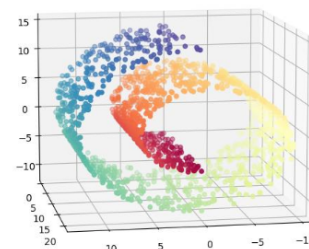
Riduzione Non-Lineare della Dimensionalità

Tutte le tecniche trattate finora cercano una rappresentazione più piccola dei dati usando una struttura sostanzialmente lineare.

Quindi, se i punti sono distribuiti più o meno su un piano possiamo trovare un piano che li rappresenta abbastanza bene.

Se i punti formano una superficie arrotolata, ad esempio uno Swiss Roll, stanno su una superficie che è curva.

Nello spazio 3D alcuni punti si trovano vicini perché si trovano uno sopra l'altro, ma se si vuole camminare lungo la superficie, si deve fare un percorso molto più lungo.



Per calcolare la distanza seguendo la superficie, si utilizza la **distanza geodetica**:

$$D_G(A, B)$$

ossia il percorso più breve tra due punti situati su una superficie curva.

Non si utilizza la distanza euclidea perché ci potrebbe far credere che due punti siano vicini quando in realtà sono molto lontani considerando la struttura dei dati.

Isomap(Isometric Feature mapping) risolve il problema, ossia che se non si può misurare direttamente la distanza lungo la superficie, prova a ricostruire la superficie con i punti a disposizione.

La ricostruzione avviene in 3 passaggi:

- 1) **si crea un grafo con il k-nearest neighbors** → dove ogni punto viene collegato ai punti vicini. I punti vengono considerati vicini non nello spazio 3D ma **punti che appartengono a parti diverse del rotolo**.
- 2) **si calcolano le distanze geodetiche** → si calcola la distanza dei nodi per andare da A a B e poi si sommano le distanze. Il percorso più corto viene chiamato cammino minimo. Per trovare i cammini minimi si utilizzano algoritmi come Dijkstra.
- 3) **si devono rappresentare i punti in 2D mantenendo le distanze** → si utilizza il **MDS**(multidimensional scaling) che cerca una rappresentazione a bassa dimensione che mantenga il più possibile le distanze che si sono calcolate.

Non è lineare perché si sta cercando di capire la geometria della superficie su cui si trovano i dati. Infatti, Isomap quello che fa è capire bene la superficie curva e poi srotolarla.

Il **t-distributed Stochastic Neighbor Embedding(t-SNE)** ha un obiettivo un po' diverso non gli interessa mantenere tutte le distanze, gli **interessa mantenere le vicinanze**.

t-SNE cerca di fare in modo che: **punti vicini nello spazio originale rimangano punti vicini nello spazio ridotto**. Così si ottengono dei cluster ben separati ed è molto utilizzato per visualizzare dati ad alta dimensionalità.

È stocastico perché **trasforma la vicinanza tra punti in una probabilità**, infatti, queste probabilità vengono calcolate nello spazio originale.

Per **misurare quanto le due distribuzioni differiscono** si utilizza la: **divergenza KL** e si cerca di minimizzarla.

Nello spazio ridotto t-SNE usa una distribuzione t di Student per rappresentare le distanze tra i punti. Questa scelta aiuta a evitare che, quando si mettono moltissimi punti in sole 2 dimensioni, finiscano schiacciati tutti insieme.

UMAP(uniform manifold approximation and projection) è una tecnica simile a t-SNE, ma che **cerca di mantenere sia i vicini sia la struttura generale dei dati.**

Supponendo di avere dei dati con 100 dimensioni, UMAP cerca di trasformarli in 2 dimensioni in modo da poterli visualizzare. UMAP cerca innanzitutto di mantenere il fatto che i punti vicini rimangono vicini, e questo riguarda la **struttura locale.**

Inoltre, cerca di mantenere una certa informazione su come i gruppi sono disposti gli uni rispetto agli altri, e questo riguarda la **struttura globale.**

SISTEMI DI RACCOMANDAZIONE

Un sistema di raccomandazione è un'applicazione che cerca di prevedere cosa potrebbe interessare a un determinato utente e, sulla base della previsione gli propone gli elementi.

Il sistema non sa con certezza che la raccomandazione fatta è giusta. Fa una previsione sulla base delle preferenze conosciute e raccomanda un prodotto/servizio.

Ad esempio, l'algoritmo di Spotify se vede che ascolti:

- the strokes;
- deftones;
- cigarettes after sex;

potrebbe suggerire i radiohead.

Esistono due grandi famiglie di sistemi di raccomandazione che differiscono tra loro per la fonte dell'informazione. Queste famiglie sono:

- **sistemi content-based** → guardano le caratteristiche dell'oggetto che potrebbe interessarti → non guardano principalmente cosa hanno fatto altri utenti, ma il contenuto/caratteristiche degli elementi.
Il sistema **analizza le caratteristiche degli articoli** e costruisce una sorta di **profilo degli interessi**, dopodiché raccomanda un nuovo elemento.
- **collaborative filtering** → il sistema non si concentra principalmente sulle caratteristiche dell'oggetto ma su cosa hanno fatto gli altri utenti. Infatti, **utenti con comportamenti simili probabilmente avranno preferenze simili** anche in futuro.

Si chiama "Collaborative" perché sfrutta la collaborazione implicita tra gli utenti, ossia le interazioni come:

- like;
- visualizzazioni;
- recensioni;

Da queste informazioni cerca di trovare pattern comuni.

Nel **collaborative filtering**, inoltre, **si possono utilizzare misure di similarità tra utenti e/o prodotti per effettuare raccomandazioni**.

Perciò, se si ha che $UserA \approx UserB$ allora se A ha apprezzato X consiglio X a B.

I sistemi di raccomandazione soprattutto nel web sono divenuti importanti.

Infatti, in un negozio fisico lo spazio è limitato e quindi il negozio deve scegliere quali prodotti tenere. Una grande quantità di prodotti poco popolari non viene esposto.

Su internet non si ha lo stesso problema fisico, ed è per questo motivo che si possono mettere a disposizione milioni di contenuti/prodotti. È possibile offrire anche prodotti che hanno una domanda molto bassa.

È nato così il fenomeno della **Long Tail**, ossia di avere pochi prodotti molto popolari e tantissimi prodotti poco popolari.



Avere tante scelte non significa strettamente stare meglio, per questo motivo **entrano in gioco i sistemi di raccomandazione** che sono una sorta di filtro intelligente.

Difatti, non vengono mostrati i 10.000.000 prodotti bensì cerca di selezionare i 20 prodotti probabilmente più rilevanti.

La situazione paradossale è proprio la seguente:

più prodotti riesci ad offrire, più aumenta la scelta; ma più aumenta la scelta, più diventa necessario un buon sistema per aiutare gli utenti a orientarsi. Se questo sistema non viene offerto, si crea un problema di **information overload**.

I problemi che si devono affrontare per i sistemi di raccomandazione sono:

- come costruire la utility matrix;
- predire valutazioni nuove e elevate;

La **utility matrix** rappresenta le interazioni tra utenti e oggetti, e generalmente è una matrice molto sparsa.

Ad esempio:

	ODISSEA NELLO SPAZIO	EYES WIDE SHUT	STRADE PERDUTE
MARCO	3	?	?
LUCA	3	?	1
SANTO	5	5	5

Le celle che contengono un numero rappresentano valutazioni conosciute e il '?' rappresenta una valutazione che non si conosce. Le valutazioni conosciute derivano solitamente da: voto, like, acquisto e/o visualizzazione.

La predizione può avvenire come abbiamo detto in due modi. In ogni caso, il sistema può predire che tu possa dare una valutazione alta:

- o da film simili;
- o da utenti che hanno una profilazione simile e hanno visto il film.

L'obiettivo non è predire tutte le celle mancanti della matrice, bensì trovare gli oggetti che l'utente probabilmente apprezzerà maggiormente e raccomandarglieli.

Quindi, la utility matrix è il punto di partenza del collaborative filtering. Si sanno solo le preferenze degli utenti e il sistema cerca di completare intelligentemente le celle mancanti.

Il punto difficile è proprio riempire l'utility matrix, infatti l'utente non dice direttamente cosa gli piace. Si possono ottenere queste informazioni in due modi:

- ♦ **valutazione esplicita** → si chiede direttamente all'utente cosa pensa dell'item → lo svantaggio principale è che l'utente non ha sempre voglia di dare valutazioni;
- ♦ **valutazione implicita** → si osservano le azioni dell'utente → guarda un video, compra un prodotto, clicca su un articolo → da queste azioni si cerca di inferire la preferenza dell'utente.

Nei sistemi odierni si generano continuamente dati comportamentali senza rendercene conto.

Content Based

Come già riportato su, l'idea fondamentale del Content-Based è che gli oggetti raccomandati all'utente U sono simili agli oggetti valutati positivamente dallo stesso. Quindi, se **piacciono alcuni oggetti, il sistema cerca altri oggetti con caratteristiche simili.**

Il **problema è descrivere un oggetto**, difatti dobbiamo creare un profilo per l'oggetto. Un profilo è una rappresentazione dell'oggetto attraverso un insieme di caratteristiche importanti. Per esempio, per un film potremmo avere:

FILM = {azione, fantascienza, spazio, Nolan}

Dobbiamo però preservare solo le caratteristiche più importanti, ad esempio in un giornale ci sono molte parole ma poche sono essenziali per effettuare una profilazione.

È qui che entra in gioco la **TF-IDF(Term Frequency-Inverse Document Frequency)** è un **metodo statistico usato** nell'analisi dei testi e **nel recupero delle informazioni.**

In breve, serve a dare un peso alle parole di un documento attraverso la frequenza. La frequenza però è relativa, infatti la parola deve essere frequente nel documento target ma non deve comparire frequentemente in tutti gli altri documenti.

Il **Term Frequency** misura quanto frequentemente compare un termine all'interno di un documento e la formula è:

$$\frac{f_{ij}}{\max_k f_{kj}}$$

dove:

- f_{ij} è la **frequenza del termine i nel documento j**;
- $\max_k f_{kj}$ è la **frequenza del termine più frequente nel documento j.**

L'**Inverse Document Frequency** consiste nell'analizzare i documenti a disposizione e attraverso il logaritmo ottenere:

$$IDF_i = \log \frac{N}{n_i}$$

dove: N è il numero di documenti presi in considerazione e n_i è il numero di documenti in cui compare i.

Quindi:

- se una parola compare in tantissimi documenti, n_i è grande → IDF piccolo;
- se una parola compare in pochissimi documenti, n_i è piccolo → IDF grande.

A questo punto se si combinano le due informazioni:

$$W_{ij} = TF_{ij} \cdot IDF_{ij}$$

Il profilo del documento può essere rappresentato come:

$$x = (0.8, 0.7, .0.6 \dots, 0.001)$$

dove a ogni TF-IDF è associata a una parola.

Ora che si ha il profilo degli oggetti si deve anche costruire il profilo dell'utente. Si può costruire il profilo dell'utente combinando i profili dei documenti.

Si fa la **media ponderata dei profili degli oggetti**, e concettualmente:

$$\vec{u} = \frac{w_A \vec{A} + w_B \vec{B} + w_C \vec{C}}{w_A + w_B + w_C}$$

dove A, B, C sono i pesi che dipendono dal rating dati dall'utente.

Ma come faccio a fare la predizione? La predizione si fa prendendo in considerazione un profilo dell'utente $\vec{u}$ e il profilo di un prodotto $\vec{i}$.

Attraverso la **cosine similarity** si ha:

$$\cos(\vec{u}, \vec{i}) = \frac{\vec{u} \cdot \vec{i}}{|\vec{u}| |\vec{i}|}$$

L'idea è confrontare l'angolo tra i due vettori e verificare:

$\cos(\theta) \approx 1 \rightarrow$ sono molto simili

$\cos(\theta) = 0$ non sono simili

Il sistema sceglierà gli item con similarità più elevata.

I vantaggi sono i seguenti:

- non servono dati sugli altri utenti;
- può consigliare a utenti con gusti unici;
- può raccomandare articoli nuovi e non popolari;
- è facile da interpretare.

Gli svantaggi sono:

- trovare le feature giuste è difficile $\rightarrow$ per documenti testuali si può fare, ma per definire un film?
- problema con i nuovi utenti $\rightarrow$ il sistema non sa ancora cosa gli piace.

Collaborative Filtering

Il **collaborative filtering** prova a trovare altri utenti che hanno dato rating simili a quelli di x, e si utilizzano i loro rating per prevedere cosa potrebbe piace a x.

Ogni utente può essere rappresentato attraverso il proprio vettore di rating, per esempio:

$$r_x = (5,4,?,2) \quad r_y = (5,4,5,2)$$

Il problema diventa quindi capire quanto sono simili r_x e r_y . Per risolvere questo problema si utilizzano le misure di similarità, ossia:

- **Jaccard Similarity** $\rightarrow$ problema principale è che la Jaccard ignora quanto sono diversi i voti;
- **Cosine similarity** $\rightarrow$ i rating distribuiti in modo simili permettono di avere una cosine similarity molto alta;
- **Correlazione di Pearson** $\rightarrow$ misura quanto due utenti seguono una relazione lineare nei loro rating, considerando anche la loro media personale $\rightarrow$ rimuove l'effetto che alcuni utenti sono più generosi e altri più severi con i voti.

Dopo che si sono trovati gli utenti più simili a x e si vuole prevedere il rating di x sull'item i, si prende:

$$N(i; x)$$

ossia l'insieme dei k utenti più simili a x che hanno valutato l'item i.

La formula per la predizione è sostanzialmente:

$$\hat{r}_{xi} = \frac{\sum_{y \in N(i; x)} s_{xy} r_{yi}}{\sum_{y \in N(i; x)} s_{xy}}$$

dove:

- **x** è l'utente per cui vogliamo fare la previsione;
- **y** è l'utente simile;
- **i** è l'item preso in considerazione;
- **s_{xy}** è la similarità tra x e y;
- **r_{yi}** è il rating che y ha dato all'item i.

Si dà più peso agli utenti che sono più simili a x.

Il **problema** è dato dalle scale di **valutazioni diverse utilizzate**. Per questo si può utilizzare la **media dell'utente e arrivare alla baseline estimation**.

Si stima la base del rating e la deviazione dell'utente rispetto alla media generale.

I vantaggi nell'uso del Collaborative Filtering sono: il lavorare con tutti i tipi di oggetti (non è necessario conoscere le caratteristiche dell'oggetto) e la feature selection irrilevante.

Gli svantaggi principali sono:

- arrivo di un nuovo utente → non si sa nulla di lui e non si possono trovare utenti simili;
- arrivo di un nuovo item → non si hanno informazioni sui gusti degli utenti riguardo quell'item;
- matrice dei rating sparsa → ogni utente avrà valutato solo una piccolissima parte dei prodotti.

La **possibile soluzione al problema dovuto alla sparsità della matrice è la dimensionality reduction**, ossia invece di lavorare in uno spazio enorme si conservano solo le informazioni più importanti. Qui entrano in gioco il concetto di PCA, SVD, LSI...

Il **LSI** o **Latent Semantic Index** è una **tecnica originariamente utilizzata soprattutto per il text mining/ information retrieval**. Questa tecnica **serve per scoprire automaticamente le strutture latenti(nascoste)**. Quindi, parole hanno distribuzioni simili nei documenti e appartengono allo stesso concetto latente.

I concetti li trova attraverso la SVD, prendendo i primi k valori singolari dato che LSI cerca una rappresentazione compatta e con dimensioni ridotte.

Anche se LSI nasce per Termini nei Documenti, si possono sostituire questi elementi con Utenti e Oggetti.

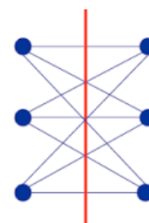
Metodi Graph-Based

I **metodi graph-based** prendono l'idea del Collaborative Filtering e la rappresentano in modo diverso.

Se il Collaborative Filtering **prende in considerazione le relazioni utenti-item**, le stesse relazioni vengono proposte nei metodi graph-based ma **rappresentate come un grafo**.

Le raccomandazioni sono ottenute a partire dalla struttura della rete bipartita. Formalmente si ha $G(U, O, E, W)$ dove:

- **U = utenti;**
- **O = oggetti;**
- **E = insieme degli archi;**
- **W = pesi degli archi.**



Il **peso dell'arco** rappresenta **quanto è forte la preferenza dell'utente** per quell'oggetto. Difatti, W è una funzione che lavora nel seguente modo:

$$W(U_1, O_1) = 5 \rightarrow \text{l'utente 1 preferisce molto l'elemento 1}$$

$$W(U_1, O_2) = 2 \rightarrow \text{l'utente 1 non preferisce l'elemento 2}$$

Ovviamente il range cambia in base all'informazione che riusciamo ad estrapolare. Quindi, se avessi semplicemente like/unlike, allora si avrebbe $W_{ij} \in \{0, 1\}$.

Si utilizza il grafo perché contiene direttamente tutte le informazioni sulle interazioni e si può utilizzare la struttura della rete per generare raccomandazioni.

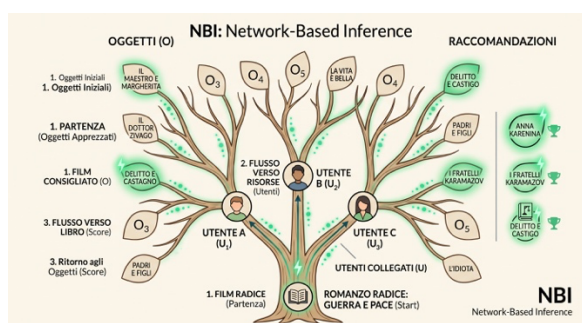
NBI – Network Based Interface

Uno degli algoritmi graph-based è l'**NBI o Network-Based Interface**, che utilizza proprio la struttura della rete bipartita per produrre raccomandazioni. La sua idea fondamentale è quella del flusso delle risorse.

Una risorsa è l'indicatore della probabilità che un oggetto sia interessante per l'utente. Il processo è il seguente:

$$O \rightarrow U \rightarrow O$$

- 1) Si parte dagli oggetti già apprezzati dall'utente;
- 2) Si analizza quali altri utenti hanno apprezzato l'elemento;
- 3) Dagli utenti si trasferisce parte della risorsa (probabilità di gradimento) verso altri elementi che hanno ricevuto un esito positivo;
- 4) Alla fine, si analizzano gli elementi che ricevono più risorse e ottengono uno score maggiore;
- 5) Gli oggetti con score elevato vengono raccomandati.



Quando si parla dei **graph-based** si può proiettare il grafo bipartito sugli oggetti, eliminando gli utenti e **collegando gli oggetti che hanno utenti in comune**.

Esiste un arco che va da O_i e O_j se esiste almeno un utente che ha interagito con entrambi. Si ottiene quindi una rete di oggetti e il peso dell'arco indica quanto è forte questa relazione.

Il calcolo dei pesi avviene mediante la seguente equazione:

$$w_{ij} = \frac{1}{\Gamma(i,j)} \sum_{l=1}^{|U|} \frac{a_{il}a_{jl}}{D(u_l)}$$

dove:

- a_{il} e a_{jl} sono elementi della matrice di adiacenza e valgono 1 se c'è un collegamento altrimenti 0;
- $D(u_l)$ è il **grado dell'utente l**, ossia a quanti oggetti è connesso;
- $\Gamma(i,j)$ è un **fattore di normalizzazione e dipende l'algoritmo cambia**.

Il fattore di normalizzazione $\Gamma(i,j)$ cambia; infatti:

- in **NBI** si divide per $D(o_j)$ ossia il **grado dell'oggetto di partenza**(favorisce i più popolari);
- in **HeatS** si divide per anche i contributi dagli utenti collegati favorendo oggetti di nicchia.

Tutto però si può riscrivere matricialmente, ottenendo così la matrice delle raccomandazioni R come:

$$R = A \times W$$

La matrice sparsa può diventare densa, difatti sfruttando la struttura delle connessioni si ottiene un'informazione maggiore, non essendo necessario che ogni utente abbia valutato ogni oggetto.

Un vantaggio considerevole è che funziona con tutti i tipi di oggetti, ergo **non è utile conoscere le feature interne dell'oggetto**, basta sapere le relazioni.

I problemi principali sono legati al costo computazionale e all'inserimento di nuovi utenti/oggetti.

Infine, **si possono utilizzare approcci ibridi**, cioè approcci diversi combinati e che producono score diversi. Questo produce un vantaggio, ossia migliorare le qualità delle raccomandazioni risolvendo debolezze di modelli diversi.

$$z_i = \lambda x_i + (1 - \lambda)y_i$$

dove λ è il **peso che diamo ai due modelli**.

Il problema dell'approccio ibrido è che è costoso computazionalmente.

Valutazione delle previsioni

Le previsioni fatte dai sistemi devono poi essere valutate, per capire se il sistema funziona bene. Quindi, le previsioni fatte devono essere poi confrontate con ciò che è realmente.

Per capire quanto sono vicine le predizioni ai rating reali, si può calcolare l'errore per ogni elemento:

$$\hat{Y}_i - Y_i$$

Si elevano al quadrato gli errori, perché gli stessi potrebbero compensarsi e dare risultato nullo. Ad esempio:

$$+2+(-2) = 0$$

Ci sono alcune metriche di valutazione come:

- ♦ **RMSE** (Root Mean Square Error) $\rightarrow \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2}$ $\rightarrow$ utile a predire un numero $\rightarrow$ più basso è e più è vicino è ai valori reali;
- ♦ **PRECISION** $\rightarrow \frac{TP}{TP+FP}$ $\rightarrow$ tra tutti gli oggetti raccomandati, quanti erano effettivamente buoni $\rightarrow$ precisione alta = raccomandazioni molto affidabili;
- ♦ **RECALL** $\rightarrow \frac{TP}{TP+FN}$ $\rightarrow$ tra tutti gli oggetti che sarebbero piaciuti all'utente, quanti tra essi ne ho raccomandati;
- ♦ **COVERAGE** $\rightarrow$ per quanti utenti il sistema è in grado di produrre una raccomandazione;
- ♦ **ROC** $\rightarrow$ curva di tradeoff tra falsi positivi e falsi negativi $\rightarrow$ quanto il nostro sistema raccomanda $\rightarrow$ curva alta aumentano i falsi negativi e diminuiscono i falsi positivi e viceversa.

La sfida Netflix

Agli arbori di Netflix, c'erano circa mezzo milione di utenti e più di 15 mila film. Questo implicava, con un sistema di raccomandazione classico, una potenza di calcolo enorme. Infatti, addestrare il sistema a imparare le relazioni tra utenti e film era molto costoso (matrici enormi).

La metrica di valutazione era l'RMSE e la competizione chiedeva di migliorare questo risultato.

Per sviluppare un sistema di raccomandazione più efficiente, fu messo in palio un premio di un 1.000.000 di dollari.

Il risultato fu lo sviluppo del **Multi-Scale Modeling** (ideato dalla Bellkor). Questo modello non cerca di prevedere un rating guardando una sola cosa, ma si possono guardare i dati a diversi livelli. I livelli sono:

- **Globale** $\rightarrow$ il livello globale cerca di capire le **tendenze generali** calcolando il rating medio dei film, definendo poi la baseline per il singolo utente;
- **Fattorizzazione** $\rightarrow$ entrano in gioco i fattori latenti ossia il **sistema scopre che esistono dimensioni nascoste** come: azione vs commedia...;
- **Local patterns** $\rightarrow$ si guardano le relazioni specifiche (ad esempio i film piaciuti all'utente): utenti con comportamenti simili o film che hanno comportamenti di rating simili.

Conclusioni

Utilizzare un modello predittivo può avere molti vantaggi, infatti, si impara dai dati di training e attraverso una misura d'errore come la SSE(Sum Squared Error) si perfeziona il modello.

Quindi, si cerca una funzione/modello che utilizziamo per fare le predizioni, che ha la seguente forma:

$$\hat{r}_{xi} = b_{xi} + \sum_j w_{ij}(r_{xj} - b_{xj})$$

dove:

- $\hat{r}_{xi}$ è il rating che prevediamo che l'utente x darà all'item i;
- b_{xi} è la baseline, o la prima stima che si fa;
- $r_{xj} - b_{xj}$ è il rating effettivo per l'elemento j vicino a i;
- w_{ij} è il peso che si dà al rating effettivo.

Si cercano i valori di w che fanno commettere meno errori al modello, e questi pesi vengono imparati utilizzando la discesa del gradiente.

Analisi di Network

Un sistema complesso è formato da moltissime componenti che interagiscono tra loro. Ad esempio: la società, internet, conoscenza(concetti e informazioni collegati tra loro), sistema biologico, cervello.

La caratteristica comune non è guardare i singoli elementi, ma bisogna **capire come sono collegati tra loro i componenti**.

Si può rappresentare un sistema complesso attraverso una network, o rete. Una network è costituita da:

- **Nodi(nodes)** → rappresentano componenti/oggetti del sistema;
- **Link(edges)** → rappresentano le connessioni o interazioni tra gli oggetti.

Ad esempio, in una rete sociale: i nodi sono le persone, mentre i link rappresentano le amicizie.

Il punto fondamentale è che non si può comprendere un sistema complesso senza che si comprenda la struttura delle connessioni che lo compongono.

La network analysis cerca di rispondere a domande come:

- quali nodi sono i più importanti?
- quali nodi sono maggiormente connessi?
- esistono gruppi di nodi strettamente collegati?
- quanto è facile raggiungere un nodo partendo da un altro?

L'intero sistema viene rappresentato dal grafo:

$$G(N, E)$$

Per esempio, in una rete sociale:

- nodi = persone;
- archi = relazioni di amicizia.

Una Network e un Grafo sono concetti simili, ma c'è una piccola distinzione. Le **Network indicano principalmente il sistema reale che si vuole studiare**; invece, **il grafo indica la rappresentazione matematica del sistema**.

Quindi possiamo pensare:

$$SISTEMA REALE \rightarrow NETWORK \rightarrow GRAFO$$

Il problema più importante è: come costruire una rete?

Si devono decidere: i nodi, gli archi e quando due nodi possono essere considerati collegati. Questa scelta è fondamentale perché la rappresentazione scelta determina ciò che possiamo studiare dalla rete.

I grafi possono essere diretti e non diretti, e questo è influenzato dall'esistenza della direzione degli archi nel grafo.

Un **grafo non diretto** è un grafo **dove gli archi non hanno una direzione**; quindi, se A è collegato a B, allora anche B è collegato ad A. Un esempio è una relazione di amicizia.

Un **grafo diretto** è un grafo **dove gli archi hanno una direzione**; quindi, se A è collegato a B, non necessariamente B è collegato ad A. Un esempio è il following su Instagram.

Il grado di un nodo indica quanti archi sono collegati a quel nodo. In un grafo non diretto:

k_i = numero di archi incidenti al nodo i

Il degree misura, in un certo senso, quanto è connesso un nodo al resto del grafo.

Per un grafo non diretto, per calcolare il grado medio si può utilizzare la formula:

$$\bar{k} = \frac{2E}{N} = \bar{k} = \frac{1}{N} \sum_{i=1}^N k_i$$

Distinguiamo nei grafi diretti i gradi a seconda della direzione:

- **in-degree k_i^{in}** è il numero di archi che arrivano al nodo;
- **out-degree k_i^{out}** è il numero di archi che partono al nodo;

il grado totale è $k_i = k_i^{in} + k_i^{out}$.

Il **grado medio** si può calcolare attraverso la seguente formula:

$$k_i = \bar{k}_i^{in} + \bar{k}_i^{out} = \frac{2E}{N}$$

Un **grafo completo** è un **grafo nel quale ogni nodo è collegato a tutti gli altri nodi**. Quindi, se abbiamo N nodi quanti archi si possono avere al massimo?

Il numero di coppie possibili è:

$$E_{max} = \binom{N}{2}$$

cioè:

$$E_{max} = \frac{N(N-1)}{2}$$

Il grado medio è:

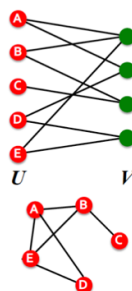
$$\bar{k} = N - 1$$

Un **grafo bipartito** è un grafo in cui i **nodi sono divisi in due sistemi distinti**:

- U;
- V

e gli **archi possono collegare solo un nodo di U con un nodo di V**.

Si può trasformare una rete bipartita in una rete costituita da un solo tipo di nodo. Si collegano due nodi di U se hanno almeno un vicino comune di V. Ad esempio:



Un **grafo può essere rappresentato in modi diversi**, e le tre rappresentazioni fondamentali sono:

- 1) **Matrice di adiacenza** → matrice di $n \times n$ (con n numero di nodi) dove se esiste un arco da i a j allora $M_{ij} = 1$ altrimenti 0;
- 2) **Lista degli archi** → (i, j) se esiste un arco;
- 3) **Lista di adiacenza** → lista di liste, dove alla lista i corrisponde una lista di nodi collegati a i stesso.

La **maggior parte delle reti reali è sparsa**, ossia che il numero di archi è molto più piccolo del numero massimo possibile:

$$E \ll E_{max}$$

Un arco può avere degli attributi, cioè informazioni aggiuntive che descrivono la relazione tra due nodi.

Il **peso indica quanto è forte, frequente, grande o importante una relazione**. Ad esempio, in una rete stradale, il peso potrebbe rappresentare la distanza.

Il **rank indica una posizione o una graduatoria della relazione**. Quindi non interessa sapere se due persone sono collegate, bensì è anche importante sapere la relazione rispetto alle altre.

L'**arco può avere una tipologia espressa attraverso un'etichetta**. Questa, ad esempio, in un'azienda potrebbe essere:

$$A \rightarrow^{CAPO} B$$

e

$$A \rightarrow^{SUBBORDINATO} B$$

Infine, l'**arco può avere anche un segno**.

Nel grafo non pesato è utile sapere solamente se esiste o meno un collegamento.

Nel grafo pesato, gli archi hanno invece un valore numerico e il valore a cui si riferisce e il peso descritto prima.

$$A = \begin{pmatrix} 0 & 2 & 0.5 & 0 \\ 2 & 0 & 1 & 4 \\ 0.5 & 1 & 0 & 0 \\ 0 & 4 & 0 & 0 \end{pmatrix}$$

Un **self-edge**, o **self-loop**, è un **arco che parte da un nodo e ritorna allo stesso nodo** di partenza. Nella matrice di adiacenza compare lungo la diagonale principale il valore 1, per esempio: $A_{11} = 1$.

In un multigrafo possono esistere più archi tra la stessa coppia di nodi, e nella matrice si ha $A_{ij} = n$, con n numeri di archi che vanno dal nodo i al nodo j .

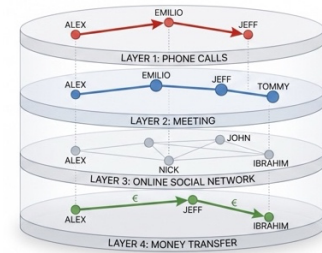
Da non confondere con il grafo pesato dove n è il peso e non il numero di archi nel grafo.

Grafi Multilayer

I grafi possono essere **grafi multilayer**, cioè **grafi dove si hanno più reti sovrapposte, chiamate layer**. Ogni layer rappresenta un tipo diverso di relazione.

Bisogna fare una distinzione tra **grafi multiplex** e **grafi multilayer**.

Infatti, i **grafi multiplex** hanno nodi comuni a tutti i layer e cambiano solo gli archi del singolo layer; invece, i **grafi multilayer** sono grafi in cui in ogni layer si hanno relazioni, sistemi o tipi di nodi differenti.



Un **grafo non diretto è connesso**, se presi due nodi qualsiasi, esiste almeno un cammino che permette di andare da uno all'altro. È connesso perché si può raggiungere qualsiasi nodo partendo da qualsiasi altro nodo.

Un grado è disconnesso quando esistono almeno due gruppi di nodi tra i quali non esiste alcun cammino. Ad esempio, si hanno due gruppi separati:

- 1) {A, B};
- 2) {C, D}.

Questi gruppi sono chiamati componenti connesse.

Un **arco ponte** è un arco che, **se viene rimosso, fa diventare disconnesso il grafo**. Ad esempio, se si ha $A - B - C$, l'arco $B - C$ è un ponte perché se viene eliminato si hanno le componenti connesse:

- 1) {A,B};
- 2) {C}.

Un **punto di articolazione** è invece un **nodo che, se viene rimosso, fa diventare il grafo disconnesso**.

Riproponendo l'esempio di prima, B è un punto di articolazione perché non esiste un arco che collega A con C.

Se un grafo è disconnesso, si possono avere diverse componenti connesse di dimensioni differenti.

La **componente connessa più grande viene chiamata componente connessa gigante**, quando costituisce una componente particolarmente grande rispetto le altre.

Nei grafi diretti la situazione è più complicata perché gli archi hanno una direzione e quindi se $A \rightarrow B$ non necessariamente $B \rightarrow A$.

Una **componente è fortemente connessa quando è un insieme di nodi nel quale, per ogni coppia di nodi, esiste un percorso in entrambe le direzioni**. Non è necessario che esista un arco diretto tra ogni coppia, bensì è sufficiente che esista un cammino orientato.

Un **grafo diretto è debolmente connesso** se, quando si ignora la direzione degli archi, il grafo diventa connesso.

Per descrivere una network si introducono tre proprietà fondamentali ossia:

- ♦ **distribuzione del grado** → qual è la probabilità che, scegliendo casualmente un nodo, questo abbia grado k ? Se abbiamo N nodi e N_k di questi hanno grado k , allora:

$$P(k) = \frac{N_k}{N}$$

Quindi $P(k)$ ci dice come sono distribuiti i gradi nella rete.

- ♦ **path** → è una **sequenza di nodi in cui ogni nodo è collegato al successivo, la cui lunghezza viene calcolata attraverso gli archi attraversati.**
- ♦ **coefficiente di clustering** → **quanto i vicini di un nodo sono collegati tra loro.**

Un'importante distinzione da fare è tra cammino e cammino minimo; infatti, il cammino minimo è il cammino con il minor numero di archi.

La distanza tra due nodi è proprio la lunghezza del loro cammino minimo.

Come facciamo a calcolare se esiste almeno un cammino di lunghezza k tra i e j ?

Moltiplicando la matrice di adiacenza per se stessa tante volte quanto è lungo il cammino che si cerca.

Perciò:

$$[A^k]_{ij} = \text{numero di cammini di lunghezza } k \text{ da } i \text{ a } j$$

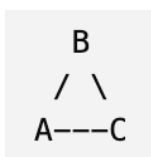
Non vengono contanti i cammini di lunghezza inferiore anche se potrebbero esserci.

Il **diametro della rete** è invece la **distanza massima** (sapendo che la distanza è la lunghezza del cammino minimo) tra due nodi. Si possono calcolare anche quanto sono mediamente lontani i nodi, infatti, si fa la **media delle distanze** tra tutte le coppie di nodi. La formula è la seguente:

$$\bar{h} = \frac{1}{2E_{max}} \sum_{i < j} h_{ij}$$

h_{ij} è la distanza (cammino minimo tra i e j) e si specifica $i < j$ per evitare ripetizioni.

Il coefficiente di clustering misura quanto sono collegati tra loro i vicini in un determinato nodo. Ad esempio:



A ha due vicini: B, C.

Gli stessi nodi vicini sono collegati tra loro, per questo motivo i vicini di A sono molto aggregati.

Il clustering di A è $C_A = 1$.

La formula per il calcolo del coefficiente di clustering è:

$$C_i = \frac{2e_i}{k_i(k_i - 1)}$$

dove:

- k_i = numero di vicini del nodo i ;
- e_i = numero di archi tra i vicini di i .

Il valore va da 0 a 1 e quando il valore del coefficiente è uguale a 1, vuol dire che questi vicini sono completamente collegati tra loro. Si calcola il coefficiente di clustering di una rete facendo la media.

Modello Nullo

Prendendo in considerazione un esempio reale, si può analizzare l'attività di MSN Messenger (app messaggistica istantanea di microsoft) di giugno 2006. Gli utenti dell'app, posti in un grafo, vengono rappresentati mediante i nodi; invece, gli archi rappresentano le conversazioni tra utenti.

Calcolando le proprietà della rete si ha che:

- $\bar{k} = 14.4$;
- $\bar{h} = 6.6$;
- $C = 0.11$.

Ma questi dati soltanto non possono essere presi in considerazione, si devono avere dati di confronto. Per questo motivo si introduce il modello nullo.

L'idea alla base del **modello nullo** è: **la costruzione di una rete di riferimento che non abbia particolari caratteristiche interessanti, utile solo al confronto con la rete reale.**

Per esempio, si può creare una rete casuale con:

- stesso numero di nodi;
- stesso numero di archi;

ma con gli archi distribuiti casualmente.

La domanda da porsi è: **osservando un pattern questo è reale o casuale?** Il modello nullo ci permette di rispondere.

La prima cosa che facciamo è formulare un'**ipotesi nulla**, indicata con: H_0 . L'ipotesi nulla rappresenta normalmente lo scenario in cui non c'è l'effetto che stiamo cercando.

In un certo senso, è il **punto di partenza scettico in statistica**, assumendo che non ci sia alcuna relazione speciale, nessun pattern strutturato e nessun effetto reale, ma solo puro caso.

Ad esempio, con una moneta lanciata che produce 15 teste su 17 potremmo chiederci:

H_0 : la moneta è imparziale

Si parte da H_0 perché si ha bisogno di un mondo di riferimento e si vuole sapere come sarebbero i dati se l'effetto che stiamo cercando non esistesse.

A questo punto arriva la **randomizzazione** che **serve a creare tanti esempi di ciò che potrebbe succedere se H_0 fosse vera.**

Ad esempio, se avessimo la seguente ipotesi nulla:

H_0 : il farmaco non ha un effetto maggiore del placebo

la randomizzazione consiste nell'associare ai valori delle persone l'etichetta Farmaco/Placebo in modo random. Questo viene fatto perché se l'ipotesi nulla fosse vera, mescolare casualmente le etichette D e P non altera i valori.

Calcolando la media dei valori prima e dopo la randomizzazione dei valori, potremmo accorgerci che compare una differenza. Però anche quando non c'è alcun effetto reale, il caso può produrre delle differenze.

La randomizzazione si ripete più volte ottenendo una distribuzione di differenze ottenibili per puro caso. La distribuzione ottenuta è la distribuzione nulla, perché è stata costruita assumendo che H_0 sia vera.

Quante volte, pur mescolando casualmente le etichette, si ottiene una differenza almeno grande come quella ottenuta senza randomizzazione.

Supponendo che capitì 10 volte su 10000 allora:

$$p = \frac{10}{10000} = 0.001 = 0.1\%$$

Il **p-value**, dice **quanto è raro il risultato osservato**, supponendo che l'ipotesi nulla sia vera.

Se il farmaco non avesse realmente alcun effetto, solo circa lo 0.1% delle randomizzazioni produrrebbe una differenza almeno grande quanto quella osservata (pochissimo).

Quindi il risultato è molto raro sotto l'ipotesi nulla, per questo dubitiamo della stessa.

Si è introdotto il modello nullo per capire se il valore $C_{MSN} = 0.11$ è un clustering realmente caratteristico della rete MSN oppure può comparire anche in una rete casuale?

Se quasi nessuna rete casuale arriva a $C_{MSN} = 0.11$ allora il p-value è piccolo e il clustering di MSN è sorprendentemente alto.

Se molte reti casuali arrivano a 0.11 il p-value è grande e il risultato non è particolarmente sorprendente.

Modello di Erdős-Rényi

Il **modello di Erdős-Rényi** è uno dei **modelli più semplici per costruire una rete casuale**. È importante soprattutto perché può essere utilizzato come modello nullo.

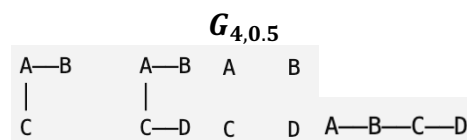
L'idea fondamentale è che avendo n nodi le connessioni vengono create casualmente. Non si decide manualmente quali archi esistono: si fanno comparire casualmente secondo determinate regole.

Ci sono due versioni principali, cioè:

- $G_{n,p} \rightarrow$ grafo non direzionato con **n numero di nodi** e **p probabilità che ogni possibile arco esista**.

Per ogni coppia di nodi si decide: con probabilità p di mettere l'arco e con probabilità $1-p$ di non mettere l'arco.

Con n e p non si determina un unico grafo, bensì una distribuzione di grafi. Ad esempio:



- $G_{n,m} \rightarrow$ grafo non direzionato con **n numero di nodi** e **m numero esatto di archi**.

Si scelgono casualmente gli m archi da inserire, tra tutti quelli possibili.

Come nel caso precedente non si determina un unico grafo, ma una distribuzione di grafi.

Per la prima tipologia di grafi quanti archi avrà alla fine il grafo? Non si può sapere con certezza. La formula della probabilità per ottenere esattamente E archi è la seguente:

$$P(E) = \binom{E_{max}}{E} p^E (1-p)^{E_{max}-E}$$

Questa è una **distribuzione binomiale**, perché si hanno E_{max} possibili archi e per ogni arco si hanno due possibilità: esiste / non esiste.

È esattamente la situazione di una binomiale e si può pensare: $E \sim \text{Binomiale}(E_{max}, p)$.

Il grado di un nodo è X_u :

$$X_u = X_{u,1} + X_{u,2} + \dots + X_{u,n-1} \text{ con } X_{u,v} = \begin{cases} 1 & \text{se l'arco } (u,v) \text{ esiste} \\ 0 & \text{altrimenti} \end{cases}$$

Il grado atteso di un nodo in un grafo Erdős-Rényi è:

$$E[X_{u,v}] = p$$

e quindi:

$$E[X_u] = p(n-1)$$

Anche il grado k del nodo segue una distribuzione binomiale, infatti la distribuzione del grado di $G_{n,p}$ è la seguente:

$$P(k) = \binom{n-1}{k} p^k (1-p)^{n-1-k}$$

La varianza del grado del singolo del nodo è la seguente:

$$\sigma^2 = p(1-p)(n-1)$$

questo è utile a capire quanto si disperde attorno alla distribuzione.

Per calcolare il clustering di $G_{n,p}$ bisogna fare un ragionamento un po' più complesso. Infatti, ricordando che:

$$C_i = \frac{2e_i}{k_i(k_i-1)}$$

Sapendo che:

$$e_i = p \frac{k_i(k_i-1)}{2}$$

Sostituendo nella formula si avrà che:

$$C_i = \frac{2e_i}{k_i(k_i-1)} = \frac{2\left(p \frac{k_i(k_i-1)}{2}\right)}{k_i(k_i-1)} = p \frac{k_i(k_i-1)}{k_i(k_i-1)} = p$$

Infine, dato che:

$$\bar{k} = p(n-1) \text{ e } \frac{\bar{k}}{(n-1)}$$

allora:

$$C \approx \frac{\bar{k}}{(n-1)}$$

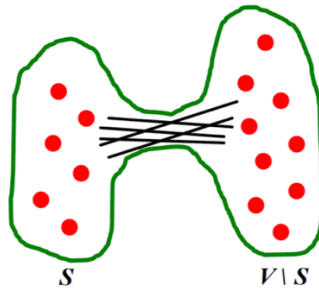
Espansione

L'espansione misura quanto sia difficile tagliare a metà o isolarne un sottoinsieme di nodi rimuovendo gli archi. Questa tecnica risponde alla seguente domanda:

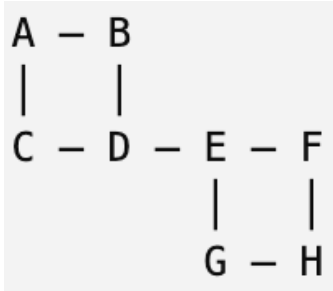
“Se prendo un gruppo di nodi e lo separo dal resto della rete, quanti archi devo tagliare?”

Una rete è molto espansa se, qualunque gruppo di nodi scelti, questo gruppo ha molti collegamenti con il resto della rete.

Una rete ha bassa espansione se si possono trovare un gruppo di nodi collegati al resto da pochissimi archi.



Avendo ad esempio un grafo del genere:



Supponendo di poter dividere la rete in due parti si avrebbe:

- $S = \{A, B, C, D\}$
- $V \setminus S = \{E, F, G, H\}$

Gli archi uscenti da S sono quelli che attraversano il confine tra le due parti.

Nell'esempio: D - E è un arco uscente da S.

Quindi si ha che:

$$\# \text{archi uscenti} = 1$$

L'espansione allora è definita come:

$$\alpha = \min_{S \subseteq V} \frac{\# \text{archi che escono da } S}{\min(|S|, |V \setminus S|)}$$

In questo modo si cerca il modo peggiore di dividere la rete in due parti e si guarda quanti archi si devono attraversare per collegare le due parti.

Il minimo serve proprio a trovare il collo di bottiglia della rete.

Si divide per $\min(|S|, |V \setminus S|)$, perché si vuole rapportare il numero di archi che attraversano il confine alla dimensione della parte più piccola. Questo permette di confrontare anche separazioni di diverse dimensioni.

L'espansione, quindi, misura quanto è ben collegata una rete e:

- se α è basso allora la rete è facilmente separabile;
- se α è alto allora la rete è robusta.

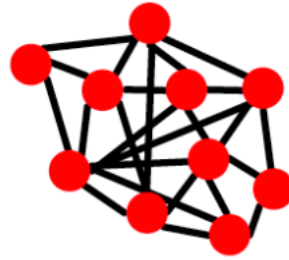
Per disconnettere l nodi si devono rimuovere almeno $\alpha \cdot l$ archi. Se una parte della rete contiene l nodi, allora:

$$\# \text{archi uscenti} \geq \alpha l$$

Bassa Espansione



Espansione Elevata



Un **grafo k-regolare** è un **grafo** in cui per ogni nodo il grado sia k .

Per un grafo 3-regolare randomico con n nodi esiste una costante $\alpha > 0$, indipendente da n , tale che l'espansione è almeno α .

Quindi anche se facciamo crescere enormemente il grafo, la rete continua ad avere una buona espansione.

α non dipende da n , quindi anche se facciamo crescere enormemente il grafo, la rete continua ad avere una buona espansione. Un random 3-regular graph è robusto e ben connesso.

In un grafo con espansione α , per ogni coppia di nodi s, t esiste un path con lunghezza:

$$O\left(\frac{\log n}{\alpha}\right)$$

Questo significa che l'**espansione elevata aiuta a mantenere breve i cammini**.

La lunghezza dei path è $O(\log n)$, e cresce molto lentamente quando aumenta il numero di nodi.

Questa è una caratteristica importante nelle reti casuali, infatti, anche enormi reti possono avere distanze relativamente piccole tra i nodi.

Reti reali vs modello Erdős-Rényi

Come si è spiegato precedentemente, si è visto che $G_{n,p}$ produce reti casuali in cui: si hanno n nodi e ogni possibile arco compare indipendentemente con probabilità p .

Il problema è che le **reti reali non si comportano completamente come reti casuali**. In comune possono avere una componente connessa gigante e shortest path relativamente corti, ma non la stessa distribuzione del grado e lo stesso coefficiente di clustering.

Questo non vuol dire che il modello è completamente inutile; infatti, rappresenta una sorta di baseline minima.

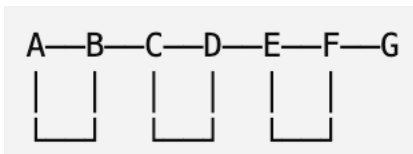
Se la proprietà della rete reale è ancora molto particolare rispetto ad una rete casuale, significa che non è semplicemente una conseguenza del numero di nodi e degli archi.

Il problema dei gradi è che non li controlliamo direttamente, ma seguono una distribuzione binomiale casuale.

Supponendo di volere una rete i cui nodi abbiano un esatto degree, il **configuration model** fa proprio questo.

Infatti, partendo dalla sequenza di gradi $k_1, k_2, \dots, k_n$ (gradi reali dei nodi), si randomizza solo la disposizione con la quale sono collegati.

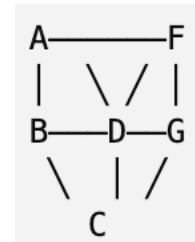
Le reti reali hanno spesso: clustering alto e short path. È possibile avere contemporaneamente questa combinazione nei grafi?



Infatti, immaginando una rete molto regolare, i nodi vicini tendono ad essere collegati tra loro. Si hanno tanti triangoli che influenzano un coefficiente di clustering alto e uno short path alto.

Al contrario, in una rete completamente casuale si possono raggiungere rapidamente zone lontane.

In questo caso si ha il coefficiente di clustering basso e i cammini corti, e sembra che si debba scegliere.



In realtà non è così, infatti, il modello Small-World dimostra che si possa avere entrambi contemporaneamente.

Un esempio intuitivo è l'esempio del **Bacon Number**, ossia la distanza di un attore da Kevin Bacon.



Costruendo un grafo dove:

- i nodi sono attori;
- due nodi sono collegati se gli attori hanno recitato nello stesso film

Sorprende il fatto che, nonostante ci siano tantissimi attori, generalmente bastano pochi passaggi per collegare due persone.

Un altro caso è l'**esperimento di Milgram**, che cercava di capire quanto è lungo mediamente il percorso sociale tra due persone.

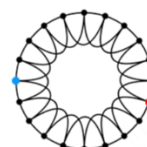
In questo caso, preso un campione di 300 persone in alcuni stati americani si doveva spedire una lettera passandola solo agli amici. Gli step effettuati sono solo 6 (da cui nasce il 6 gradi di separazione), dimostrando che una rete sociale enorme può avere distanze molto piccole.

Perciò, non è semplicemente $G_{n,p}$ dato che **nelle reti sociali esiste la chiusura triadica**.

La chiusura triadica, tipica nelle reti sociali si può esprimere come: l'amico di un mio amico ha una buona probabilità di essere anche mio amico. Questo implica che si formino tanti triangoli, ergo il coefficiente di clustering è alto nonostante la distanza sia bassa.

Il **modello Small-World** risolve il problema della rete casuale che non rispetta una rete sociale. L'**idea fondamentale è capire che aggiungendo pochi collegamenti casuali può accorciare enormemente i cammini senza distruggere il clustering**.

Partendo quindi da una rete regolare, si collega ogni nodo ai nodi più vicini senza alcuna randomizzazione.



Si aggiunge il parametro p che controlla quanto randomizzare la rete. **Per ogni arco si considera la possibilità di modificarne la destinazione con probabilità p** . Questo processo chiamato processo di **rewiring** permette di collegare possibili collegamenti a lunga distanza.

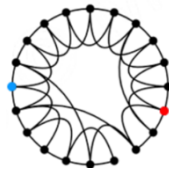
Supponendo quindi di avere un **lattice** (grafo in cui i nodi e collegamenti formano una struttura regolare e ripetitiva nello spazio), cioè:

$$A - B - C - D - E - F$$

con il rewiring si può prendere l'arco A – B, cambiare la destinazione e ottenere A – E. In questo modo si crea un **collegamento a lunga distanza**.

Il collegamento a lunga distanza viene chiamato **shortcut** o scorciatoia, e un singolo collegamento a lunga distanza **può ridurre drasticamente la distanza tra regioni lontane della rete**.

L'idea geniale di questo modello sta proprio nell'assegnare a p un piccolo valore e mantenere la maggior parte degli archi collegati localmente.



Il limite principale del modello è il riuscire a trovare quei cammini corti. Infatti, nonostante esistano collegamenti a lunga distanza che rendono le distanze globali molto piccole, non si conosce il grafo per intero.

Supponendo che ogni nodo sia una persona, un nodo conosce soltanto:

- i propri amici;
- gli amici degli amici;
- qualche informazione sul target.

Si deve decidere accuratamente quale persona contattare localmente.

Questa è la **navigazione** e significa: **riuscire a trovare un percorso breve tra due nodi utilizzando solamente informazioni locali**.

L'algoritmo può essere greedy, dove per raggiungere un nodo si deve chiedere ai nodi vicini e ricorsivamente si arriva a destinazione.

Un altro **limite del modello small-world è che gli archi vengono inseriti casualmente**. Non c'è una regola che dica dove devono andare gli shortcut.

Per questo motivo **Kleinberg cerca di risolvere proprio questo limite, imponendo di non inserire totalmente in modo casuale gli shortcut**.

Gli shortcut devono avere una struttura, sono distribuiti secondo una regola che tiene conto della distanza dei nodi.

Ricerca decentralizzata

La **ricerca decentralizzata** è il **processo utile per raggiungere un nodo target senza conoscere tutta la rete**.

Il nodo di partenza deve prendere una decisione solo con le informazioni che possiede localmente. La decisione, secondo Kleinberg, deve essere presa seguendo un principio semplice: andare dal vicino geograficamente più vicino al target.

Il nodo corrente decide localmente quale vicino sembra avvicinarlo di più al target. L'obiettivo è ottenere T (numero di step necessari per raggiungere il target) molto piccolo anche senza conoscere la rete completa.

Gli shortcut presi nel modello di Kleinberg hanno la seguente probabilità:

$$P(u \rightarrow v) \propto \frac{1}{d(u, v)^\alpha}$$

dove:

- $d(u, v)$ è la distanza tra i due nodi nella griglia;
- α è l'**esponente che determina quanto penalizzare i nodi lontani**.

Utilizzando una griglia 2D per rappresentare il grafo, la distanza è la distanza Manhattan:

$$d(u, v) = |x_u - x_v| + |y_u - y_v|$$

Il valore di alpha influenza la probabilità di scegliere nodi lontani. Infatti:

- se $\alpha = 0$ allora tutti i nodi hanno la stessa probabilità $\rightarrow$ idea simili al small-world;
- se α molto grande la probabilità diminuisce all'aumentare della distanza $\rightarrow$ si hanno pochi veri shortcut;
- se α molto piccolo la probabilità aumenta all'aumentare della distanza $\rightarrow$ i collegamenti non hanno abbastanza struttura geografica.

Si preferisce avere $\alpha = d$ (dimensioni $\rightarrow$ in questo caso 2) dato che la rete diventa navigabile tramite un algoritmo decentralizzato.

Il risultato è che le diverse scale di distanza ricevono una probabilità comparabile. In maniera intuitiva si ha: shortcut locali e shortcut molto lontani sono entrambi sufficientemente presenti.

È questo che permette la navigazione, supponendo di essere lontani Kleinberg rende abbastanza probabile che esista un collegamento che permetta di avvicinarsi al target.

Si ha perciò una strategia a livelli: lontano $\rightarrow$ medio $\rightarrow$ vicino $\rightarrow$ target.

Kleinberg non vuole semplicemente creare una rete in cui esistano cammini corti, ma anche una rete in cui un individuo non conoscendo tutta la rete riesca effettivamente a trovare un cammino breve, usando solo le informazioni locali.

Misure per la centralità

La **centralità** è una **misura che quantifica quanto un nodo può essere considerato importante, influente o strategico all'interno di una rete.**

Esistono diverse misure di centralità, ognuna delle quali considera un aspetto diverso dell'importanza di un nodo.

Le principali misure di centralità sono:

- **degree centrality** → è il **numero di archi direttamente collegati al nodo.**

In un grafo non diretto $C_D(i) = k_i$ dove k_i è il degree del nodo i . In un grafo diretto si può distinguere:

- $C_D^{in}(i) = k_i^{in} \rightarrow$ archi entranti;
- $C_D^{out}(i) = k_i^{out} \rightarrow$ archi uscenti.

Questa **misura semplicemente considera quante connessioni dirette ha il nodo.** Più connessioni ha il nodo è più e considerato centrale.

- **betweenness centrality** → misura **quanto un nodo si trova nel mezzo degli shortest tra le coppie di nodi.** Formalmente:

$$C_B(i) = \sum_{j < k} \frac{g_{jk}(i)}{g_{jk}}$$

dove:

- g_{jk} è il numero di shortest path tra j e k ;
- $g_{jk}(i)$ è il numero di shortest path che passano per i .

Questa misura pone al centro dell'attenzione quanto un nodo svolge il ruolo di: ponte/intermediario.

- **closeness centrality** → misura quanto un nodo è vicino, in termini di shortest path, a tutti gli altri nodi della rete. Una definizione comune è:

$$C_c(i) = \frac{n - 1}{\sum_{j \neq i} d(i, j)}$$

Un nodo ha alta closeness se può raggiungere gli altri nodi con pochi passaggi in media. Più alto è il valore di closeness più la distanza complessiva è minore.

Preferential Attachment

Il **Preferential Attachment** è un **modello di formazione delle reti in cui i nuovi nodi tendono a collegarsi preferenzialmente ai nodi che hanno già un alto degree**.

Intuitivamente si ha che:

$$P(\text{nuovo collegamento verso } i) \propto k_i$$

dove k_i è il grado del nodo i .

Questo meccanismo può produrre **hub**, ossia un nodo con un numero di connessioni molto maggiore rispetto alla maggior parte degli altri nodi.

La maggior parte dei nodi può avere pochi collegamenti, mentre pochi hub hanno tantissimi collegamenti. Questo porta a una **distribuzione del degree con coda pesante** e approssimativamente:

$$P(k) \propto k^{-\alpha} \text{ che viene chiamata } \textbf{power-law}$$

dove: α è il parametro che determina quanto rapidamente $P(k)$ diminuisce al crescere di k .

Una **rete scale-free** è una rete la cui distribuzione dei degree segue, almeno approssimativamente, una power-law:

$$P(k) \propto k^{-\alpha}$$

Si chiama scale-free perché non ha una scala caratteristica. In una distribuzione normale, ad esempio, la maggior parte dei valori è concentrata attorno alla media.

In una power-law non si segue un degree tipico, si può avere:

$$k = 2, 5, 10, 20, 100, 1000...$$

con una quantità di valori molto grandi.

In una rete scale-free la varianza può divergere per N molto grande. Si ricorda:

$$\sigma^2 = E[k^2] - E[k]^2$$

dove:

- $E[k]$ = degree medio della rete;
- $E[k^2]$ = media dei quadrati dei degree.

Se la distribuzione ha una coda power-law sufficientemente pesante, i valori molto grandi di k contribuiscono enormemente a $E[k^2]$.

Gli hub nelle scale-free sono molto importanti, perché se rimossi si possono perdere moltissime connessioni contemporaneamente. Ciò che possiamo dedurre è che la rete è molto vulnerabile agli attacchi mirati agli hub.

Ci sono molte reti di tipo power-law, ad esempio:

- Popolazione delle città(tendono a stare molto concentrate al centro);
- Terremoti;
- Numero di follower;
- Ricchezza.

Una distribuzione heavy-tailed è una distribuzione che diminuisce lentamente, permettendo una probabilità relativamente elevata di osservare valori molto grandi.

Quindi, nella heavy tail c'è una probabilità relativamente alta di avere valori estremi.

La forma fondamentale nelle power-law è la seguente:

$$p(x) = Zx^{-\alpha}$$

dove: Z è la costante di normalizzazione dato che la distribuzione deve avere probabilità totale uguale a 1.

Analisi di grandi grafi

Il web può essere visto come un enorme grafo diretto, dove:

- ogni nodo rappresenta una pagina web;
- ogni arco rappresenta un link ipertestuale da una pagina a un'altra.

Una domanda importante sorge spontanea: come facciamo a trovare le informazioni importanti all'interno del Web?

Il Web è enorme e non si possono semplicemente controllare manualmente tutte le pagine.

Un primo tentativo fu quello di avere delle **directory nelle quali delle persone decidevano quali siti fossero importanti**. Degli esempi sono:

- Yahoo
- DMOZ.

Il problema principale è che il web cresce troppo rapidamente per essere organizzato manualmente.

Per questo motivo si passò ai **motori di ricerca**.

L'idea principale dei motori di ricerca è:

dato un insieme di parole, trovare le pagine che contengono le informazioni rilevanti.

Questo è il problema dell'**information retrieval**. Il motore di ricerca deve trovare quindi documenti pertinenti, ma dato che il Web è:

- enorme;
- non strutturato;
- pieno di pagine di bassa qualità;
- pieno di spam;

non basta trovare le pagine che contengono le parole cercate.

Qui entra in gioco la struttura dei link e le sfide importanti poste sono:

- quale fonte è affidabile?
- qual è la migliore risposta?

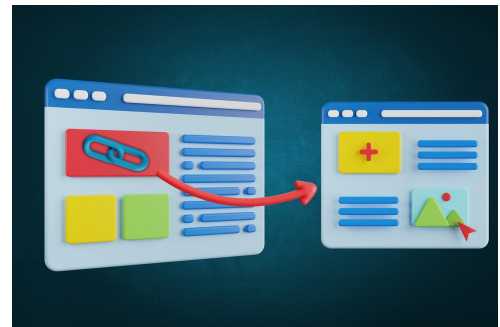
Per rispondere alla prima domanda si può dire che: **le pagine importanti e affidabili tendono ad essere linkate da altre pagine importanti e affidabili.**

Se molte pagine autorevoli fanno riferimento a una determinata pagina, essa si può considerare importante. Quindi il collegamento può essere visto come una sorta di voto e più autorevole è la pagina e più dovrebbe valere il collegamento.

Mentre per rispondere alla seconda domanda, ossia quale pagina sia più corretta, si devono ordinare le pagine per importanza/rilevanza.

Il **ranking dei nodi** è fondamentale perché **non tutte le pagine hanno la stessa importanza**. Il web presenta una grande variabilità nella connettività.

Ad esempio, una pagina può avere pochissimi link in entrata e un'altra pagina ne potrebbe avere tanti in modo spropositato.



Si vuole assegnare ad ogni nodo un rank che rappresenti la sua importanza, che dipende dalla struttura dei link. L'analisi volta a studiare la struttura dei collegamenti si chiama Link Analysis.

Pagerank

Il **pagerank** è un algoritmo che assegna a ogni pagina Web un valore numerico, chiamato rank, che rappresenta la sua importanza.

L'idea fondamentale è intuitiva: una pagina è importante se viene indicata/linkata da altre pagine importanti.

Ogni **link è un voto** e **i voti non sono tutti uguali**, si deve analizzare ricorsivamente le pagine che puntano ad una specifica pagina per sapere quanto sono quelle pagine.

L'importanza di una pagina dipende dall'importanza delle pagine che la linkano.

Supponendo che la pagina i abbia rank r_i e 3 link uscenti, la pagina deve distribuire il proprio voto tra 3 link.

Quindi ogni link riceve:

$$\frac{r_i}{3}$$

Supponendo che j riceva link da i e k, allora il rank di j sarà la somma dei due rank. Se k avesse 4 link uscenti si ha che:

$$r_j = \frac{r_i}{3} + \frac{r_k}{4}$$

Si divide per il numero di link perché una pagina deve distribuire il proprio rank tra tutte le pagine verso cui punta.

Generalizzando, per una pagina j si considerano tutte le pagine che puntano verso la stessa. Il rank di j è la somma dei contributi ricevuti:

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

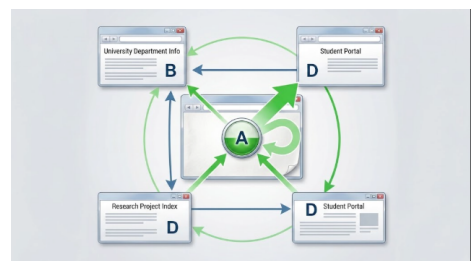
dove:

- d_i è il numero di link uscenti di i

Questa è la formulazione a flusso del pagerank.

Il rank totale del Web è 1 e il web ha miliardi di pagine.

Non si può risolvere manualmente un enorme sistema di equazioni con la Gaussian Elimination, ossia il metodo per risolvere sistemi di equazioni lineari.



Serve una formulazione adatta ai large-scale graphs, ossia la formulazione matriciale. Invece di avere tante equazioni separate, si rappresenta il problema con una matrice.

Si definisce una **matrice M** come segue:

- se $i \rightarrow j$ allora $M_{ji} = \frac{1}{d_i}$;
- se $M_{ji} = 0$;

È una **matrice stocastica per colonne**, dato che i valori rappresentano probabilità/distribuzioni e che la loro somma è 1. Ogni colonna ha somma 1 e ogni pagina distribuisce tutto il proprio rank tra tutti i link uscenti.

La colonna rappresenta da dove parte il rank e le righe rappresentano dove arriva il rank.

Mettendo tutti i rank in un vettore r e imponendo la somma di tutti i rank. Può quindi essere riscritta in forma matriciale come: **Mr** .

Si ha la matrice M con i nodi uscenti e entranti, e il vettore dei rank. Ad esempio:

$$\begin{pmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{pmatrix} \begin{pmatrix} r_y \\ r_a \\ r_m \end{pmatrix} \quad Mr = \begin{pmatrix} \frac{r_y}{2} + \frac{r_a}{2} \\ \frac{r_y}{2} + r_m \\ \frac{r_a}{2} \end{pmatrix}$$

Attraverso la forma matriciale, si può fare il collegamento diretto con la **power iteration**. Infatti, continuando a **moltiplicare per M si arriva verso l'autovettore dominante**, e quello è il **pagerank**.

All'inizio si considerano tutte le pagine ugualmente importanti:

$$r^{(0)} = \begin{bmatrix} 1/N \\ 1/N \\ \vdots \\ 1/N \end{bmatrix}$$

Poi facciamo continuamente:

$$r^{(t+1)} = Mr^{(t)}$$

si prende il PageRank attuale e si fa propagare attraverso il link.

Random Surfer

Il **random surfer** è una **persona che: si trova su una pagina e sceglie casualmente uno dei link disponibili per andare alla pagina successiva**.

$p(t)$ rappresenta **dove probabilisticamente si trova il surfer al tempo t**. Supponendo di avere 3 link, all'inizio potremmo avere il seguente vettore:

$$p(0) = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

Il surfer segue casualmente un link, e la matrice M descrive come le probabilità vengono trasferite da una pagina alle altre. Quindi:

$$p(t+1) = Mp(t)$$

Si usa la matrice M per calcolare dove probabilmente sarà il surfer al passo successivo.

Sapendo che la somma della probabilità è 1, si ha che si arriva ad una distribuzione delle probabilità che non cambia più.

Quello che stiamo facendo esattamente è la power iteration, infatti il prodotto $p(t + 1) = Mp(t)$ è esattamente quello che si fa anche per il rank.

La distribuzione delle probabilità rimane stabile, e quindi anche se il surfer si muove, la distribuzione probabilistica complessiva rimane quella calcolata.

Se prima si era trovata l'equazione $r = Mr$, ora il random surfer ci dà: $p = Mp$. Queste due equazioni sono la stessa equazione e quindi si può interpretare il PageRank nel seguente modo:

Il PageRank di una pagina rappresenta la probabilità, nella situazione stazionaria, di trovare un random surfer su quella pagina.

Teoria delle random walk (catene di Markov)

Per grafi che soddisfano certe condizioni, la distribuzione stazionaria è unica e viene raggiunta indipendentemente dalla distribuzione iniziale.

Le condizioni necessarie sono:

- **il grafo deve essere irriducibile** → ogni nodo deve poter raggiungere ogni altro nodo (anche passando da altri nodi);
- **la catena deve essere aperiodica** → il random surfer non deve essere costretto a seguire un ciclo con una periodicità fissa → non deve essere costretto a fare A,B,A,B..., e quindi che oscilla continuamente e non si stabilizza.

Il problema è che il PageRank base non funziona sempre bene, dato che ha due principali problemi:

- **Spider Trap** → il nodo m ha solamente un link verso se stesso ergo non si può più uscire → tutto il pagerank viene assegnato a m e questo non è quello che vogliamo, dato che m diventerebbe la pagina più importante solo perché è una trappola;
- **Dead End** → il surfer arriva a m, ma non sa dove andare, dato che non ci sono altri link → la somma della colonna è 0 anche se la matrice stocastica dovrebbe avere colonne che sommano a 1.

La soluzione proposta da Google è il **random teleport**, cioè il surfer non deve essere obbligato a seguire sempre un link. Ad ogni passo ha due possibilità:

- con probabilità β → segue un link casuale dalla pagina corrente;
- con probabilità $\beta - 1$ → si teletrasporta verso una pagina casuale del web.

Possiamo infatti vedere, che anche se c'è spider trap, con probabilità $\beta - 1$ prima o poi farà un teleport. La medesima soluzione è per il problema della dead end.

La **nuova formula del PageRank** è la seguente:

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

Nessuna pagina può avere PageRank definitivamente uguale a zero.

La nuova matrice viene chiamata **Google Matrix**:

$$A = \beta M + (1 - \beta) \left[\frac{1}{N} \right]$$

dove:

- la prima parte βM rappresenta il comportamento normale del surfer;
- la seconda parte $(1 - \beta) \left[\frac{1}{N} \right]$ rappresenta il teleport.

La Google Matrix ha una proprietà fondamentale:

tutte le pagine possono essere raggiunte, direttamente o indirettamente, grazie al teleport.

Inoltre, la matrice rimane stocastica ed elimina i problemi di: spider trap, dead end, mancata convergenza o distribuzioni degeneri.

Come si calcola il PageRank?

Matematicamente il PageRank è semplice: si deve fare ripetutamente una moltiplicazione matrice-vettore. Il problema principale è che la matrice del Web è troppo grande per essere memorizzata normalmente.

Supponendo di avere una matrice $A = N \times N$ con $N = 1$ miliardo e se ogni valore occupasse solo 4 byte, occuperemmo exabyte di memoria.

Ma allora perché possiamo lavorare con il web? Perché la matrice è sparsa e la maggior parte delle N^2 possibili connessioni non esiste.

Perciò, invece di memorizzare interamente la matrice si memorizzano solo i link che esistono.

Non si costruisce A esplicitamente, bensì si utilizza il grafo source $\rightarrow$ destination. Questo ci permette di calcolare Mr , senza costruire M .

Importante è considerare che il PageRank ha anche il teleport, quindi oltre a calcolare il peso dei link esistenti si deve calcolare pure $(1 - \beta) \left[\frac{1}{N} \right]$.

Supponendo che esista una dead-end, il suo PageRank non viene distribuito a nessuno. Una parte del PageRank viene persa.

L'algoritmo allora calcola:

$$S = \sum_j r'_j \text{ con } r'_j = \sum_{i \rightarrow j} \beta \frac{r_i^{old}}{d_i}$$

Se $S < 1$ significa che si è persa una quantità di PageRank: $1 - S$.

Questa quantità viene allora redistribuita uniformemente:

$$r_j^{new} = r'_j + \frac{1 - S}{N}$$

E quando i dati sono enormi come si fa? Entra in gioco il disco, infatti:

- M sta sul disco;
- r^{old} sta sul disco;
- r^{new} risiede nella RAM;

Il **Block-Stripe Update Algorithm** è una tecnica che **permette di calcolare la moltiplicazione a blocchi, dividendo sia il vettore r^{new} e sia la matrice sparsa M in parti.**

Questo permette di calcolare separatamente i corrispondenti blocchi del vettore PageRank.

Il problema principale è che si deve leggere tutta la matrice M per il calcolo di ogni blocco(molto costoso). Perciò se si hanno k blocchi:

$$k(|M| + |r|) + |r|$$

Invece, di dividere solo r^{new} si divide anche M, in questo modo non si deve leggere inutilmente tutta M per ogni blocco.

Il PageRank ha alcuni problemi, quali:

- 1) **misura la popolarità generale** → una pagina può avere PageRank elevato perché è molto importante in generale, ma magari non è importante per un certo argomento;
- 2) **link spam** → qualcuno può creare tanti link verso una pagina per aumentarne il PageRank;
- 3) **una sola misura** → PageRank produce un unico punteggio di importanza e per questo vengono introdotti altri metodi, come:
 - a. Topic-Specific PagerankM
 - b. Hubs & Authorities;
 - c. TrustRank;
- 4) **Topic-Specific PageRank** → si pone la domanda su quanto è importante la pagina rispetto a un determinato argomento → si può avere un PageRank diverso per ogni topic.

Il **PageRank** si rende specifico attraverso il random surfer. Difatti, quando si fa il teleport verso una pagina casuale, **si può andare verso qualsiasi pagina.**

Però **nel Topic-Specific PageRank si cambia la regola ponendo il teleport in pagine relative al topic.**

Supponendo che l'argomento sia lo sport, il teleport proporrà pagine casuali relative a questo topic.

Perciò la formula diventa:

$$A_{ij} = \begin{cases} \beta M_{ij} + \frac{1-\beta}{|S|} & \text{se } i \in S \\ \beta M_{ij} & \text{altrimenti} \end{cases}$$

Manipolando β si può decidere se:

- **β alto** → seguire maggiormente i link;
- **β basso** → seguire il teleport/ topic.

Per scegliere quali pagine mettere dentro S, si creano diversi sottoinsiemi(per topic) e poi si sceglie il sottoinsieme più appropriato.

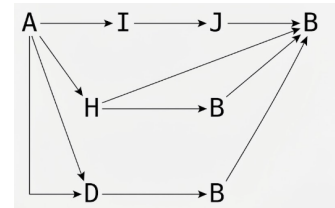
Si utilizza il contesto della query e il contesto dell'utente per capire quale sottoinsieme scegliere.

Prossimità nei grafi

Dato un grafo, quanto sono “vicini” o “simili” due nodi?

Supponendo che per andare da A a B esistano:

- più percorsi;
- percorsi brevi;
- connessioni dirette;
- connessioni indirette;



A e B sono fortemente collegati, ma come definiamo la prossimità tra essi.

La prossimità, infatti, cerca di trasformare questa idea intuitiva in un numero. Una prossimità che ha valore 0.8 indica che A e B sono molto vicini, diversamente da una prossimità di 0.01.

Non si può considerare solo lo shortest path, dato che non considera tutto il resto dei percorsi e l'importanza degli altri nodi collegati/quanto sono importanti.

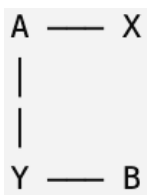
Un'altra idea è analizzare il flusso, cercando capire quanti percorsi esistono tra A e B e la loro struttura/importanza.

Una **buona misura di prossimità dovrebbe considerare** più caratteristiche contemporaneamente, in particolare:

- **connessioni multiple** → se esistono più percorsi;
- **lunghezza dei percorsi** → un percorso più corto dovrebbe contribuire di più
- **tipo di connessioni** → collegamento diretto/indiretto;
- **peso degli archi** → se gli archi hanno dei pesi, questi possono influenzare la prossimità;
- **struttura complessiva dell'arco.**

Entra in gioco **SimRank**, ossia una **misura di similarità tra nodi basata sull'idea che due nodi sono simili se sono collegati a nodi simili.**

La similarità si propaga attraverso il grafo, e il calcolo è ricorsivo. Ad esempio se si ha:



Se X e Y sono simili, allora A e B saranno considerati simili.

E se X e Y sono simili perché i loro vicini sono simili, si ha una definizione ricorsiva.

Perciò: nodi simili hanno vicini simili e ricorsivamente si può dire hanno vicini simili.

La **random walk** è un **modo per esplorare il grafo e ad ogni grafo sceglie casualmente uno degli archi uscenti e segue quell'arco.**

Partendo da X:

- con una certa probabilità va verso A;
- con una certa probabilità va verso B.

Quindi attraverso la random walk possiamo rispondere alla domanda: quali nodi raggiungo frequentemente?

Se il walker tende a raggiungere frequentemente un nodo, questo è un'indicazione che il nodo di partenza è strutturalmente vicino al nodo più frequente.

Gli score prodotti rappresentano una sorta di prossimità rispetto al nodo di partenza.

Utilizzando il **Topic-Specific**, il **teleport** è indirizzato verso un insieme specifico di pagine. Si può restringere ancora di più il teleport attraverso il **SimRank**.

Viene quindi assegnato uno score a pagine di un sottoinsieme, calcolando anche la prossimità. La random walk funziona perché tiene conto della struttura del grafo e fa aumentare lo score a nodi percorsi più frequentemente.

La similarità non deve essere necessariamente dovuta a un collegamento diretto, ma attraverso molteplici percorsi intrapresi più volte.

Si può applicare l'algoritmo anche ad un grafo k-partito (con nodi di natura diversa), basta analizzare sempre quale nodo viene raggiunto frequentemente.

C'è però un problema principale nell'utilizzo del SimRank e RandomWalk; infatti, per conoscere la similarità di ogni nodo con tutti gli altri, potenzialmente si devono eseguire moltissime computazioni.

Quindi è adatto soprattutto a grafi non enormi/ sub-Web-scale, mentre sul Web completo il costo potrebbe essere molto elevato.

Web Spam



Il **Web Spam** è l'insieme delle tecniche utilizzate per manipolare artificialmente il ranking di una pagina nei motori di ricerca, facendo apparire la pagina più importante o rilevante di quanto lo sia realmente.

L'obiettivo è quasi sempre ottenere più visibilità e quindi più utenti.

Ad esempio, si considera un sito che vende pistacchi e vorrebbe comparire quando qualcuno cerca "Assassin's Creed".

Potrebbe quindi inserire migliaia di volte nella pagina la parola chiave sopraccitata. Il motore di ricerca potrebbe pensare che sia rilevante.

I primi motori di ricerca utilizzavano soprattutto le parole contenute nella pagina e per determinare la rilevanza si poteva considerare:

- quante volte compare la parola;
- dove compare, ad esempio nel titolo, header, ecc.

Questa situazione creava una vulnerabilità. Uno spammer poteva inserire queste parole senza che fossero visibili (ad esempio lo stesso colore dello sfondo).

Il **Term Spam** è una tecnica di Web Spam che **consiste nell'inserimento artificiale di molte parole o termini rilevanti, al fine di manipolare il ranking del motore di ricerca.**

L'utente, in questo modo, non vede il testo, ma il motore di ricerca potrebbe analizzarlo.

Per questo motivo google introdusse un'idea molto importante:

Non si guarda soltanto quello che una pagina dice di sé stessa. **Guardiamo anche quello che le altre pagine dicono di lei.** Dopo che google è diventato uno dei motori di ricerca più importanti, gli spammer si sono adattati.

Il **link spam** è una tecnica di Web Spam che **consiste nel creare o manipolare collegamenti artificiali tra pagine web allo scopo di aumentare artificialmente il ranking di una determinata pagina.**

In questo modo, si falsificano i voti utilizzati dal PageRank ed è molto più sofisticato del semplice Term Spam.

Tantissime pagine considerano importanti un sito, ma le stesse pagine sono state create dallo spammer.

Una **sparm farm** è una **struttura artificiale di pagine web e collegamenti creata appositamente per manipolare il PageRank e aumentare il ranking di una o più pagine.**

L'idea è creare una rete di pagine che si collegano strategicamente tra loro per concentrare il PageRank sulla pagina obiettivo.

Ci sono tre tipi di pagine per lo spammer, e si differiscono per quanto lo spammer stesso può controllare. Esse sono:

- **Pagine inaccessibili** → sono pagine che lo spammer non può modificare e sulle quali non ha controllo. Per esempio:
 1. Wikipedia;
 2. Siti universitari;
 3. Giornali;
 4. Blog che non permettono commenti.
- **Pagine accessibili** → sono pagine che lo spammer non possiede direttamente, ma sulle quali esso può comunque riuscire a inserire contenuti o link. Un esempio classico:
 1. Commenti dei blog;
 2. Forum;
 3. Pagine con contenuti generati dagli utenti.
- **Pagine owned** → sono pagine che lo spammer possiede, che controlla completamente e che costituiscono la base della link farm.

La farm page riceve un PageRank e si può calcolare. Si considerano:

- N = numero totale di pagine del Web;
- M = numero di pagine possedute dallo spammer;
- x = PageRank che arriva alla target dalle pagine accessibili;
- y = PageRank totale della target page.

Capiamo quanto PageRank ha ogni farm page:

$$r_{farm} = \frac{\beta y}{M} + \frac{1 - \beta}{N}$$

Come ritorna il PageRank alla pagina target? L'equazione è la seguente:

$$y = x + \beta M \left[\frac{\beta y}{M} + \frac{1 - \beta}{N} \right] + \frac{1 - \beta}{N}$$

che diventa:

$$y = \frac{x}{1 - \beta^2} + c \frac{M}{N} \text{ con } c = \frac{\beta}{1 + \beta}$$

TrustRank

La situazione è grave perché il PageRank nasce come un'idea molto sensata, ma lo spammer cerca di falsificare le informazioni.

Quindi il PageRank da solo non è sufficiente, ed è qui che nasce **TrustRank**.

Il TrustRank è un algoritmo di Web Spam Detection che assegna un punteggio di affidabilità alle pagine web propagando la fiducia da un piccolo insieme di pagine considerate affidabili verso le altre pagine attraverso i link.

Il TrustRank cerca di rispondere alla domanda:

Quanto è probabile che questa pagina sia affidabile, considerando da quali pagine proviene la fiducia?

Il concetto fondamentale di TrustRank è il **seed set**, ossia un **insieme di pagine web che vengono considerate affidabili manualmente o tramite un processo di selezione**. Questi sono i punti di partenza della fiducia, il cui valore di trust è pari a 1.

Il seed set viene manualmente curato e quindi deve essere il più piccolo possibile. Inoltre, con uno short path devono essere raggiunte dalle pagine buone.

La **costruzione del seed set** può avvenire in due modi:

- 1) **PageRank** → **scegliere le top k pagine ottenute col PageRank** → in teoria affidabile dato che non si possono ottenere pagine cattive con un rank alto;
- 2) **Usare domini trusted la cui appartenenza è controllata** → .edu, .mil, .gov

La fiducia viene suddivisa tra gli out-link, ossia il metodo chiamato **trust splitting**, e più out-link ha una pagina, meno trust trasferisce a ciascun link.

Se considerassimo l'Università affidabile, si potrebbe propagare parte della fiducia a pagine a cui punta. Più ci allontaniamo dalle pagine affidabili e generalmente la fiducia tende a diminuire.

Ogni pagina riceve un certo trust score e si potrebbe utilizzare una soglia e decidere se la pagina è affidabile o possibile spam.

Nella realtà c'è un problema più sofisticato: un basso TrustRank non dimostra automaticamente che una pagina sia spam.

Perciò cambia anche il teleport che se nel Topic-Specific è concentrato su un insieme di pagine rilevanti per un determinato argomento, nel TrustRank è concentrato su un insieme di pagine affidabili.

HITS

HITS è un altro modo per affrontare il problema del ranking delle pagine web, ma con un'idea diversa da PageRank.

HITS assegna due punteggi: uno per quanto una pagina è una buona Authority e uno per quanto è un buon Hub.

L'algoritmo HITS o **Hyperlink-Induced Topic Search** è un algoritmo il cui obiettivo è sempre quello di utilizzare la struttura dei link del Web per capire quali pagine siano importanti. Però HITS introduce una distinzione fondamentale:

- Alcune pagine sono importanti perché contengono informazioni di qualità;
- Altre pagine sono importanti perché indicano molte pagine di qualità.

Queste due tipologie di pagine vengono chiamate: Authority e Hub.

Un' **Authority** è una pagina che contiene informazioni autorevoli o di alta qualità su un determinato argomento. Ad esempio, i siti dell'università, quotidiani online ecc. .

Una buona Authority è una pagina verso cui puntano molti buoni Hub.

Un Hub è invece una pagina che contiene link verso molte buone Authority. Per esempio, una pagina che classifica i migliori quotidiani italiani.

Questa pagina non necessariamente contiene le informazioni sui migliori quotidiani, però è un ottimo punto di collegamento verso le pagine importanti.

La caratteristica fondamentale di Hits è che: Hubs e Authority si rafforzano reciprocamente.

C'è della ricorsione in HITS, cioè:

- una pagina è una buona Authority se riceve link da buoni Hub;
- una pagina è un buon Hub se punta a buone Authority.

Una pagina può essere quindi:

- 1) un'ottima Authority;
- 2) un ottimo Hub;
- 3) entrambe;
- 4) nessuna delle due.

L'authority si calcola assegnando all'inizio un Hub score iniziale. Poi si aggiornano gli Hub e si ricalcolano le Authority. HITS è un algoritmo iterativo e le formule di Authority e Hub sono le seguenti:

- **Authority** → è la somma degli Hub che puntano verso i:

$$a_i = \sum_{j \rightarrow i} h_j$$

- **Hub** → la somma delle Authority verso cui i punta:

$$h_i = \sum_{i \rightarrow j} a_j$$

Se si continua a sommare, i valori diventano sempre più grandi, per questo bisogna normalizzare. La norma euclidea dei vettori viene portata a 1.

Si può collegare HITS alla formulazione matriciale. Sia A la matrice di adiacenza dove:

- $A_{ij} = 1$ se esiste un arco che parte da i e va a j ;
- $A_{ij} = 0$ altrimenti.

Si hanno infine i due vettori degli Hub e Authorities. Per il calcolo degli hub la funzione è la seguente:

$$h_i = \sum_j A_{ij} a_j$$

ossia $h = Aa$.

Invece, per il calcolo della matrice delle Authority si ha:

$$a_i = \sum_j A_{ji} h_j$$

che corrisponde a: $a = A^T h$.

Si può utilizzare anche in questo caso la power iteration e si ha:

$$a = (A^T A) a$$

e

$$h = (A A^T) h$$

si può risolvere questo problema utilizzando gli autovettori.

La differenza tra HITS e il PageRank è quindi sostanziale:

- 1) PageRank ha un solo score r_i che rappresenta l'importanza, invece in HITS ogni pagina ha due score h_i e a_i ;
- 2) PageRank è pensato come una misura globale dell'importanza delle pagine, HITS è legato alla query/topic → HITS è fortemente collegato al contesto della ricerca.

Nonostante risolvano lo stesso problema, sono due soluzioni che offrono vantaggi diversi.

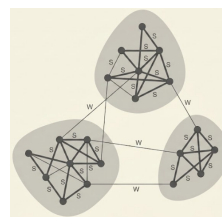
Comunità nelle Reti

Le **reti possono essere divise in comunità densamente connesse**.

I legami interni tendono a essere forti e ridondanti, mentre i legami tra comunità tendono ad essere deboli ma molto importanti per far circolare nuove informazioni.

Una comunità è un insieme di nodi che hanno molte connessioni tra loro e relativamente poche connessioni con nodi esterni.

Elevata densità di connessioni interne e bassa densità di connessioni verso l'esterno.



Quando si analizza una rete vogliamo capire come è organizzata. Ad esempio:

- quali nodi appartengono allo stesso gruppo?
- quali nodi fanno da collegamento tra gruppi diversi?
- quali nodi sono importanti per la diffusione dell'informazione?
- quali connessioni sono ridondanti?

Non interessa sapere quali nodi sono collegati, ma perché sono collegati e quale ruolo hanno le connessioni.

The **Stretght of Weak Ties** è l'idea fondamentale che sviluppò Mark Granovetter, che apparentemente paradossale dice:

I legami deboli possono essere più utili dei legami forti per ottenere informazioni nuove.

Questo perché l'informazione all'interno dei legami forti può essere ridondante, infatti per esempio:

probabilmente gli amici stretti di una persona conoscono le stesse persone e quindi le informazioni non sono nuove tra di loro. Immaginando di avere un conoscente che appartiene ad un gruppo completamente diverso: è collegato ad una rete che normalmente non è raggiungibile.

Il conoscente *k* può portare: informazioni nuove, opportunità di lavoro, contatti nuovi e informazioni proveniente da altre comunità.

Questa è l'idea fondamentale dei weak ties o legami deboli.

Importante è distinguere due prospettive diverse dalle quali si può guardare un'amicizia:

1. **prospettiva strutturale** → dove si colloca il legame all'interno della rete → la posizione quindi è importante, infatti si ha:
 - a. **legami interni** → collegano persone appartenenti alla stessa comunità;
 - b. **legami tra comunità** → collegano parti diverse della rete;
2. **prospettiva intrapersonale** → guarda alla forza della relazione tra due nodi, infatti si ha:
 - a. **strong tie**(relazione forte) → migliore amico, familiare o collega con cui si lavora spesso → **tende ad avere molta sovrapposizione di vicini**;
 - b. **weak tie**(relazione debole) → conoscente, persona incontrata occasionalmente, contatto professionale → **può collegare due gruppi diversi**;

La **chiusura triadica** è un principio secondo cui, se due nodi (A e B) hanno un nodo in comune(C), è molto probabile che prima o poi ci sarà un arco tra due nodi.

La chiusura triadica è strettamente collegata al coefficiente di clustering, ossia il coefficiente che misura quanto i vicini di un nodo sono collegati tra loro.

Un alto coefficiente di clustering implica una forte presenza di chiusure triadiche.

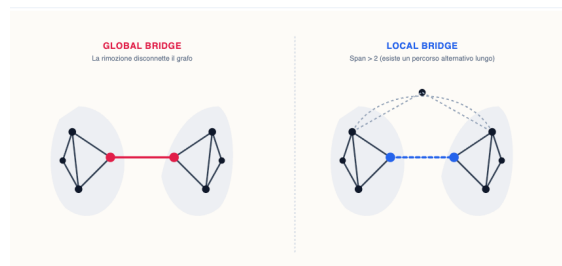
Un **bridge** è un **arco** che, **se rimosso, rende il grafo disconnesso**. In questo caso, possiamo considerare un ponte un conoscente che conosce due persone di gruppi diversi.

Il **local bridge** è diverso, infatti, un arco $i - j$ è considerato local bridge se, **eliminandolo, la distanza tra i suoi estremi diventa maggiore di 2**.

Si definisce lo span, come la distanza tra i e j dopo aver eliminato l'arco $i - j$. Si definisce:

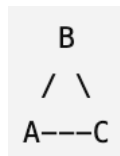
$$\text{span}(i, j) = d_{G-(i,j)}(i, j)$$

Una distinzione importante e utile è la differenza tra il bridge e il local bridge. Il bridge se rimosso rende il grafo disconnesso, invece il local bridge se eliminato gli estremi dell'arco diventano molto più lontani, precisamente ad una distanza >2 .



In una chiusura triadica, un local bridge tende ad essere weak. Prendendo in considerazione un triangolo, se rimosso un arco, ad esempio $A - B$ il percorso è di lunghezza 2.

In questo caso $A - B$ non può essere un local bridge perché esiste un percorso breve.



Un local bridge associato ad un nodo che soddisfa la chiusura triadica forte deve essere un legame debole.

L'overlap dei vicini misura quanto due nodi condividono i loro vicini. La formula è:

$$O_{ij} = \frac{|N(i) \cap N(j)|}{|N(i) \cup N(j)|}$$

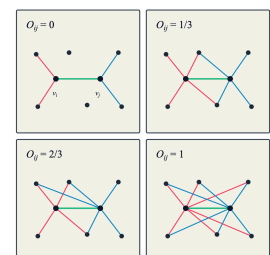
dove:

- $N(i)$ = insieme dei vicini di i ;
- $N(j)$ = insieme dei vicini di j ;
- $|N(i) \cap N(j)|$ = vicini comuni;
- $|N(i) \cup N(j)|$ = vicini comuni a tutti i nodi.

È una similarità di Jaccard tra i neighborhood.

L'overlap ci da informazioni sul tipo di nodo, infatti:

- se l'overlap è alto significa che i nodi coinvolti dall'arco appartengono alla stessa comunità;
- se l'overlap è basso significa che i nodi coinvolti fanno parte di una connessione tra comunità.



Se $A - B$ è un bridge, non può esistere un vicino comune C , per questo motivo hanno $\text{Overlap} = 0$. Per i local bridge, dato che la distanza è maggiore di 2, l'overlap sarà anch'esso 0. Quindi: $O_{ij} \rightarrow 0$ tende a collegare diverse zone della rete, mentre $O_{ij} \rightarrow 1$ significa che i due nodi sono immersi in una struttura molto simile.

Se vengono rimossi gli archi con basso neighborhood overlap sono spesso strutturalmente importanti perché collegano comunità diverse. Se rimossi, potrebbero causare disconnessione della rete.

La teoria dei legami deboli di Granovetter porta naturalmente a pensare che una rete sociale non sia completamente uniforme, ma possa essere organizzata in gruppi di nodi fortemente connessi tra loro. Una community presenta:

- molte connessioni interne;
- poche connessioni verso l'esterno.

A seconda del contesto, si può trovare:

- community;
- cluster;
- gruppo;
- modulo.

Una volta che si ha una rete, ci possiamo chiedere: si può individuare automaticamente i gruppi che la compongono?

Questo è il problema della **Community Detection**, che si pone come obiettivo, trovare una partizione della rete tale che i nodi appartenenti allo stesso gruppo siano fortemente connessi.

Uno degli esempi più famosi è il Zachary's Karate Club. Questa è una rete sociale che rappresenta le relazioni tra i membri di un club di karate.

Durante lo studio della rete, il club si divide realmente in due fazioni. Si può quindi usare l'esempio per verificare un algoritmo:

- diamo all'algoritmo la rete;
- l'algoritmo individua le due comunità;
- si confrontano le comunità trovate con le due fazioni realmente osservate.

Questo è un esempio importante perché si ha una sorta di ground truth, cioè una suddivisione reale con cui confrontare il risultato.

Come si trovano le comunità? **L'intuizione è quella dei ponti, infatti un arco che collega due comunità tende ad essere attraversato da molti cammini che devono andare da una comunità all'altra.**

Questo porta alla misura di Edge Betweenness, che misura quanto un arco è importante nel collegare diverse parti della rete. Si considerano gli shortest path passanti attraverso un arco e si calcolano nel seguente modo.

$$EB(e) = \sum_{s \neq t} \frac{\sigma_{st}(e)}{\sigma_{st}}$$

Si fa questo calcolo per tutte le coppie di nodi e si sommano i contributi.

L'algoritmo di Girvan-Newman è un algoritmo di Community Detection gerarchica basato sulla edge betweenness. L'idea è molto intuitiva: **se gli archi con betweenness elevata sono probabilmente ponti tra comunità, si rimuovono.**

I passi dell'algoritmo sono i seguenti:

- 1) si calcola la edge betweenness di tutti gli archi;
- 2) si trova l'arco con betweenness maggiore;
- 3) si rimuove;
- 4) si ricalcola e rimuove l'arco con betweenness maggiore iterativamente.

Si ricalcola la betweenness perché quando si elimina un arco, si cambia la struttura della rete.

Girvan-Newman non restituisce solo le comunità, ma bensì **produce una gerarchia di suddivisioni.**

All'inizio si ha l'intera rete e si arriva ad un **dendrogramma**, ossia un albero che mostra come i dati o gli oggetti sono raggruppati in base alla loro somiglianza.



Si può infine scegliere un determinato livello della gerarchia e ottenere il numero di comunità a cui si è interessati.

Questa è la ragione per cui Girvan-Newman è definito clustering gerarchico.

I due problemi principali di questo algoritmo sono:

- 1) come si calcola efficientemente la betweenness degli archi?
- 2) come vengono selezionati i cluster?

Per calcolare la betweenness degli archi, si parte da un nodo sorgente, per esempio A. Si utilizza una **Breadth-First Search (BFS)**, ossia una ricerca in ampiezza, e si visita il grafo per livelli di distanza.

Si considera un grafo non pesato, dove ogni arco ha costo 1, e la BFS garantisce che quando si raggiunge un nodo per la prima volta si raggiunge mediante attraverso un shortest path.

Considerando l'albero, **si può calcolare il numero di shortest path verso un nodo attraverso la seguente formula:**

$$\sigma(A, X) = \sum_{P \in \text{Pred}(X)} \sigma(A, P)$$

dove **Pred(X)** sono i predecessori di X nell'albero BFS.

Il numero di shortest path verso un nodo è la somma del numero di shortest path verso i suoi predecessori.

Se esistono più shortest path, il contributo viene distribuito frazionariamente.

Per fare questo calcolo viene introdotto il **concetto di flow**, e quindi che ogni nodo abbia una certa quantità di "flusso" che deve essere distribuita verso i suoi predecessori.

Il flow di un nodo è pari a:

$$\text{flow del nodo} = 1 + \sum \text{flow dei figli}$$

si aggiunge 1 perché si considera anche il nodo stesso come destinazione.

Quindi ogni nodo inizialmente ha un'unità di flusso destinata a sé stesso e poi si aggiunge il flusso che arriva dai nodi che dipendono dal nodo preso in considerazione.

Dopo che la BFS costruisce la struttura per calcolare il flow si deve procedere dal basso verso l'alto, per capire il flusso derivato dai figli.

Se i predecessori non hanno lo stesso numero di shortest path, il flusso viene diviso in proporzione.

Supponendo che J abbia un numero di shortest path doppio rispetto a I, si ha che:

- $\sigma(I) = 1$
- $\sigma(J) = 2$

E quindi il flow disponibile sarà:

- $I \rightarrow \frac{1}{3}$
- $J \rightarrow \frac{2}{3}$

Alla fine di questo procedimento si può sapere: quanto contributo di shortest path passa attraverso ogni arco.

Si ripete tutto il procedimento, ossia BFS + flow, per ogni nodo della rete come sorgente. Il risultato è la Edge Betweenness di ogni arco.

Questo risponde alla prima domanda, ossia: quali archi hanno maggiore edge betweenness?

L'algoritmo Girvan-Newman rimuove l'arco con EB massima e ricomincia il calcolo perché la rete è cambiata.

La seconda domanda, ossia: come facciamo a capire se una certa partizione della rete è realmente una buona divisione in comunità? troverà risposta nella parte sottostante.

Per rispondere introduciamo la **modularità**, che non cerca semplicemente gruppi con molti archi: **confronta gli archi che si osservano nella rete con quelli che ci aspetteremmo di osservare in una rete casuale equivalente.**

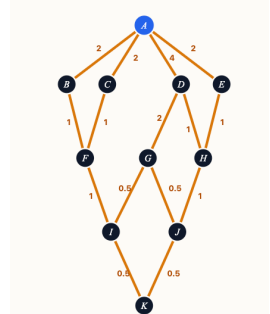
Una buona comunità ha più connessioni interne di quante ne avrebbe per caso.

In forma intuitiva, la modalità Q è:

$$Q \propto \sum_{s \in S} [\text{archi osservati dentro } s - \text{archi attesi dentro } s]$$

Quindi:

- se ci sono molti archi interni rispetto all'atteso $\rightarrow$ Q alta;
- se il numero di archi è simile all'atteso $\rightarrow$ Q circa 0;
- se il numero di archi è minore dell'atteso $\rightarrow$ Q negativa.



Il null model è un modello di riferimento che conserva alcune caratteristiche importanti della rete reale, ma elimina la struttura che si vuole verificare.

Perciò si costruisce una rete casuale che mantenga la degree distribution, ma che assegni gli archi casualmente.

Dato: $G = (V, E)$ con:

- n nodi;
- m archi;
- grado k_i del nodo i ;

si costruisce una rete casuale G' in cui ogni nodo mantiene il proprio grado, ma le connessioni vengono randomizzate.

Generalmente si considera un multi-grafo, dato che potrebbero esserci self-loop o archi multipli.

Il grado si deve mantenere, perché altrimenti considereremmo la rete con un grafo completamente casuale, ed il configuration model evita questo problema.

Considerando due nodi i e j , il numero atteso di archi tra loro è:

$$\frac{k_i k_j}{2m}$$

Più grande è il valore di k_i e k_j , più è probabile che i due nodi siano collegati nel modello casuale.

Per un grafo non diretto:

$$\sum_i k_i = 2m$$

Questa proprietà viene utilizzata nella derivazione della modularità.

La formula completa è:

$$Q(G, S) = \frac{1}{2m} \sum_{s \in S} \sum_{s' \in S} \sum_{s'' \in S} (A_{ij} - \frac{k_i k_j}{2m})$$

Dove $(A_{ij} - \frac{k_i k_j}{2m})$ indica quanto la connessione osservata differisce da quella attesa.

Solitamente un valore di $Q > 0.3$ indica una struttura comunitaria significativa, anche se non esiste una soglia universale.

La modularità può essere utilizzata anche per decidere quante comunità si devono creare. Infatti, si può scegliere la partizione che massimizza Q .

Il problema diventa un problema di ottimizzazione per trovare la partizione S che massimizza la modularità.

Supponiamo inizialmente di voler dividere la rete in due gruppi. Si introduce un vettore:

$$s \in \{-1, +1\}^n$$

e per ogni nodo:

$$s_i = \begin{cases} +1 & \text{comunità 1} \\ -1 & \text{comunità 2} \end{cases}$$

Si considera allora la formula:

$$\frac{s_i s_j + 1}{2}$$

e se i nodi appartengono alla stessa comunità il valore è 1 altrimenti 0.

La **matrice di modularità** si definisce come:

$$B_{ij} = (A_{ij} - \frac{k_i k_j}{2m})$$

- se $B_{ij} > 0$ i due nodi sono collegati più di quanto ci aspetteremmo;
- se $B_{ij} < 0$ i due nodi sono collegati meno di quanto ci aspetteremmo;
- se $B_{ij} \approx 0$ comportamento simile al modello casuale.

La modularità può essere riscritta come:

$$Q(G, s) = \frac{1}{4m} s^T B s$$

Adesso si deve trovare s che massimizza $s^T B s$. Questo permette di utilizzare gli autovalori e gli autovettori della matrice B .

Dato che la matrice è simmetrica, può essere diagonalizzata tramite:

$$B = X \Lambda X^T$$

dove:

- **X** contiene gli autovettori;
- **Λ** contiene gli autovalori.

Questo è equivalente a:

$$B = \sum_i \lambda_i x_i x_i^T$$

dove:

$$B x_i = \lambda_i x_i$$

L'autovalore indica, quanto è importante quella direzione nella struttura rappresentata dalla matrice.

Possiamo quindi sostituire nella formula principale:

$$Q(G, s) = \frac{1}{4m} \sum_i (s^T x_i)^2 \lambda_i$$

Se un autovalore è molto grande, il relativo autovettore può contribuire molto alla modularità. Per questo ci interessa l'autovettore associato al massimo autovalore della matrice di modularità.

Dopo che si è trovato per ogni nodo i :

$$s_i = \begin{cases} +1 & \text{comunità 1} \\ -1 & \text{comunità 2} \end{cases}$$

Si può ripetere il procedimento gerarchicamente sui gruppi ottenuti. Continuando a dividere finché uno split non migliora più la modularità.

Anche in questo caso, come visto precedentemente, si può utilizzare la power iteration per trovare il massimo autovalore in valore assoluto. Introducendo questo modo si riduce il calcolo degli autovalori.

In questo modo, troveremo efficientemente una partizione con modularità elevata.

Spectral Clustering

La **Spectral Clustering** è una **tecnica per dividere i nodi di un grafo in gruppi**, chiamati **cluster** (o comunità), facendo in modo che:

- i nodi dello stesso gruppo siano fortemente connessi tra loro;
- i nodi appartenenti a gruppi diversi siano poco connessi.

L'idea fondamentale è trasformare il problema in un problema di algebra lineare, studiando autovettori e autovalori.

Il termine spectral deriva proprio dallo spettro della matrice, o insieme degli autovalori.

Supponendo di avere un grafo non direzionato $G = (V, E)$, si vogliono dividere i vertici in due gruppi: $A \cup B = V$. Ogni nodo deve appartenere ad uno e un solo gruppo.

L'obiettivo è capire se questa divisione è una buona partizione. Le domande fondamentali che si devono porre sono:

- 1) come si definisce matematicamente una buona partizione?
- 2) come si trova efficientemente questa partizione?

Una buona partizione dovrebbe avere due caratteristiche:

- **molte connessioni all'interno dei gruppi** → ci devono essere molti archi tra nodi appartenenti allo stesso gruppo → **archi intra-group**;
- **pochissime connessioni tra gruppi diversi** → pochi archi che attraversano la divisione → **archi inter-group**.

Il **cut** misura il peso degli archi che attraversano la partizione. Considerando due gruppi A, B formalmente:

$$cut(A, B) = \sum_{i \in A, j \in B} \omega_{ij}$$

dove ω_{ij} è il peso dell'arco tra i e j.

Il **cut** ci dice quanto è forte il collegamento tra due gruppi e un cut piccolo si traduce in gruppi poco collegati tra loro.

Una prima idea potrebbe essere cercare la partizione che minimizza il cut. Formalmente:

$$\arg \min_{A, B} cut(A, B)$$

Quindi si vogliono trovare A e B tali che gli archi tra loro siano il minor numero possibile.

Si potrebbe ottenere:

$A = \{6\}$ e $B = \{1, 2, 3, 4, 5\}$ e il cut potrebbe essere soltanto 1.

Anche se matematicamente si è trovato un ottimo cut, questa è una partizione poco interessante. Si è trovato un nodo isolato.

Il **minimum-cut** considera solamente: **quanti archi attraversano il confine e non quanto sono grandi o densamente connessi i due gruppi**. Questo è il problema che cerca di risolvere il Normalized Cut.

Il **Normalized Cut**, cerca di tenere conto anche della dimensione/connettività dei gruppi. La formula è:

$$\text{ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{vol}(A)} + \frac{\text{cut}(A, B)}{\text{vol}(B)}$$

e analogamente:

$$\text{vol}(A/B) = \sum_{i \in A/B} k_i$$

Qui k_i è il grado del nodo i , ossia il numero/peso degli archi. In sostanza, il **volume** è la **quantità totale di connessioni associate ai nodi di A**.

Il Normalized Cut misura qualcosa di intelligente, ossia:

quanto è importante il collegamento tra i due gruppi rispetto alla loro connettività complessiva.

Produce partizioni più bilanciate, dato che isolare un singolo nodo non è necessariamente una soluzione conveniente.

Dato che si è definito un criterio molto più interessante, cioè min cut, il problema è che trovare esattamente il miglior Normalized Cut è un problema NP-hard.

Provare tutte le possibili partizioni non è efficiente, per questo motivo si arriva a Spectral Graph Partitioning.

Invece di lavorare direttamente con tutti i possibili gruppi, si rappresenta il grafo attraverso una matrice. La prima matrice considerata è quella di adiacenza A .

Per un grafo non pesato:

$$A_{ij} = \begin{cases} 1 & \text{se } (i, j) \in E \\ 0 & \text{altrimenti} \end{cases}$$

Si considera un vettore x dove x_i è un'etichetta/valore assegnato al nodo i . Quando si fa $y = Ax$ si ottiene:

$$y_i = \sum_{j=1}^n A_{ij}x_j$$

Il nuovo valore del nodo i è la somma dei valori dei suoi vicini. La matrice di adiacenza permette di propagare le informazioni dai nodi ai loro vicini.

Si arriva al cuore della Spectra Graph Theory, chiedendoci se esiste un vettore x che moltiplicato per A mantiene la stessa forma, cambiando solo la scala.

Cioè:

$$Ax = \lambda x$$

Un vettore che soddisfa questa proprietà è un autovettore di A e λ è il suo autovalore.

Alcuni autovettori permettono di distinguere gruppi di nodi fortemente connessi. Il problema del partizionamento del grafo si tramuta nell'analisi delle coordinate ottenute dagli autovettori.

Se due nodi appartengono alla stessa comunità, ci aspettiamo che abbiano valori simili nell'autovettore utilizzato per la partizione.

Per analizzare il problema potremmo utilizzare un esempio molto semplice. Utilizzando un grafo d-regular con $d=3$, si può assegnare a tutti i nodi 1, ovvero la componente del vettore x associato a ciascun nodo.

Applicando A , ogni nodo somma i valori dei suoi vicini e dato che ogni nodo vicino vale 1, si ottiene sempre $d \rightarrow$ il vettore rimane nella stessa direzione.

Supponendo adesso che il grafo non sia connesso e abbia due componenti d-regular, si può costruire il vettore:

$$x' = (1, \dots, 1, 0, \dots, 0)$$

dove:

- i nodi appartenenti ad A hanno la componente del vettore $x = 1$;
- i nodi appartenenti ad B hanno la componente del vettore $x = 0$;

Si ottiene quindi:

$$Ax' = (d, \dots, d, 0, \dots, 0)$$

ossia:

$$Ax' = dx'$$

Perciò, si trova un altro autovettore con:

$$\lambda = d$$

Si hanno quindi due autovettori distinti con lo stesso autovalore d . Questo ci dà un'informazione sulla struttura del grafo: più componenti separate implicano un autovalore massimo ripetuto.

Nel caso di due componenti:

$$\lambda_n = \lambda_{n-1}$$

Se le due componenti sono debolmente collegate, non sono perfettamente separate. Gli autovalori non saranno esattamente uguali, ma molto vicini.

$$\lambda_n - \lambda_{n-1} \approx 0$$

Se due archi sono collegati da pochissimi archi, dal punto di vista della struttura del grafo sono quasi due componenti separate, e lo spettro della matrice lo mostra.

Dato che la matrice di adiacenza è simmetrica, i suoi autovettori possono essere scelti ortonormali.

Quindi:

$$x_n \cdot x_{n-1} = 0$$

Ma $x_n = (1, \dots, 1)$ e perciò:

$$x_n \cdot x_{n-1} = \sum_i x_{n-1}[i]$$

Quindi, $\sum_i x_{n-1}[i] = 0$, dato che si sommano tutte le componenti dell'autovettore e si ottiene 0.

Se si avessero 6 nodi, il vettore potrebbe essere:

$$x_{n-1} = \begin{bmatrix} 0.8 \\ 0.7 \\ 0.6 \\ -0.6 \\ -0.7 \\ -0.8 \end{bmatrix}$$

Allora, $\sum_i x_{n-1}[i] = 0$ dato che si somma: $0.8 + 0.7 + 0.6 - 0.8 - 0.7 - 0.6 = 0$.

Il secondo autovettore più grande ci dà una direzione lungo la quale si può separare il grafo. Il primo contiene un'informazione poco utile per separare i nodi.

Dato che il secondo autovettore introduce una variazione dei nodi, si può sfruttare per fare il clustering.

Ora si possono costruire le matrici necessarie, infatti si ricorda che: la matrice di adiacenza A è una matrice $n \times n$ simmetrica che soddisfa la proprietà $A = A^T$. È necessario che sia simmetrica per avere autovalori reali e gli autovettori che possono essere scelti ortogonali (il cui prodotto è 0).

Si introduce la matrice dei gradi D , cioè una matrice diagonale contenente i gradi del nodo i .

$$D = \begin{bmatrix} d_1 & 0 & 0 & 0 \\ 0 & d_2 & \dots & 0 \\ 0 & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & d_n \end{bmatrix}$$

Infine, si arriva ad una matrice fondamentale per la Spectral Clustering, ovvero la **matrice Laplaciana** del grafo:

$$L = D - A$$

È costruita sottraendo la matrice di adiacenza dalla matrice dei gradi. Quello che troviamo nella matrice risultante è:

- sulla diagonale si trovano i gradi;
- fuori diagonale si trova -1 se esiste un arco, 0 altrimenti.

La proprietà più importante della Laplaciana è la seguente:

$$Lx = (D - A)x$$

ricordando che $Dx = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_n \end{bmatrix}$ e $Ax = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_n \end{bmatrix}$ si ha che $Lx = 0x$.

Pertanto, $\lambda_1 = 0$ e $x = (1, 1, \dots, 1)$ è un autovettore della Laplaciana associato all'autovalore 0.

La Laplaciana ha una proprietà fondamentale e molto interessante cioè:

$$\lambda_i \geq 0$$

quindi tutti gli autovalori sono reali e non negativi. Si possono ordinare:

$$0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$$

Il secondo autovalore della Laplaciana viene spesso chiamato algebraic connectivity o Fielder Value. Il suo valore dà informazioni su quanto il grafo sia connesso, maggiore è e più fortemente è connesso il grafo.

La matrice di incidenza serve a rappresentare la relazioni tra nodi e archi di un grafo. Se il grafo ha:

- n nodi;
- m archi;

Nel caso di un grafo orientato, per ogni arco assegniamo:

- -1 al nodo da cui parte l'arco(coda);
- +1 al nodo verso cui arriva l'arco(testa);
- 0 se non esiste l'arco.

La **matrice di incidenza permette di costruire la Laplaciana**. Dopo aver dato un orientamento arbitrario agli archi di un grafo non orientato, si può scrivere:

$$L = N^T N$$

Le proprietà della Laplaciana sono le seguenti:

- 1) **gli autovalori non sono negativi** $\rightarrow \lambda \geq 0$;
- 2) **forma quadratica non negativa** $\rightarrow$ per qualsiasi vettore x :

$$x^T L x \geq 0$$

- 3) **la Laplaciana può essere scritta come:**

$$L = N^T N$$

Le tre proprietà sono collegate tra loro. Si può spiegare perché $x^T L x \geq 0$.

Partendo da: $L = N^T N$, allora $x^T L x = x^T N^T N x$, che si può riscrivere come $x^T N^T N x = (N x)^T (N x)$, ossia il prodotto scalare di un vettore con sé stesso.

Se $z = N x$, si ha che: $z^T z = z_1^2 + z_2^2 + \dots + z_m^2$ e una somma di quadrati non può essere negativa. Perciò:

$$x^T L x \geq 0$$

Da qui possiamo dire anche che gli autovalori non sono negativi perché $x^T L x \geq 0$ e per un autovettore non nullo $x^T x > 0$. Pertanto, necessariamente:

$$\lambda \geq 0$$

Infine:

$$x^T L x = \sum_{(i,j) \in E} (x_i - x_j)^2$$

Assegnando un valore x a ogni nodo, se due nodi collegati hanno valori simili allora $(x_i - x_j)^2 \approx 0$. $x^T L x$ misura quanto i valori assegnati ai nodi cambiano lungo gli archi del grafo.

Gli unici archi che producono un costo elevato sono quelli che attraversano la divisione. Bisogna minimizzare $x^T L x$, cercando un'assegnazione dei valori che eviti grandi differenze lungo gli archi.

Per una matrice simmetrica, il secondo autovalore può essere caratterizzato tramite un problema di minimizzazione:

$$\lambda_2 = \min_x \frac{x^T L x}{x^T x}$$

con i vincoli appropriati, in particolare che x sia ortogonale al primo autovettore.

Sostituendo con l'interpretazione di $x^T L x$ si ha:

$$\lambda_2 = \min \frac{\sum_{(i,j) \in E} (x_i - x_j)^2}{\sum_i x_i^2}$$

con $\sum_i x_i = 0$ e $\sum_i x_i = 1$ perché il primo autovettore della Laplaciana è $\mathbf{1} = (1, 1, \dots, 1)$.

In pratica si impedisce al problema di scegliere la soluzione banale.

Minimizzare la quantità tende a trovare una divisione con pochi archi tra i nodi, ed equivale sostanzialmente a minimizzare il cut.

Il problema principale è che y può avere solo due valori $y_i \in \{-1, +1\}$, perché ogni nodo può stare in A oppure in B. Si ammette pertanto di avere valori reali e avere: 0.7, 0.4, -0.2, 0.8

Si è trasformato un problema combinatorio difficile in un problema di algebra lineare.

Si utilizza il **teorema di Rayleigh**, ossia se L è una matrice simmetrica, il minimo del rapporto tra $x^T L x$ e $x^T x$, rispettando l'ortogonalità al primo autovettore, è proprio il secondo autovalore.

Utilizzando il **quoziente di Rayleigh** si ha:

$$\lambda_2 = \min \frac{\sum_{(i,j) \in E} (y_i - y_j)^2}{\sum_i y_i^2}$$

Questo vettore viene chiamato Fiedler vector, ed è il secondo autovettore della matrice Laplaciana, associato al secondo autovalore più piccolo λ_2 .

Ma quanto è buona la soluzione trovata dallo Spectral Clustering? Supponendo di avere una partizione A, B con $|A| \leq |B|$, si definisce:

$$\alpha = \frac{\#archi \text{ tra } A \text{ e } B}{|A|}$$

Quindi α misura **quanto è costoso il taglio rispetto alla dimensione del gruppo più piccolo**. C'è un bound teorico per la Spectral Clustering, cioè:

$$\lambda_2 \leq 2\alpha$$

e non è pessimo.

Si può dimostrare questa approssimazione. Si pone $a = |A|$ e $b = |B|$ ed $e = \#archi \text{ tra } A \text{ e } B$.

Si costruisce un vettore particolare con:

$$x_i = \begin{cases} -\frac{1}{a} & i \in A \\ +\frac{1}{b} & i \in B \end{cases}$$

Il vettore è ortogonale al vettore di tutti 1, infatti $a(-\frac{1}{a}) + b(+\frac{1}{b}) = -1+1 = 0$. Avendo tutti lo stesso peso, gli archi interni non contribuiscono al costo. Ad esempio, se si ha $i, j \in A$ allora:

$$(x_i - x_j)^2 = 0$$

Se gli archi attraversano il cut, molto semplicemente se $i \in A$ e $j \in B$ allora:

- $x_i = -\frac{1}{a};$
- $x_j = +\frac{1}{b};$

Si avrà infine: $\left(-\frac{1}{a} - (+\frac{1}{b})\right)^2$

Ci sono e numero di archi di questo tipo e quindi otteniamo che:

$$e\left(-\frac{1}{a} - (+\frac{1}{b})\right)^2$$

Dato che $a \leq b$ e che $\frac{1}{b} \leq \frac{1}{a}$, si può dire:

$$\frac{1}{b} + \frac{1}{a} \leq \frac{2}{a}$$

Siccome si considerano gli archi e allora:

$$\lambda_2 \leq \frac{2e}{a}$$

ma $\frac{e}{a} = \alpha$, perciò:

$$\lambda_2 \leq 2\alpha$$

Algoritmo Spectral Clustering

La **Spectral Clustering** è una tecnica di clustering che sfrutta le proprietà spettrali di una matrice associata ai dati, normalmente la matrice Laplaciana del grafo.



L'algoritmo può essere visto come composto da tre fasi principali:

- **Pre-processing** → si costruisce un grafo che rappresenta le relazioni tra i punti e poi una matrice che lo rappresenta → $L = D - A$;
- **Decomposizione spettrale** → si calcola $Lx = \lambda x$ → gli autovettori forniscono una nuova rappresentazione dei nodi. Per una bipartizione potrebbe essere utile x_2 , il secondo vettore della Laplaciana, cioè il Fiedler Vector.
- **Grouping** → si utilizzano i valori contenuti nell'autovettore per raggruppare i nodi → ad esempio $A = \{i : x_2[i] > 0\}$ e $B = \{i : x_2[i] < 0\}$ → si ottengono i cluster.

Solitamente la **soglia per il grouping è 0**, ma si possono avere approcci più sofisticati per trovare il valore di splitting.

Si può effettuare lo **split mediante mediana**, garantendo anche una certa forma di bilanciamento.

Un altro metodo per splittare è prendere gli elementi contenuti nel Fiedler vector e **provare diversi punti di separazione**.

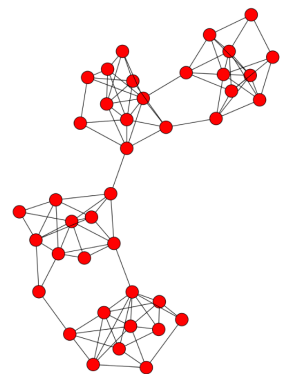
Da questi punti di separazione si calcola il normalized cut e si sceglie quello con costo minore.

Si è sempre considerato il caso in cui si avevano 2 cluster, e se si avessero più cluster?

Il secondo autovettore è principalmente associato alla prima grande separazione del grafo. Allora utilizziamo più autovettori per catturare altre strutture più fine.

Questa è l'idea fondamentale della k-Way Spectral Clustering, dove ogni nodo non è rappresentato da un solo numero ma da molteplici.

Utilizzando ad esempio una coppia, nel nuovo spazio i nodi diventano facilmente separabili e a questo punto si può utilizzare un normale algoritmo di clustering, per esempio il K-Means.



Come possiamo trovare k cluster? La prima soluzione è la **recursive bi-partitioning**, ovvero sempre una divisione in due. Il problema è che si devono eseguire diverse bipartizioni una dopo l'altra. Ci sono molteplici svantaggi:

- inefficiente;
- sensibile alle scelte fatte nelle divisioni precedenti;
- instabile.

Il secondo approccio è il **Multiple Eigenvectors**, che segue direttamente più autovettori contemporaneamente. Se si vogliono k cluster, si costruisce uno spazio usando diversi autovettori. Per esempio:

$$U = \begin{bmatrix} | & | & | \\ x_2 & x_3 & x_4 \\ | & | & | \end{bmatrix}$$

Ogni riga rappresenta un nodo, quindi il nodo i diventa:

$$(x_2[i], x_3[i], x_4[i])$$

Si trasforma il problema originale in un problema di clustering in uno spazio di dimensione più bassa.

La rappresentazione funziona perché gli autovettori sono ortogonali e catture diverse direzioni/strutture del grafo.

Si amplificano le differenze tra i cluster, e anche se si hanno punti difficili da separare, dopo la trasformazione spettrale:

- i punti appartenenti allo stesso cluster tendono a trovarsi vicini;
- i punti appartenenti a cluster diversi tendono a trovarsi lontani.

Sweep

Lo **Sweep Algorithm** è un **algoritmo per trovare cluster/comunità dense in un grafo, partendo da un particolare nodo chiamato seed**(nodo seme).

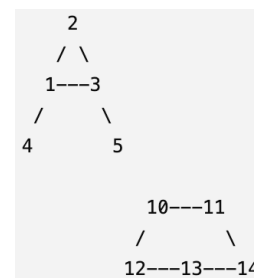
L'idea fondamentale è: si parte da un nodo s, uso il Personalize PageRank per capire quali nodi sono più vicini/importanti rispetto a s, si ordinano e poi si prova progressivamente a costruire un insieme di nodi.

È molto utile quando si hanno grafi enormi, dato che analizzarli per intero sarebbe molto costoso. L'obiettivo dello Sweep è fare una ricerca locale, concentrandosi sui nodi appartenenti alla stessa zona del grafo del nodo seme.

Questo è l'idea alla base del **Local Graph Clustering**.

Per esempio, scelto il nodo s=1 chiamato seed, il primo gruppo è una comunità molto più vicino rispetto al secondo. Si vuole quindi scoprire automaticamente {1,2,3,4,5} senza dover analizzare inutilmente tutta la rete.

La tecnica utilizzata è il Personalized PageRank o PRR, dove il teleport torna sempre verso il nodo s.



Il PRR sarà fortemente concentrato intorno al nodo 1 e ai nodi facilmente raggiungibili da 1. La random walk si muove all'interno della comunità, difficilmente uscendo dalla comunità.

Questo perché il nodo 1 fa parte di una comunità molto densa, e la random walk rimane intrappolata nel cluster.

Inoltre, i nodi del cluster ricevono score PRR elevati; invece, i nodi molto lontani o separati dal cluster ricevono score più bassi. Sulla base degli score viene fatto un ordinamento ed è questo il punto di partenza dello Sweep.

Ma cosa è lo Sweep? La parola sweep significa spazzare e dopo che si sono ordinati i nodi secondo il PRR si costruiscono progressivamente degli insiemi.

A1	= {1}
A2	= {1,3}
A3	= {1,3,2}
A4	= {1,3,2,5}
A5	= {1,3,2,5,4}
A6	= {1,3,2,5,4,10}

Per ogni insieme, si misura quanto è buona la comunità. Come si misura se un cluster è buono? Si utilizza la **conduttanza $\phi(S)$** che misura quanto un insieme S è ben separato dal resto del grafo.

Una forma della definizione è:

$$\phi(S) = \frac{cut(S, \bar{S})}{\min(vol(S), vol(\bar{S}))}$$

dove:

- **$cut(S, \bar{S})$** → numero/peso degli archi che escono da S;
- **$vol(S)$** → somma dei degree dei nodi di S;
- **$\bar{S}$** → V - S.

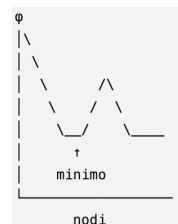
Più bassa è la conduttanza più la comunità è ben separata. Questo perché al numeratore si ha il cut(A, B) che misura gli archi uscenti.

Non basta però semplicemente minimizzare il cut, perché si potrebbe anche ottenere un insieme minuscolo. Infatti, la conduttanza normalizza il cut rispetto alla dimensione/densità del gruppo.

Questo evita di favorire automaticamente gruppi piccolissimi.

La conduttanza sarà ciò che permette di valutare quando fermarsi; infatti, **non si cerca il minimo assoluto nella curva, ma un minimo locale** ossia una zona in cui aggiungere altri nodi fa peggiorare la qualità.

Per avere uno sweep efficiente, non si calcola ogni insieme da zero, ma si prende il gruppo precedente e si aggiunge il nodo successivo.



Per l'aggiornamento del volume si riprende lo stesso concetto:

$$Vol(A_{i+1}) = Vol(A_i) + d_{i+1}$$

dove d_{i+1} è il degree che ha il nodo $i+1$.

L'aggiornamento del cut è un po' più difficile, la formula è:

$$Cut(A_{i+1}) = Cut(A_i) + d_{i+1} - 2\#(archi\ da\ u_{i+1}\ ad\ A_i)$$

Si rimuovono i due archi perché i 2 archi da u_{i+1} verso A_i , non devono essere considerati più come cut perché non più uscenti ma interni.

Si mantiene una tabella hash dei nodi in A_i , in modo tale che se si vuole controllare se u_{i+1} è già dentro A_i è molto più veloce.

Con una hash table si può controllare in un tempo medio $O(1)$.

Il vantaggio è che la curva dello Sweep può essere calcolata in tempo lineare nella dimensione del grafo, ovvero $O(n+m)$. Soprattutto nelle versioni locali dell'algoritmo, la porzione di grafo da analizzare è quella intorno al seed.

PageRank approssimato

Il **PageRank approssimato**, chiamato anche **PageRank-Nibble**, è un algoritmo che **calcola in modo efficiente un Personalized PageRank (PPR) partendo da un nodo seme s** .

Il problema principale è che il PageRank classico può essere costoso su grafi enormi.

L'idea di PageRank-Nibble è: **concentrarsi solo sui nodi che hanno ancora una quantità significativa di PageRank da distribuire**.

Si introducono due vettori:

- **$r \rightarrow$ PageRank già calcolato**, r_u rappresenta la **quantità di PageRank che si è già assegnato al nodo u** .
- **$q \rightarrow$ PageRank residuo**, q_u rappresenta invece **la quantità di PageRank che deve essere ancora distribuita**.

Si utilizza un residuo perché non si vuole calcolare tutto il PageRank contemporaneamente. Si può seguire questo approccio:

- 1) mettere tutto il PageRank nel seed;
- 2) prendere una parte del residuo;
- 3) trasformare in PageRank definitivo r ;
- 4) distribuire il resto ai vicini;
- 5) continuare finché il residuo è sufficientemente piccolo.

Quest'operazione è chiamata **push**.

PageRank-Nibble utilizza una Lazy Random Walk, ossia una variante della Random Walk su un grafo in cui, ad ogni passo il walker ha una certa probabilità di:

- **rimanere fermo sul nodo corrente;**
- **spostarsi verso uno dei nodi vicini.**

La probabilità di rimanere ferma è $\frac{1}{2}$, invece la probabilità di spostarsi è $\frac{1}{2}$. Se u ha degree d_u , quando cammina ogni nodo riceve:

$$\frac{1}{2d_u}$$

della probabilità.

La formula della Lazy Random Walk è:

$$r_u^{(t+1)} = \frac{1}{2}r_u^{(t)} + \frac{1}{2}\sum_{i \rightarrow u} \frac{1}{d_i}r_i^{(t)}$$

Come intuibile, o rimane fermo o riceve probabilità dai vicini.

Inoltre, il PRR utilizza anche β che **controlla quanto spesso la random walk continua a camminare invece di tornare al teleport.**

Con una probabilità di $1 - \beta$, viene trasformato il residuo direttamente in PageRank definitivo.

La parte fondamentale è l'operazione push che prende il residuo e lo distribuisce. Supponendo di avere un nodo u con $q_u = 0.4$ e $\beta = 0.5$, il push prende il residuo e lo distribuisce.

Infatti:

$$(1 - \beta)q_u = 0.5 \cdot 0.4 = 0.2$$

Si trasforma quindi 0.2 di residuo in PageRank definitivo.

Il secondo pezzo rimane nel nodo, quindi si conserva:

$$\frac{1}{2}\beta q_u = \frac{1}{2} \cdot 0.5 \cdot 0.4 = 0.1$$

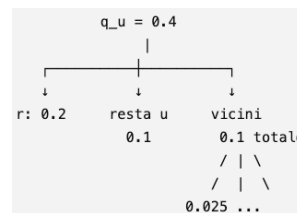
Avremo che:

$$q'_u = 0.1$$

Infine, il terzo pezzo, ossia la rimanente quantità va ai vicini. Se u ha 4 vicini allora $d_u = 4$ e allora:

$$\frac{1}{2}\beta \frac{q_u}{d_u} = \frac{1}{2} \cdot 0.5 \cdot \frac{0.4}{4} = 0.025$$

Ogni nodo vicini riceve quindi 0.025 di questo residuo q .



Nel caso in cui $\frac{q_u}{d_u} \geq \epsilon$, allora il residuo è ancora significativo e si fa il push, altrimenti non si tocca il nodo.

PageRank-Nibble utilizza $\frac{q_u}{d_u}$ come **criterio per decidere se vale la pena continuare.**

Il ciclo prevede che finché esiste un nodo tale che $\frac{q_u}{d_u} \geq \epsilon$ si fa push(u , r , q).

Quando per tutti i nodi non vale la condizione di ciclo allora a quel punto r è una buona approssimazione del Personalized PageRank.

È più veloce del Power Iteration perché il PI aggiorna tutto il vettore ad ogni iterazione. Il PageRank-Nibble invece lavora solo sui i nodi che hanno un residuo significativo.

Quindi se il cluster intorno al seed è piccolo, ci si può concentrare principalmente su quella zona. La complessità è dell'ordine di: $O(\frac{1}{\epsilon(1-\beta)})$, rispetto ad una quantità maggiore per la Power Iteration.

Possiamo collegare il PageRank-Nibble con lo Sweep, sostituendo al PageRank iniziale. Questo rende molto più efficiente l'algoritmo.

Motif-Based Clustering

Un **Motif** è un **piccolo sottografo che rappresenta un pattern di connessioni ricorrente all'interno di una rete**.

Considerando ad esempio 3 nodi A, B, C, se esiste un triangolo tra loro, ossia A – B – C – A, si ha un motif in cui tutti e tre i nodi sono collegati tra loro.

Un motif permette di descrivere come sono collegati più nodi tra loro. Nel clustering classico, due nodi sono fortemente collegati se hanno molti archi tra loro, ma non è sempre sufficiente.

Quindi, potremmo voler costruire cluster non semplicemente sulla base degli archi, ma sulla base della presenza di motif particolari. In base al motif di interesse si può determinare che tipo di struttura della rete si vuole considerare importante.

Prima si aveva la conduttanza classica: $\phi(S)$ che misura quanto un gruppo S è separato dal resto del grafo. Adesso invece si vuole fare la stessa cosa, ma considerando i motif.

Per questo motivo si introduce:

$$\phi_M(S)$$

dove M indica il motif.

L'idea è: **quanto spesso i motif attraversano il confine tra il cluster S e il resto della rete?**

Il volume si ridefinisce, infatti con l'utilizzo dei motif rappresenta il numero di endpoint dei motif che appartengono a S.

$$Vol_M(S) = \# \text{endpoint dei motif in } S$$

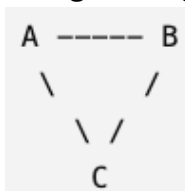
La conduttanza diventa concettualmente:

$$\frac{\# \text{ motif cut}}{Vol_M(S)}$$

Si può definire una **matrice dei pesi** $W^{(M)}$ dove $W_{ij}^{(M)}$ è il **numero di volte in cui (i, j) partecipa a un motif**.

Si conta allora quante volte ogni coppia di nodi partecipa al motif.

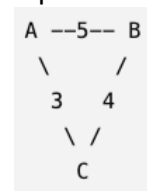
Partenza: grafo originale



Matrice dei pesi

$$W^{(M)} = \begin{pmatrix} 0 & 5 & 3 \\ 5 & 0 & 4 \\ 3 & 4 & 0 \end{pmatrix}$$

Grafo pesato $W^{(M)}$



Si trasforma il grafo in modo tale che si possano utilizzare gli algoritmi studiati. Invece di inventare completamente un nuovo algoritmo si applica il PageRank su questo nuovo grafo pesato $W^{(M)}$.

A questo punto si sceglie il nodo seme s e si calcola il Personalized PageRank approssimato. La differenza è che questa volta non si lavora sul grafo originale, ma sulla matrice $W^{(M)}$.

Il PageRank tenderà a rimanere nelle zone dove sono presenti molte connessioni che partecipano al motif.

Dopo aver ottenuto il PRR, facciamo esattamente lo stesso Sweep studiato precedentemente.

L'utilizzo del motif quindi è una scelta di pre-processing, dove il peso di ogni arco indica quell'arco che partecipa ai motif.

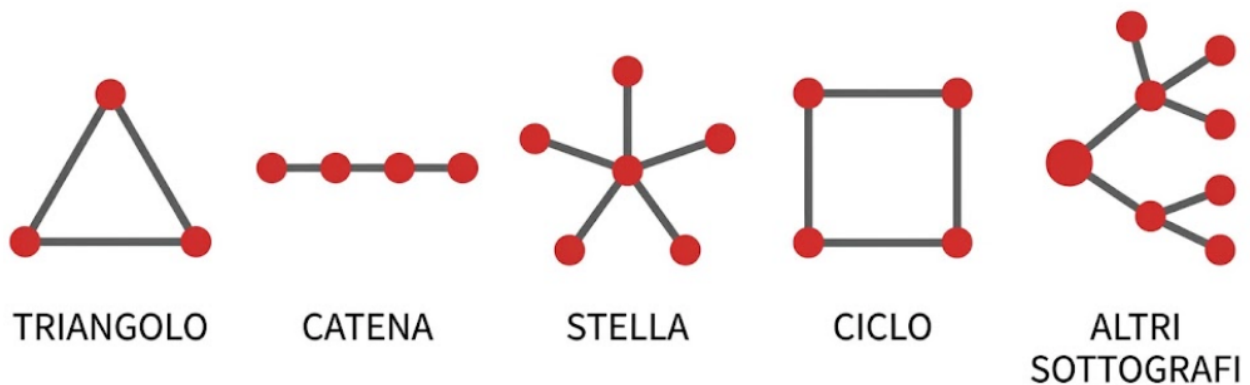
La differenza importante con il clustering classico è che:

- **clustering classico** → considera quali nodi hanno molte connessioni tra loro;
- **motif-based clustering** → quali nodi partecipano frequentemente allo stesso tipo di struttura.

Ad esempio, si può scegliere come motif:

- triangolo;
- catena;
- stella;
- ciclo;
- altri sottografi.

Quindi cambiando motif, si cambia anche il concetto di comunità che si sta cercando.



Ottimizzazione modularità Louvain

Il **Louvain Algorithm** è un **algoritmo greedy per la community detection**. Il suo obiettivo è trovare gruppi di nodi che abbiano:

- molti collegamenti all'interno della comunità;
- pochi collegamenti verso altre comunità.

Per capire se una divisione è buona, utilizza una funzione chiamata: Modularità Q.

Louvain cerca perciò di massimizzare la modularità e a differenza dello Spectral Clustering, non cerca direttamente gli autovettori del grafo.

La modularità, come accennato nei capitoli precedenti, misura quanto la struttura delle connessioni all'interno delle comunità è maggiore di quella che ci aspetteremmo casualmente.

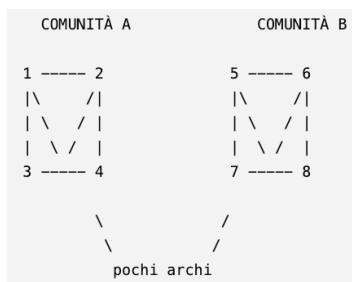
La formula è:

$$Q = \frac{1}{2m} \sum_{i,j} (A_{ij} - \frac{k_i k_j}{2m}) \delta(c_i, c_j)$$

Ricordiamo quindi che:

- A_{ij} è il peso dell'arco tra i e j , e se il grafo non è pesato indica l'esistenza;
- k_i è il grado del nodo i ;
- $2m$ per un grafo non orientato è la somma dei degree di tutti i nodi;
- $\delta(c_i, c_j)$ è una funzione indicatrice che considera le coppie che appartengono alla stessa comunità $\begin{cases} 1 & \text{se } c_i = c_j \\ 0 & \text{altrimenti} \end{cases}$

Il pezzo $(A_{ij} - \frac{k_i k_j}{2m})$ confronta la connessione reale dalla connessione attesa casualmente. La situazione che Louvain vuole trovare è la seguente:



Dentro A molte connessioni.

Dentro B molte connessioni.

Tra A e B poche connessioni.

L'algoritmo è greedy, perché prende decisioni localmente vantaggiose senza cercare di risolvere direttamente il problema globale perfetto.

Louvain propone di spostare nella comunità del vicino i nodi che aumentano la modularità. Quindi cerca continuamente piccoli miglioramenti.

L'algoritmo è composto principalmente da 2 fasi, ossia:

- **Fase 1 → ottimizzazione locale;**
- **Fase 2 → aggregazione delle comunità.**

La prima fase è quindi il partizionamento, dove ogni nodo è una comunità diversa. Poi si prende un nodo e si mette nelle comunità dei suoi vicini e sceglie quella che produce il maggiore aumento della modularità.

Il guadagno di modularità è semplicemente la differenza tra la modularità iniziale e quella finale. Se c'è un aumento (guadagno) allora il cambiamento viene confermato, altrimenti torna allo stato pregresso.

Si considerano solo le comunità dei vicini, perché se un nodo è collegato ai suoi vicini, è ragionevole inserirlo nelle comunità di quei nodi. Non ha senso provare tutte le comunità del grafo.

Questa è una delle ragioni per cui Louvain è molto veloce sui grafi grandi. La fase continua finché esistono spostamenti che aumentano la modularità.

La prima fase continua finché esistono spostamenti che aumentano la modularità. A quel punto si è raggiunto il massimo locale della modularità e quindi per tutti i possibili movimenti, non si può migliorare localmente la soluzione.

Louvain non prova tutte le possibili partizioni, perché anche se ci fosse una partizione molto migliore, per raggiungerla sarebbe necessario prima fare una mossa che temporaneamente diminuisce Q.

Ci sono quattro quantità importanti da tenere in considerazione, ossia:

- $\sum_{in}$ è la somma dei pesi degli archi interni alla comunità C (quanto è connessa internamente la comunità);
- $\sum_{tot}$ è la somma dei pesi di tutti gli archi incidenti ai nodi della comunità (comprende anche gli archi che escono dalla comunità);
- k_i è il degree del nodo i, o la somma dei suoi pesi;
- $k_{i,in}$ è la somma dei pesi degli archi che collegano i alla comunità C.

Si può allora riscrivere il contributo di una comunità in una forma più semplice, ossia:

$$Q(C) = \frac{\sum_{in}}{2m} - \left(\frac{\sum_{tot}}{2m} \right)^2$$

Questa formula dice:

$$\text{contributo delle comunità} = \text{connessioni interne} - \text{connessioni attese}$$

Inoltre, possiamo ottenere la formula di $\Delta Q(i \rightarrow C)$, ossia il cambiamento quando si aggiunge i a C, perché cambiano $\sum_{in}$ e $\sum_{tot}$.

Allora:

$$\Delta Q(i \rightarrow C) = \left[\frac{\sum_{in} + k_{i,in}}{2m} - \left(\frac{\sum_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\sum_{in}}{2m} - \left(\frac{\sum_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right]$$

Ossia $Q_{dopo} - Q_{prima}$.

Ma non basta calcolare $\Delta Q(i \rightarrow C)$ per calcolare il guadagno totale, ma anche la variazione di modularità uscendo da D. La formula è la seguente:

$$\Delta Q(D \rightarrow i \rightarrow C) = \Delta Q(i \rightarrow C) + \Delta Q(D \rightarrow i)$$

La fase 2 è la **Community Aggregation**, cioè il **trasformare ogni comunità in un singolo super-nodo**.

Se tra due comunità esistono diversi archi, vengono sommati. Il peso tra due super-nodi è la somma dei pesi degli archi che collegavano le due comunità nel grafo originale.

Se all'interno di C_1 si avevano molti archi, dopo la contrazione questi diventano un self-loop sul super-nodo C_1 . Quindi il super-nodo conserva l'informazione sulla connettività interna della comunità.

L'idea geniale di Louvain sta proprio nell'aggregazione, infatti, supponendo di partire da 1.000.000 di nodi:

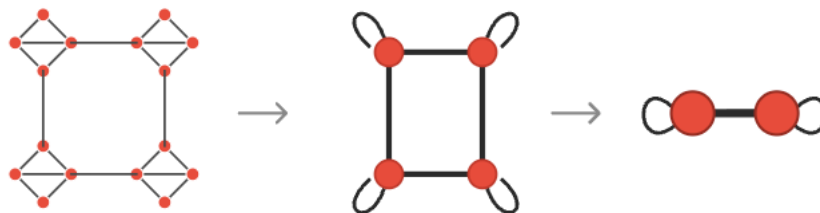
- dopo la prima fase si trovano 10.000 comunità;
- invece di lavorare su milioni di nodi si può lavorare su 10.000 super-nodi;
- possibilmente diventano 500 comunità;
- e ancora possiamo ridurre.

Per questo motivo Louvain è molto efficace sui large networks.

Il grafo diventa più semplice ad ogni iterazione e se non aumenta la modularità: l'algoritmo termina.

Louvain produce allora una struttura gerarchica e questo è utile perché si possono vedere le comunità a diverse scale.

L'ordine dei nodi è importante perché l'output dipende proprio dall'ordine con cui vengono processati i nodi. Questo succede perché Louvain è greedy e raggiunge un massimo locale.



I self-loop nell'esempio indicano appunto che ci sono altre comunità interne.

Trawling

Il **Trawling** è una tecnica per cercare piccole comunità dense all'interno del Web Graph.

Invece di cercare direttamente una comunità, si cerca una particolare struttura molto densa che può rappresentare una comunità.

Questa struttura è un grafo bipartito completo $K_{s,t}$. Nel Web Graph si possono guardare a due gruppi di nodi:

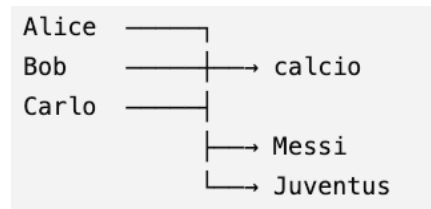
- nodi a sinistra → persone, pagine o utenti;
- nodi a destra → pagine, argomenti o risorse.

Una comunità può quindi avere una struttura del tipo:

molti utenti → molte pagine e argomenti

Ad esempio, in questo caso ogni nodo a sinistra è collegato con i nodi a destra.

Questo è un $K_{3,3}$ e questo è esattamente ciò che cerca il Trawling.



Il fatto che il grafo sia completamente connesso tra i due lati è ciò che rende questa struttura interessante.

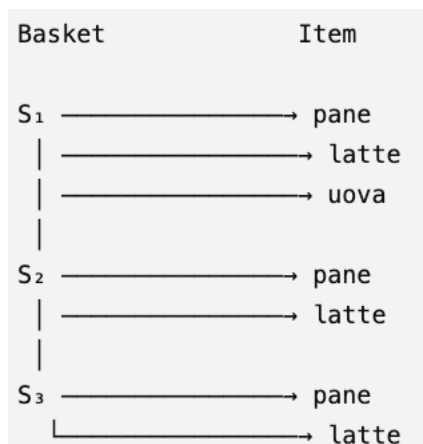
Si utilizza proprio $K_{s,t}$ perché una community molto forte può essere caratterizzata da: molti nodi che condividono gli stessi elementi t.

Il problema di trovare $K_{s,t}$, può essere trasformato nel problema del **Frequent Itemset Mining**, che nasce nella market basket analysis. Si vogliono trovare insiemi di elementi che compaiono frequentemente.

Trasformando questa situazione in un grafo, si ha che:

- ogni basket diventa un nodo a sinistra;
- ogni item diventa un nodo a destra.

L'arco tra le due partizioni esiste quando un basket contiene un determinato item.



Si deve quindi ricordare la corrispondenza fondamentale:

Frequent Itemset	Grafo Bipartito
Itemset T	Insieme Y
dimensione itemset	t
supporto dell'itemset	s
basket che contengono T	nodi X
itemset frequente	$K_{s,t}$

$$\text{frequent itemset} \Leftrightarrow K_{s,t}$$

Il supporto indica quante volte un certo insieme di elementi compare nei diversi baskets.

Si individua un itemset frequente, calcolando il supporto, e si costruisce il grafo bipartito. Quindi, quello che fa il trawling è:

- 1) si prende il web Graph;
- 2) si considera una struttura bipartita X e Y ;
- 3) si cercano le strutture dense $K_{s,t}$, cioè s nodi che condividono gli stessi t nodi;
- 4) si riconduce il problema agli itemset;
- 5) si cercano gli itemset frequenti.

È utile nel Web perché è enorme e cercare tutte le comunità è estremamente costoso.

Il trawling allora sfrutta gli algoritmi di frequent itemset mining. In questo modo si possono utilizzare le tecniche come quelle sviluppate per la market basket analysis.

Apprendimento sui grafi

L'apprendimento sui grafi consiste nell'**applicare tecniche di Machine Learning a dati rappresentati come grafi**. Un grafo è formato da: $G = (V, E)$ dove:

- V = insieme di nodi;
- E = insieme degli archi che collegano i nodi.

I nodi possono rappresentare persone, mentre gli archi le amicizie e ci interessa sapere chi è collegato a chi.

Supponendo di avere un modello $f(x; \theta)$ dove:

- x = input;
- θ = parametri/pesi del modello;
- f = modello;
- y = valore corretto che si vuole ottenere.

Il modello produce una **predizione**:

$$\hat{y} = f(x; \theta)$$

Poi si confronta il valore predetto con il valore corretto usando una **loss function**:

$$\mathcal{L}(\hat{y}, y)$$

L'obiettivo, perciò, è trovare i parametri che rendono l'errore il più piccolo possibile:

$$\theta^* = \arg \min \mathcal{L}(f(x; \theta), y)$$

Questo trova i valori di θ che producono il valore minimo della funzione di perdita, che misura quanto la predizione è distante dal risultato corretto.

Dopo l'addestramento si modificano i pesi, cercando appunto di diminuire progressivamente la loss.

Le **funzioni di attivazione** sono funzioni utilizzate nelle reti neurali per trasformare il risultato di un neurone. Introducendo non linearità e permettendo alla rete di imparare relazioni complesse.

Le funzioni di attivazione presentate sono:

- **sigmoide** ➔ la funzione è $\sigma(z) = \frac{1}{1+e^{-z}}$ che produce sempre un valore compreso tra 0 e 1. Per valori molto positivi tende a 1 e per valori molto negativi tende a 0. È utile perché si può interpretare il risultato come una probabilità, ed è tipicamente utilizzata nell'output della **classificazione binaria**.
- **softmax** ➔ viene utilizzata quando si hanno **più classi** la cui formula è:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

producendo valori in probabilità tra classi.

Si può applicare il Machine Learning ai grafi per svolgere diversi task.

Uno dei task principali è la **Node Classification**, ossia **l'assegnare una classe ai nodi del grafo**. Si utilizzano: caratteristiche dei nodi, collegamenti e caratteristiche dei nodi vicini.

Un altro problema importante è la **predizione degli archi**, capire quindi quali nodi potrebbero essere collegati tra loro.

Il problema delle feature nei grafi è il trasformare caratteristiche in rappresentazioni possibili nel grafo. Per questo motivo si vuole imparare una funzione:

$$f: u \rightarrow \mathbb{R}^d$$

che prende un nodo u e lo trasforma in un vettore di d numeri.

Un **embedding** è una **rappresentazione numerica di un oggetto in uno spazio vettoriale**.

Ad esempio, partendo da un oggetto A si vuole ottenere qualcosa simile: $f(A) = [0.2, 0.8, 0.1, 0.4]$, ovvero un node embedding.

Si vogliono gli embedding, perché **gli algoritmi di Machine Learning lavorano molto bene con vettori numerici. La similarità nello spazio di embedding conserva la similarità nella rete**. Questa è una delle idee fondamentali del graph representation learning.

I tre tipi di embedding sono:

- **node embedding** → si rappresenta ogni nodo con un vettore

$$f: V \rightarrow \mathbb{R}^d$$

e si utilizza principalmente per la node classification.

- **edge embedding** → si rappresenta un arco con un vettore, quindi un arco $(A, B) \in E$ si trasforma in $f: (A, B) \rightarrow \mathbb{R}^d$ e si utilizza principalmente per la link prediction. **Si può combinare l'embedding tra i nodi per ottenere l'edge embedding:**

$$f(A, B) = f(A) \odot f(B)$$

- **graph embedding** → si rappresenta l'intero grafo con un singolo vettore $f(G) \in \mathbb{R}^d$.

Si può rappresentare il grafo attraverso una matrice di adiacenza A , solo che quest'ultima può essere enorme. Si vuole quindi ottenere una rappresentazione compatta e utile al Machine Learning.

L'obiettivo è passare da una rappresentazione molto grande della matrice a una più piccola, e questo si fa attraverso le dimensioni latenti.

È difficile fare Learning sui grafi, dato che le CNN funzionano molto bene sulle immagini dato che hanno una struttura regolare. Un'immagine può essere vista come una matrice di pixel, ergo una struttura regolare.

Il grafo è diverso perché non è una griglia regolare, non ha un ordine naturale e può cambiare nel tempo. Infatti, i grafi possono essere dinamici dato che nuovi utenti possono entrare, altri uscire e nuovi archi possono comparire.

Inoltre, un grafo può contenere diversi tipi di informazione, per esempio:

- utente → utente;
- utente → post;
- utente → hashtag;

quindi il grafo può essere molto più ricco di una semplice struttura.

Si estraggono informazioni utili dalla struttura del grafo, attraverso le seguenti caratteristiche strutturali:

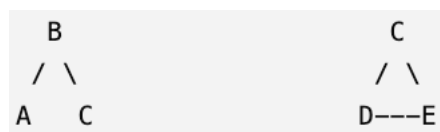
- centralità;
- graphlet;
- graph-kernel;
- Weisfeiler-Lehman;

La **centralità** si è studiata nei capitoli precedenti, e **indica quanto quel nodo è importante secondo una certa definizione**. La centralità può essere vista come:

- **eigenvector centrality** → un nodo è importante se è collegato ad altri nodi importanti;
- **betweenness centrality** → un nodo è importante se molti shortest path tra alte coppie passano attraverso di lui;
- **closeness centrality** → misura quanto un nodo è vicino a tutto il resto mediante shortest path.

Le centralità possono essere utilizzate come feature dei nodi, ma il problema è che queste feature descrivono solo alcuni aspetti.

Per questo motivo vengono introdotti i **graphlet**, molto semplicemente un **piccolo sottografo che descrive la struttura/topologia locale di un grafo**. Supponendo ad esempio di avere:



Attorno ad A si ha una struttura a catena, e attorno a B un triangolo. Anche se avessero lo stesso grado, attraverso il degree non si riuscirebbe a distinguere le due diverse situazioni.

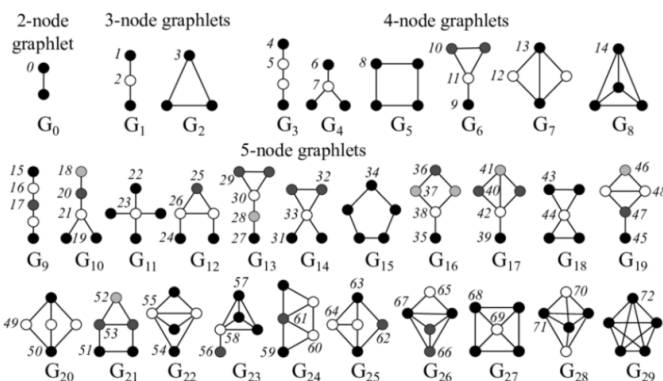
Se prendessimo tutti i possibili graphlet di una certa dimensione e contassimo come compaiono intorno ad un nodo, si otterrebbe un vettore. Questo viene chiamato **Graphlet Degree Vector (GDV).**

Per esempio:

$$GDV(u) = [2,1,0,2]$$

e significa che il nodo u partecipa a determinate configurazioni di graphlet con quelle frequenze. Il vettore è quindi una sorta di impronta digitale strutturale del nodo.

Le **varie posizioni strutturali in cui il nodo può trovarsi all'interno dei graphlet** si chiamano **orbite**. Gli estremi appartengono alla stessa orbita perché sono strutturalmente equivalenti.



Ad esempio, considerando graphlet di dimensione:

2,3,4,5

si ottengono 73 possibili orbite.

Aumentando la dimensione dei graphlet, si possono descrivere strutture locali sempre più complesse.

GDV(u) diventa un feature vector che descrive la topologia del vicinato di u.

Se si volessero confrontare due grafi interi? Quanto sono simili G e G'?

Si potrebbero costruire delle feature esplicite, ma esiste anche un'altra idea:

invece di costruire esplicitamente un vettore di feature, si definisce direttamente una funzione che misura la similarità tra due grafi. Questa funzione è il **Graph Kernel.**

Un kernel è una funzione:

$$K(G, G') \in \mathbb{R}$$

che prende due dati e restituisce un numero che misura la similarità. In base al Kernel utilizzato si hanno valori diversi di similarità.

Si può immaginare che esista una funzione:

$$\phi(G)$$

che **trasforma il grafo in un vettore di feature**. Il Kernel può essere calcolato come:

$$K(G, G') = \phi(G)^T \phi(G')$$

cioè come **prodotto scalare tra i vettori**.

Alcuni Kernel permettono di calcolare la similarità senza dover costruire esplicitamente uno spazio di feature enorme.

La **rappresentazione $\phi(G)$ potrebbe avere moltissime dimensioni e costruirlo esplicitamente potrebbe essere costoso**. Il **Kernel trick** permette di lavorare con il Kernel **senza costruire tutto il vettore**. Si utilizza una funzione Kernel K che evita di costruire il vettore ad alta dimensionalità.

Il primo Kernel presentato è il **Graphlet Kernel**, dove si confrontano due grafi contando quanti piccoli sottografi di ciascun tipo contengono.

Il prodotto scalare è grande quando i due grafi hanno distribuzioni di graphlet simili e/o grandi conteggi nelle stesse strutture.

Si normalizza il vettore, per trasformare conteggi assoluti in una sorta di distribuzione relativa. Si utilizza la formula:

$$h_G = \frac{f_G}{\sum_i f_{G(i)}}$$

Quindi, partendo da conteggi simili:

$$\begin{bmatrix} 3 \\ 4 \\ 0 \\ 1 \end{bmatrix} \quad \text{e} \quad \begin{bmatrix} 9 \\ 12 \\ 0 \\ 3 \end{bmatrix}$$

potremmo dividere per le somme, cioè 8 e 24 e ottenere:

$$h_{G_1} [0.375 \quad 0.5 \quad 0 \quad 0.125]$$

$$h_{G_2} [0.375 \quad 0.5 \quad 0 \quad 0.125]$$

Ergo, si ottiene che anche se in valori assoluti i vettori sono diversi, normalizzando si ha:

$$h_{G_1} = h_{G_2}$$

C'è però un problema legato al conteggio di tutti i graphlet, dato che potrebbe essere estremamente costoso. Infatti, considerando un grafo con n nodi e cercando graphlet di dimensione k , in modo ingenuo si deve considerare circa n^k possibili combinazioni.

Dato che il **Graphlet Kernel può essere computazionalmente pesante**, entra in gioco il **Weisfeiler-Lehman Kernel**. Quest'ultimo infatti cerca di ottenere una rappresentazione strutturale dei grafi in maniera molto più efficiente.

L'idea fondamentale è **guardare iterativamente il vicinato di ogni nodo e aggiornare la sua etichetta in base alle etichette dei suoi vicini. In questo modo l'etichetta di un nodo incorpora progressivamente informazioni provenienti da un vicinato sempre più grande.**

Ad esempio, inizialmente ogni nodo ha colore 1, quindi al primo passo:

- un nodo con due vicini vede: $1, \{1,1\}$;
- un nodo con tre vicini vede: $1, \{1,1,1\}$;
- e così via.

Queste configurazioni vengono poi trasformate tramite una funzione hash:

- $1,11 \rightarrow 3$;
- $1,111 \rightarrow 4$.

Quindi i nodi ottengono nuovi colori. Dopo il primo passo il colore non rappresenta più "sono un nodo" ma "sono un nodo con questo particolare vicinato". Dopo il secondo passo, il colore incorpora anche informazioni sui vicini dei vicini ecc.

Dopo k informazioni si ha informazione fino a K -hop neighborhood.

Si utilizza il **color refinement**, ossia il raffinamento dei colori, che permette di partire con pochi colori e ad ogni iterazione si distinguono sempre meglio i nodi in base alla struttura del loro vicinato.

La funzione HASH utilizzata è:

$$c^{(k+1)}(v) = \text{HASH}(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)})$$

cioè:

- 1) si prende il colore del nodo;
- 2) si guardano i colori dei suoi vicini;
- 3) si mettono insieme queste informazioni;
- 4) si applica una funzione hash;
- 5) si ottiene un nuovo colore.

La HASH non sta facendo Machine Learning, serve a trasformare una configurazione lunga in un identificatore compatto.

Ritornando al confronto tra G e G' , si applica lo stesso processo WL(Weisfeiler-Lehman) e alla fine si ottengono molti colori. Per G e G' , la formula per la rappresentazione è ϕ , che è un vettore contenente i conteggi di ogni colore.

Il vettore diventa quindi una specie di firma strutturale del grafo.

La differenza tra l'utilizzo di Graphlet Kernel e Weisfeiler-Lehman Kernel sta nel confronto:

- Graphlet Kernel → confronta direttamente piccoli sottografi e l'enumerazione è costosa;
- Weisfeiler-Lehman Kernel → si raffinano iterativamente le etichette dei nodi ed è generalmente molto più efficiente.

Importante è ricordare che indipendentemente della tecnica utilizzata, la matrice Kernel deve essere semidefinita positiva. Per ogni vettore z :

$$z^T K z \geq 0$$

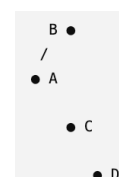
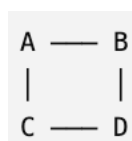
e quindi gli autovalori della matrice kernel sono non negativi.

Node Embedding

Come citato sopra, il problema è che un algoritmo di machine learning lavora con vettori di numeri, per questo motivo si utilizza il node embedding. Il node embedding serve appunto a trasformare un nodo in un vettore.

Il vettore deve conservare informazioni importanti sulla posizione del nodo del grafo. La similarità nella rete deve essere uguale alla similarità nello spazio degli embedding.

Si parte dallo spazio originale a uno a basse dimensioni.



Si vuole uno spazio a basse dimensioni, perché si vuole un vettore piccolo dato che è più pratico lavorarci.

La similarità poi verrà calcolata sui vettori contenenti i valori latenti:

$$\text{sim}(\mathbf{u}, \mathbf{v}) \approx \mathbf{z}_v^T \mathbf{z}_u$$

ed è semplicemente il prodotto scalare tra i due vettori.

Embedding simili daranno come risultato un prodotto scalare grande.

Il **problema del node embedding** può essere visto come tre passaggi:

- 1) **definire l'encoder** $\rightarrow \text{ENC}(v) = \mathbf{z}_v \rightarrow$ cioè una funzione che trasformi il nodo in vettore;
- 2) **definire la similarità** $\rightarrow$ quando due nodi sono simili?
- 3) una volta definite similarità ed encoder si vogliono **trovare embedding che facciano risultare alto il valore di similarità se sono simili o piccolo se sono diversi.**

Inoltre, si deve avere una loss del tipo:

$$\mathcal{L} = (\text{sim}(\mathbf{u}, \mathbf{v}) - \mathbf{z}_u^T \mathbf{z}_v)^2$$

e si cercano gli embedding che minimizzano questa perdita.

L'encoder è semplicemente una tabella di embedding, chiamata embedding lookup. Ogni riga della tabella corrisponde ad un nodo e l'encoder è semplicemente:

$$\text{ENC}(v) = \mathbf{Z}_v$$

$$\mathbf{Z} = \begin{bmatrix} 0.2 & 0.7 & 0.1 \\ 0.3 & 0.6 & 0.2 \\ 0.8 & 0.1 & 0.4 \\ 0.5 & 0.2 & 0.9 \\ 0.1 & 0.4 & 0.6 \end{bmatrix}$$

E per esempio:

$$\mathbf{z}_A = [0.2, 0.7, 0.1]$$

Si chiama **shallow embedding** perché non c'è una rete neurale profonda che calcola l'embedding.

Come facciamo a capire quali nodi devono avere embedding simili? Si utilizzano le random walk per definire la similarità, infatti permettono di definire una nozione di vicinanza più flessibile.

L'idea è: più facilmente si raggiunge v partendo da $u \rightarrow u$ e v sono simili.

Allora noto il concetto di similarità mediante le random walk, si può dire che:

$$P_R(v|u)$$

ossia: la **probabilità di visitare v facendo una random walk che parte da u, seguendo la strategia R.**

Mentre la **probabilità che il modello predice usando l'embedding di u è:**

$$P_R(v|z_u)$$

L'obiettivo è fare in modo che queste due cose siano coerenti, ossia:

$$P_R(v|z_u) \approx P_R(v|u)$$

Considerando $z_u^t z_v^t \approx$ similarità tra u e v, quindi:

- se u e v co-occorrono spesso nelle random walk: $z_u^t z_v^t$ grande;
- se co-occorrono raramente: $z_u^t z_v^t$ piccolo.

La Random Walk si sceglie per tre motivazioni:

- 1) **espressività** → una random walk può catturare sia informazioni locali e sia informazioni di ordine superiore → quindi non si guardano solamente i vicini diretti e si può esplorare una porzione più ampia del grafo;
- 2) **la vicinanza** → se partendo da u si arriva spesso a v allora $u \approx v$;
- 3) **efficienza** → non si vuole necessariamente calcolare esplicitamente la similarità per ogni coppia e si campiona solamente una parte delle relazioni.

Feature Learning non supervisionato

Il **feature learning non supervisionato** è l'imparare automaticamente attraverso il **modello una rappresentazione dei nodi**. È non supervisionato perché non si ha bisogno delle etichette dei nodi per imparare gli embedding.

Si capisce se **due nodi sono simili attraverso la random walk**. Si può definire il vicinato del nodo u attraverso le random walk:

$$N_R(u)$$

Se si fa una random walk si parte da un nodo e si raccolgono i vicini, ma se si fanno più random walk si può ottenere:

$$N_R(A) = \{B, B, C, D, B, C, E, \dots\}$$

La struttura sopraccitata è un **multiset**, ovvero un insieme che ammette le ripetizioni, utile per le random walk che effettuano il conteggio.

Considerando allora:

$$G = (V, E)$$

e si vuole trovare una funzione:

$$f: V \rightarrow \mathbb{R}^d$$

che associa:

$$u \rightarrow z_u$$

Partendo da u , si vuole che l'embedding sappia prevedere quali nodi compariranno nelle random walk. Per questo si massimizza:

$$\max \sum_{u \in V} \log P(N_R(u) | z_u)$$

In altre parole, si trovano gli embedding che rendono più probabile osservare i nodi che effettivamente compaiono nelle random walk.

Normalmente le reti neurali vengono addestrate minimizzando una loss. Allora si cambia il segno e si ottiene:

$$\mathcal{L} = - \sum_{u \in V} \sum_{v \in N_R(u)} \log P(v | z_u)$$

Minimizzare questa quantità significa rendere $P(v | z_u)$ più grande.

Ma come si definisce $P(v | z_u)$? Si utilizza la softmax, ovvero:

$$P(v | z_u) = \frac{e^{(z_u^T z_v)}}{\sum_{n \in V} e^{(z_u^T z_n)}}$$

Il problema è il denominatore, dove si devono considerare tutti i nodi del grafo. **È computazionalmente costosissimo confrontare un nodo preso in considerazione u con milioni di nodi.**

La soluzione è il **Negative Sampling**, ossia invece di confrontare u con tutti i nodi del grafo, si scelgono solo alcuni nodi.

L'idea principale è:

invece di confrontare una coppia positiva con tutti i nodi del grafo, la si confronta solo con pochi nodi scelti casualmente.

Supponendo che si ha la coppia positiva (A, B) , significa che sono stati osservati insieme nelle random walk. Questa è una coppia positiva. Poi si scelgono casualmente: C, D, E come nodi negativi.

Il modello allora deve imparare: A simile a B e A diverso da C, D, E .

Se una coppia non è mai stata osservata nelle random walk, probabilmente non rappresenta una relazione importante. L'esempio negativo è appunto la coppia che non è mai stata osservata.

In questo modo il confronto avviene tra **1 esempio positivo** e **k esempi negativi**, anziché 1 positivo + |V| possibili nodi.

Quindi la softmax viene sostituita da un problema di classificazione binaria e la funzione sigmoide è:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

e trasforma un numero in: (0,1).

La probabilità di avere **coppia positiva** è indicata da:

$$\sigma(z_u^T z_v) \rightarrow 1 \quad \text{se} \quad z_u^T z_v \rightarrow \text{grande}$$

Invece, la probabilità di avere **coppia negativa**:

$$\sigma(z_u^T z_v) \rightarrow 0 \quad \text{se} \quad z_u^T z_v \rightarrow \text{piccolo/negativo}$$

Quindi il modello impara a separare coppie positive e negative.

Attraverso il negative sampling si approssima la softmax, perché da alcuni nodi scelti casualmente si generalizza per tutti i nodi.

Quanti nodi negativi si scelgono?

Il **parametro k** è scelto in modo arbitrario, in generale **maggiore è il valore k e più robusta è la stima**. Si potrebbe scegliere la robustezza ma si paga computazionalmente, durante il training.

I nodi negativi non si scelgono necessariamente tutti con la stessa probabilità.

Si può dare maggiore probabilità ai nodi con degree elevato, dato che sono quelli più rappresentativi della distribuzione della rete.

La funzione di loss viene minimizzata attraverso lo studio del gradiente. Si va nel verso opposto del gradiente, dato che lo stesso indica la direzione di maggiore salita.

Si può calcolare attraverso:

- **discesa del gradiente(GD)** → si considera tutto il dataset e si calcola il gradiente complessivo;
- **discesa del gradiente stocastica(SGD)** → prende un esempio alla volta, calcola il gradiente e aggiorna gli embedding → soluzione molto più pratica su dataset enormi.

Node2Vec

Finora si è considerata un'ottima soluzione l'utilizzo delle random walk, perché verificare quali sono i nodi vicini. Ma come si scelgono le random walk?

Una random walk può esplorare il grafo in modi diversi e Node2Vec nasce per risolvere questo problema.

Il problema delle random walk è che si può decidere di preferire:

- rimanere vicino ad A(nodo seed);
- allontanarsi progressivamente da A.

Node2Vec introduce due comportamenti estremi:

- **BFS Breadth First Search** → esplora intorno al nodo di partenza → la random walk tende a rimanere nella zona di u e quindi **cattura la vicinanza locale**;
- **DFS Depth First Search** → esplora una parte più profonda del grafo → la random walk tende ad andare sempre più lontana e può **catturare la somiglianza strutturale anche tra nodi lontani**.

Node2Vec non pone però una scelta che esclude l'altra, infatti si può scegliere una random walk:

- molto locale;
- molto esplorativa;
- oppure intermedia.

Si **possono allora interpolare i due comportamenti** e questo è il **trade-off tra vicinato locale e globale**.

Vengono introdotti due parametri che controllano come viene scelta la prossima mossa. I parametri sono:

- **p(ritorno)** controlla la **probabilità di tornare indietro al nodo precedente**. I valori assunti influenzano il comportamento:
 - se p è basso è relativamente facile tornare indietro;
 - se p è alto è meno probabile tornare indietro.
- **q(esplorazione)** controlla quanto la random walk tende a rimanere nell'attuale zona o allontanarsi. I valori possono essere:
 - se q è alto tendenza maggiore a rimanere nella zona locale → comportamento BFS;
 - se q è basso tende maggiormente ad andare verso nodi più lontani → comportamento più DFS.

Perciò, **Node2Vec è una Biased Random Walk**, perché **non è completamente casuale**. Non tutti i vicini hanno la stessa probabilità di essere scelti e p e q sono utili a decidere come si vuole esplorare il grafo.

Inoltre Node2Vec è una random walk di secondo ordine perché per decidere il prossimo passo serve conoscere:

- w ;
- s_1 .

Dopo essersi mosso, il walker può compiere tre possibili passi:

- 1) si torna indietro da dove si è arrivati;
- 2) si rimane nella stessa zona, andando in un nodo alla stessa distanza/livello di quello già raggiunto → si segue l'approccio BFS;
- 3) si va verso nodi più lontani → si segue l'approccio DFS;

Node2Vec assegna dei pesi alle possibili transizioni, che non sono probabilità vere e proprie perché non sono normalizzate. Si ha:

$$\frac{1}{q}, 1, \frac{1}{p}$$

pesi non normalizzati, che dopo che si sono assegnati, vengono normalizzati per ottenere le probabilità di transizione → vengono divisi per la somma.

L'algoritmo completo è abbastanza semplice, infatti:

- 1) si calcolano le probabilità p e q , e si decide quanto è probabile ogni transizione;
- 2) partendo da ogni nodo u si fa random walk di lunghezza l ;
- 3) si costruisce il vicinato, cioè i nodi che co-occorrono con u ;
- 4) si addestrano gli embedding, utilizzando la funzione vista prima

$$\max \sum_{u \in V} \log P(N_R(u) | z_u)$$

e si ottimizza con SGD;

- 5) si ottengono gli embedding per ogni nodo tale che $u \rightarrow z_u$.

e questo è come funziona il node embedding.

Graph Embedding

Finora si sono rappresentati solo i nodi, ma cosa succede se si vuole rappresentare l'intero grafo? **La rappresentazione dell'intero grafo si chiama Graph Embedding.** Per esempio:

$$G_1 \rightarrow zG_1$$

L'approccio più semplice(1) consiste nel fare prima il node embedding e poi l'aggregazione di tutti i vettori dei nodi. Se avessimo z_A, z_B, z_C otterremmo: $z_G = z_A + z_B + z_C$ oppure la media $z_G = \frac{z_A + z_B + z_C}{3}$.

La somma è semplice, ma perde alcune informazioni sulla struttura, dato che non è molto sofisticato come metodo.

Il secondo approccio(2) è il nodo virtuale, ossia che **si aggiunge un nodo speciale che è collegato a tutti i nodi del grafo.** In questo modo, durante le random walk, il nodo G vede direttamente tutta la struttura del grafo.

L'embedding del nodo virtuale G viene quindi influenzato dagli altri nodi e dalle loro relazioni. G funziona come un rappresentante dell'intero grafo e si prende proprio il suo embedding z_G come rappresentazione dell'intero grafo.

Il terzo approccio(3) è l'Anonymous Walk, dove non interessa il nome dei nodi ma solamente la **struttura della passeggiata**. Trascurando il nome dei nodi si potrebbero trovare strutture identiche.

È utile l'Anonymous Walk perché permette di descrivere la struttura del grafo senza dipendere dai nomi dei nodi.

Le random walk possono produrre gli stessi pattern anonimi e quindi l'embedding può catturare: la struttura anziché l'identità dei nodi.

Con una anonymous walk di lunghezza k , ad ogni posizione si può:

- visitare un nodo già visto;
- introdurre un nuovo nodo.

Quindi il numero di possibili pattern cresce rapidamente, in modo esponenziale.

Si potrebbero fare tante random walk nel grafo e si potrebbe vedere quali anonymous walk compaiono e con quale frequenza.

Invece dei conteggi si potrebbero utilizzare le probabilità e questo vettore diventa l'embedding del grafo.

È interessante perché due grafi strutturalmente simili tenderanno ad avere distribuzioni simili. Si possono utilizzare questi vettori per:

- classificare grafi;
- confrontare grafi;
- fare clustering;
- cercare grafi simili.

Non si vogliono però enumerare tutte le possibili anonymous walk, perché è molto costoso, allora si fa sampling.

Invece di considerare tutte le possibilità, si fanno tante random walk casuali e si stima la distribuzione. **Per stabilire quante random walk si devono fare, si utilizza la formula:**

$$m = \left\lceil \frac{2}{\epsilon^2} (\log(2^n - 2) - \log(\delta)) \right\rceil$$

dove:

- ϵ è l'errore massimo che si accetta;
- δ è la probabilità che la stima abbia un errore maggiore di ϵ ;
- n è il numero totale di anonymous walk possibili per quella lunghezza l.

Un altro modo per fare embedding del grafo è l'embedding gerarchico. L'idea fondamentale è: invece di guardare le anonymous walk, **riduciamo progressivamente il grafo, aggregando nodi o regioni.**

È simile al concetto di pooling nelle CNN:

grafo grande → grafo piccolo → grafo ancora più piccolo

Alla fine, si ottiene una rappresentazione compatta: $G \rightarrow z_G$ che si può utilizzare per la classificazione.

Con node embedding e graph embedding si può poi applicare: node classification, link prediction e clustering.

DeepWalk

La **DeepWalk** è una **tecnica che vuole mettere vicini nello spazio di embedding i nodi che co-occorrono nelle random-walk**. Quest'idea può essere vista matematicamente come una fattorizzazione di una matrice, e non necessariamente della matrice di adiacenza.

DeepWalk costruisce implicitamente una matrice che contiene informazioni sulle co-occorrenze nelle random walk.

La Deep Walk è simile alla fattorizzazione di una matrice di similarità.

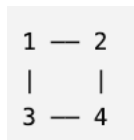
$$\log \left(vol(G) \left(\frac{1}{T} \sum_{r=1}^T (D^{-1}A)^r \right) D^{-1} \right) - \log b$$

Si ottiene una matrice che contiene informazioni su: quanto è probabile che un nodo raggiunga/visiti un altro nodo attraverso random walk di varie lunghezze.

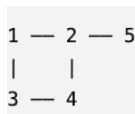
Una volta ottenuto z_G si può utilizzare per fare predizioni. Inoltre, si possono confrontare due grafi attraverso: $z_{G_1}^T z_{G_2}$.

La **limitazione è che molti metodi visti sono trasduttivi**, significa **che embedding vengono imparati per i nodi che si sono visti durante il training**.

Ad esempio, se il training venisse fatto su questi quattro nodi:



e si aggiungerebbe un nuovo nodo:



Un metodo trasduttivo classico non sa necessariamente calcolarlo.

La seconda limitazione è che non catturano bene la similarità strutturale. Infatti, A e X potrebbero avere lo stesso ruolo strutturale, ma un metodo basato sulla vicinanza potrebbe non considerarli simili.

Essere strutturalmente uguali non significa necessariamente essere vicini nella rete.

Infine, **non sfruttano le informazioni/feature dei nodi**, infatti i metodi basati solamente sulle random walk non struttano naturalmente queste informazioni.